



Natural Language Processing



BITS Pilani
Pilani Campus

Dr.Vijayalakshmi Anand
BITS-Pilani

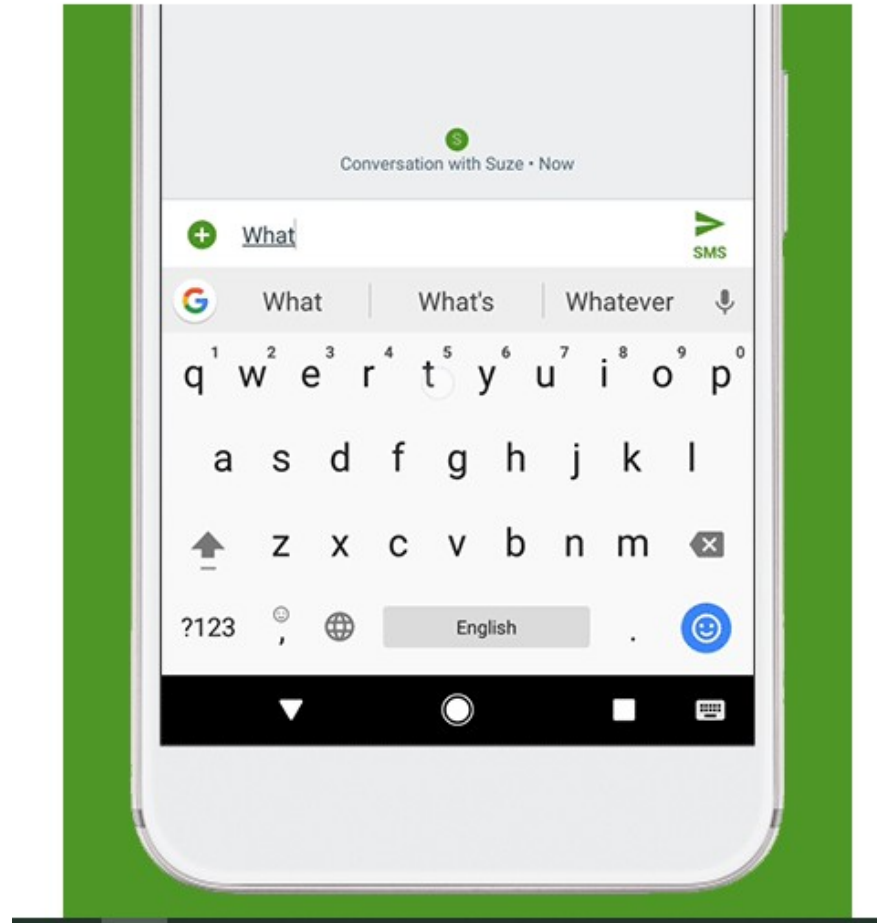
N gram Language model

- Human word Prediction
- Language Models
 - What is language model
 - Why language models
- N-gram language models
 - Uni-gram , bi-gram and N-gram
- Evaluation of Language models
 - Perplexity
- Smoothing
 - Laplace smoothing
 - Interpolation and Back off

Next word Prediction

- We may have ability to predict future words in an utterance .
- How?
 - ❖ Based on domain knowledge
red blood
 - ❖ Based on syntactic knowledge
the <adj/noun>
 - ❖ Based on Lexical knowledge
baked <potato>

Example



Language modelling



A model that computes either of these:

Probability of a sentence ($P(W)$) or Probability of an upcoming word ($P(w_n | w_1, w_2 \dots w_{n-1})$) is called a **language model**.

Simply we can say that

A language model learns to predict the probability of a sequence of words.

Language Models



1. Machine Translation:

Machine translation system is used to translate one language to another language .For example Chinese to English or German to English etc.

I eat lunch

我在吃”



I am eating

Eating am I

Me am eating

3.Speech Recognition

Speech recognition is the ability of a machine or program to identify words and phrases in spoken language and convert them to a machine-readable format.



Sentiment analysis

Sentiment analysis using a language model involves determining the emotional tone or polarity (e.g., positive, negative, or neutral) of a given piece of text.

- **Use Case in Real Life**
- **E-commerce:** Analyzing customer reviews for feedback.
- **Social Media Monitoring:** Understanding public sentiment about a brand.
- **Healthcare:** Detecting emotional distress in patients' messages.

How to build a language model



- Recall the definition of conditional probabilities

$$p(B|A) = P(A,B)/P(A) \text{ Rewriting: } P(A,B) = P(A)P(B|A)$$

- More variables:

$$P(A,B,C,D) = P(A)P(B|A)P(C|A,B)P(D|A,B,C)$$

- The **Chain Rule** in General

$$P(x_1, x_2, x_3, \dots, x_n) = P(x_1)P(x_2|x_1)P(x_3|x_1, x_2) \dots P(x_n|x_{n-1}, x_{n-2}, \dots, x_1)$$

The Chain Rule applied to compute joint probability of words in sentence



-)

- For ex:

$P(\text{"its water is so transparent"}) =$

$P(\text{its}) \times P(\text{water} | \text{its}) \times P(\text{is} | \text{its water})$

$\times P(\text{so} | \text{its water is}) \times P(\text{transparent} | \text{its water is so})$

- **$P(\text{most biologists and specialist believe that in fact the mythical unicorn horns derived from the narwhal})$**

Markov Assumption



- Simplifying assumption:

limit history of fixed number of words $N-1$



Andrei Markov

$P(\text{the l its water is so transparent that}) \approx P(\text{the l that})$



or

$P(\text{the l its water is so transparent that}) \approx P(\text{the l transparent that})$



Markov Assumption

$$P(w_1 w_2 \dots w_n) \approx \prod_i P(w_i \mid w_{i-k} \dots w_{i-1})$$

In other words, we approximate each component in the product

$$P(w_i \mid w_1 w_2 \dots w_{i-1}) \approx P(w_i \mid w_{i-k} \dots w_{i-1})$$

N-gram Language models

- N gram is a sequence of tokens(words)
- Unigram language model(N=1)

$$P(w_1 w_2 \dots w_n) \approx \prod_i P(w_i)$$

Example:

**$P(\text{I want to eat Chinese food}) \approx P(\text{I})P(\text{want})$
 $P(\text{to})P(\text{eat})P(\text{Chinese})P(\text{food})$**

Bigram model



N=2

$$P(w_i | w_1 w_2 \dots w_{i-1}) \approx P(w_i | w_{i-1})$$

Example:

$P(\text{I want to eat Chinese food}) \approx P(\text{I} | \text{<start>}) P(\text{want} | \text{I}) P(\text{to} | \text{want})$
 $P(\text{eat} | \text{to}) P(\text{Chinese} | \text{eat}) P(\text{food} | \text{Chinese})$
 $P(\text{<end>} | \text{food})$

N-gram models

- We can extend to trigrams, 4-grams, 5-grams

Advantages

- no human supervision, easy to extend to more data, allows querying about open-class relations,

Disadvantage:

- In general this is an insufficient model of language
 - because language has **long-distance dependencies**:
“The computer(s) which I had just put into the machine room on the fifth floor is (are) crashing.”

Estimating bigram probabilities

- Estimating the probability as the relative frequency is the *maximum likelihood estimate* (or *MLE*), because this value makes the observed data maximally likely
- The Maximum Likelihood Estimate

$$P(w_i | w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}$$

$$P(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

An example

$$P(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like green eggs and ham </s>

$$P(\text{I} | \text{<s>}) = \frac{2}{3} = .67$$

$$P(\text{Sam} | \text{<s>}) = \frac{1}{3} = .33$$

$$P(\text{am} | \text{I}) = \frac{2}{3} = .67$$

$$P(\text{</s>} | \text{Sam}) = \frac{1}{2} = 0.5$$

$$P(\text{Sam} | \text{am}) = \frac{1}{2} = .5$$

$$P(\text{do} | \text{I}) = \frac{1}{3} = .33$$

Session contents



- Skip Gram end-to-end example
- CBOW
- Glove

Skip gram example



Step - 1 The product is really good The product is wonderful The product is awful

Step - 2 Window Size 1

Step - 3 Get one hot encoding for each word

Contd..



1	The	1	0	0	0	0	0	0
2	product	0	1	0	0	0	0	0
3	is	0	0	1	0	0	0	0
4	really	0	0	0	1	0	0	0
5	wonderful	0	0	0	0	1	0	0
6	good	0	0	0	0	0	1	0
7	awful	0	0	0	0	0	0	1

Step - 1

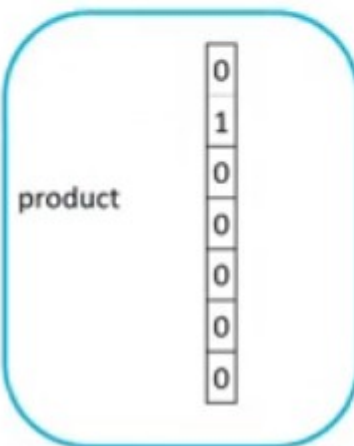
The product is really good The product is wonderful The product is awful

Step - 4

Input Words	Target Word
product	The
product	is
is	product
is	really

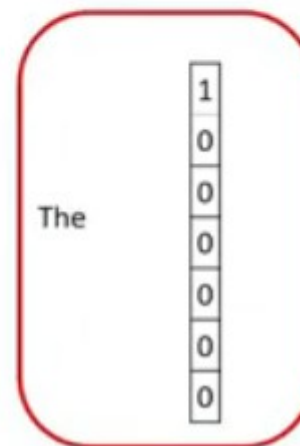
One Hot Encoding

1	the	1	0	0	0	0	0	0
2	product	0	1	0	0	0	0	0
3	is	0	0	1	0	0	0	0
4	really	0	0	0	1	0	0	0
5	wonderful	0	0	0	0	1	0	0
6	good	0	0	0	0	0	1	0
7	awful	0	0	0	0	0	0	1

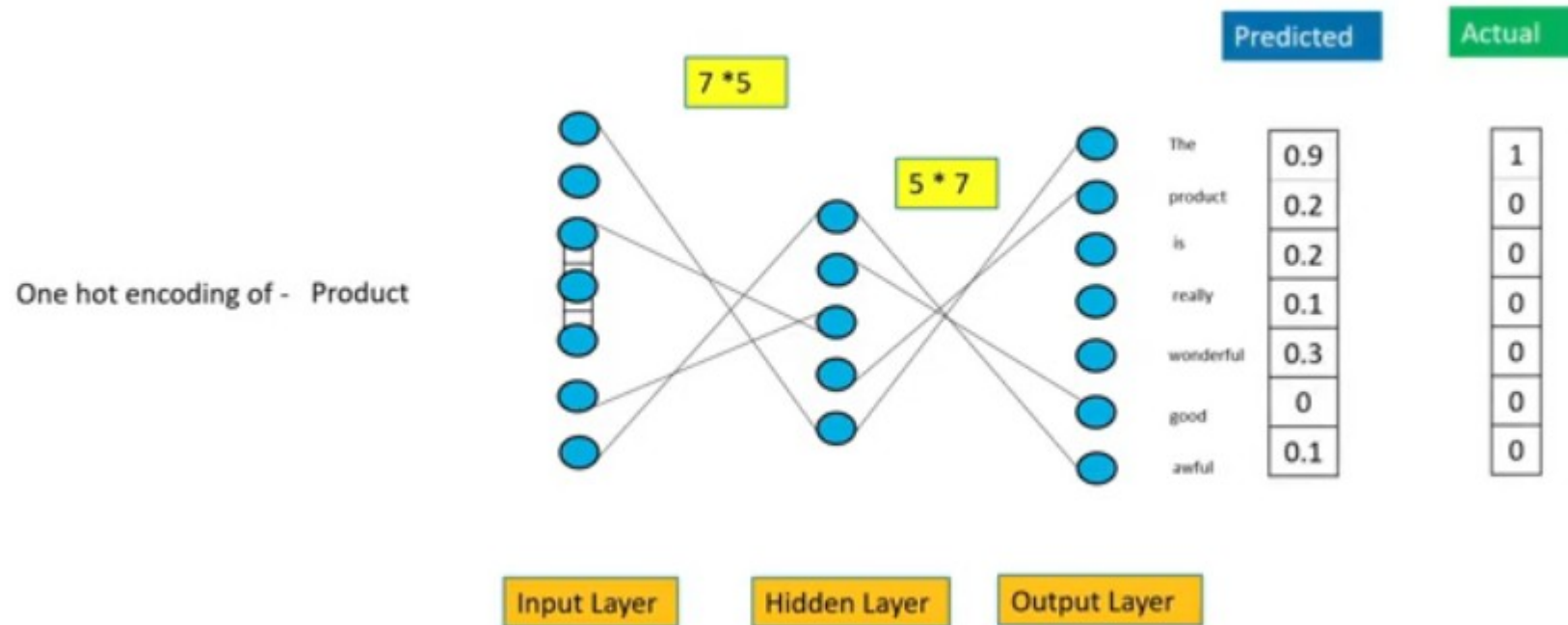


Training Data

Context Words	Target Word
The,is	Product
product,really	is
Is,good	really
Good,product	the



Contd..



Log Loss function



$$\begin{aligned} L_{CE} &= -\log \left[P(+|w, c_{pos}) \prod_{i=1}^k P(-|w, c_{neg_i}) \right] \\ &= - \left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log P(-|w, c_{neg_i}) \right] \\ &= - \left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log (1 - P(+|w, c_{neg_i})) \right] \\ &= - \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right] \end{aligned}$$

Learning the classifier

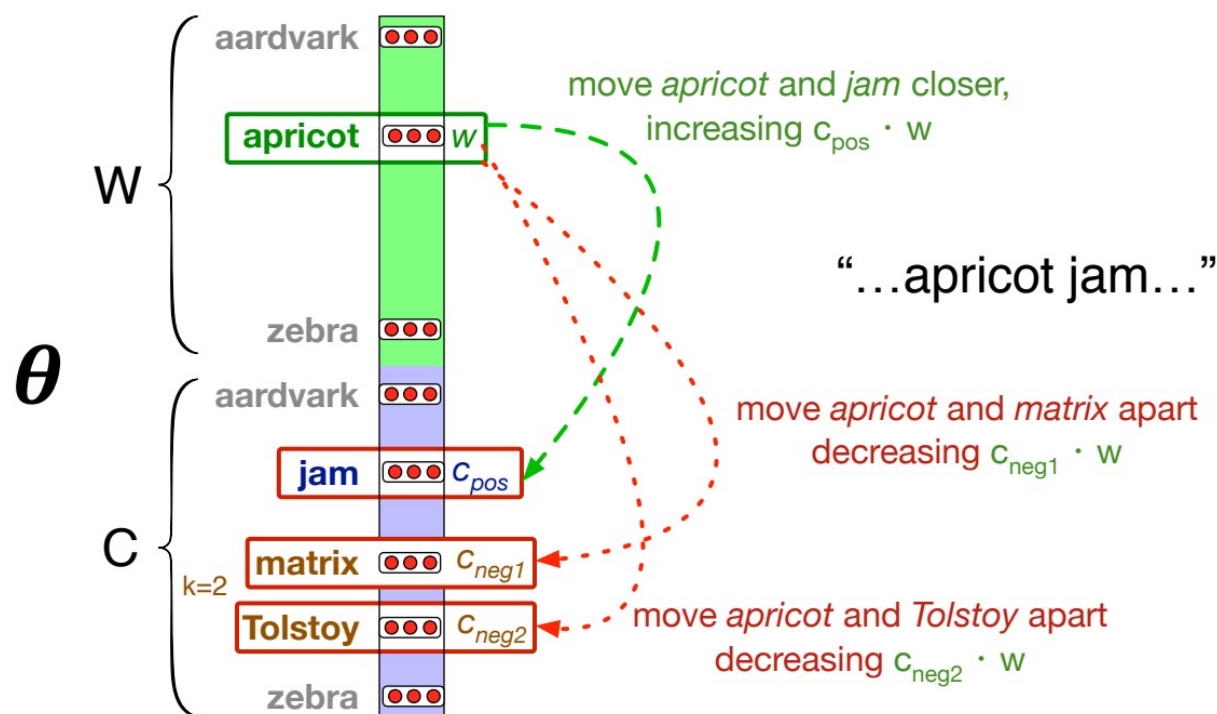
How to learn?

- Stochastic gradient descent!

We'll adjust the word weights to

- make the positive pairs more likely
- and the negative pairs less likely,
- over the entire training set.

Intuition of one step of gradient descent



- Tries to shift embeddings
- So the target embeddings (here for apricot) are closer to (have a higher dot product with) context embeddings for nearby words (here jam)
- And further from (lower dot product with) context embeddings for noise words that don't occur nearby (here Tolstoy and matrix).

Reminder: gradient descent

- At each step
 - Direction: We move in the reverse direction from the gradient of the loss function
 - Magnitude: we move the value of this gradient $\frac{d}{dw} L(f(x; w), y)$ weighted by a **learning rate** η
 - Higher learning rate means move w faster

$$w^{t+1} = w^t - \eta \frac{d}{dw} L(f(x; w), y)$$

The derivatives of the loss function

$$L_{CE} = - \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right]$$

$$\frac{\partial L_{CE}}{\partial c_{pos}} = [\sigma(c_{pos} \cdot w) - 1]w$$

$$\frac{\partial L_{CE}}{\partial c_{neg}} = [\sigma(c_{neg} \cdot w)]w$$

$$\frac{\partial L_{CE}}{\partial w} = [\sigma(c_{pos} \cdot w) - 1]c_{pos} + \sum_{i=1}^k [\sigma(c_{neg_i} \cdot w)]c_{neg_i}$$

Update equation in SGD

Start with randomly initialized C and W matrices, then incrementally do updates

$$c_{pos}^{J+1} = c_{pos}^J - \eta [\sigma(c_{pos}^J \cdot w^J) - 1] w^J$$

$$c_{neg}^{J+1} = c_{neg}^J - \eta [\sigma(c_{neg}^J \cdot w^J)] w^J$$

$$w^{J+1} = w^J - \eta \left[[\sigma(c_{pos}^J \cdot w^J) - 1] c_{pos} + \sum_{i=1}^k [\sigma(c_{neg_i}^J \cdot w^J)] c_{neg_i} \right]$$

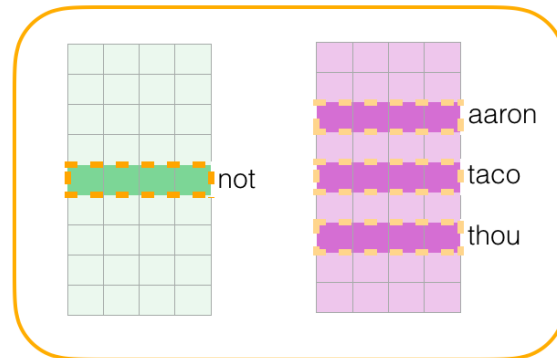
Skip-gram training example



input word	output word	target	input • output	sigmoid()	Error
not	thou	1	0.2	0.55	0.45
not	aaron	0	-1.11	0.25	-0.25
not	taco	0	0.74	0.68	-0.68

$$P(+|w, c) = \sigma(c \cdot w) = \frac{1}{1 + \exp(-c \cdot w)}$$

$$P(-|w, c) = 1 - P(+|w, c) \\ = \sigma(-c \cdot w) = \frac{1}{1 + \exp(c \cdot w)}$$



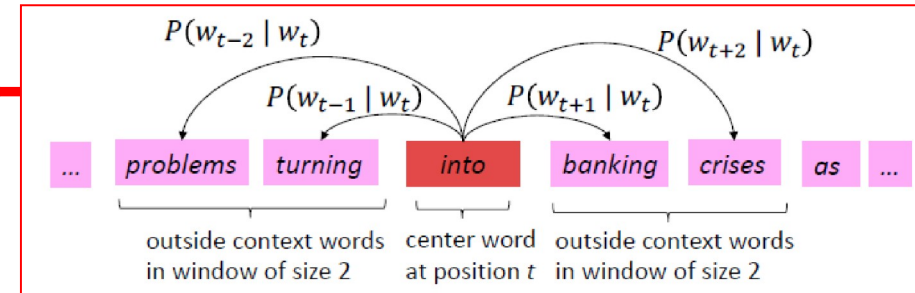
Update
Model
Parameters

<https://jalammr.github.io/illustrated-word2vec/>

How does negative sampling work



- skip-gram with negative sampling (SGNS)
- $N = 2$, $C=4$ (Here C is the context window)



Thou shalt not make a machine in the likeness of a human mind

thou	shalt	not	make	a	machine	in	the	...
thou	shalt	not	make	a	machine	in	the	...
thou	shalt	not	make	a	machine	in	the	...
thou	shalt	not	make	a	machine	in	the	...
thou	shalt	not	make	a	machine	in	the	...

input word	target word
not	thou
not	shalt
not	make
not	a
make	shalt
make	not
make	a
make	machine
a	not
a	make
a	machine
a	in
machine	make
machine	a
machine	in
machine	the
in	a
in	machine
in	the
in	likeness

Source Credit : <https://jalammar.github.io/illustrated-word2vec/>

Contd..

- skip-gram with negative sampling (**SGNS**)
- $N = 2$, $C=4$ (Here C is the context window)

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

input word	target word
not	thou
not	shalt
not	make
not	a
make	shalt
make	not
make	a
make	machine

input word	output word	target
not	thou	1
not	shalt	1
not	make	1
not	a	1
make	shalt	1
make	not	1
make	a	1
make	machine	1

input word	output word	target
not	thou	1
not		0
not		0
not	shalt	1
not	make	1

➤ Negative examples

CBOW [Continuous bag of words]



What is CBOW?

-predict a target word given the context words in a sentence

e.g The product is really good

Working of CBOW with example



Corpus

The product is really good

The product is wonderful

The product is awful

Step1:

combine the sentences

The product is really good The product is wonderful The product is awful

Step2:

Select window size

Contd..



Contd..



Step3: Get one hot encoding for each word

1	The	1	0	0	0	0	0	0
2	product	0	1	0	0	0	0	0
3	is	0	0	1	0	0	0	0
4	really	0	0	0	1	0	0	0
5	wonderful	0	0	0	0	1	0	0
6	good	0	0	0	0	0	1	0
7	awful	0	0	0	0	0	0	1

Step4: Training data

Context Words	Target Word
The,is	Product
product,really	is
Is,good	really
Good,product	the

Contd..

innovate

achieve

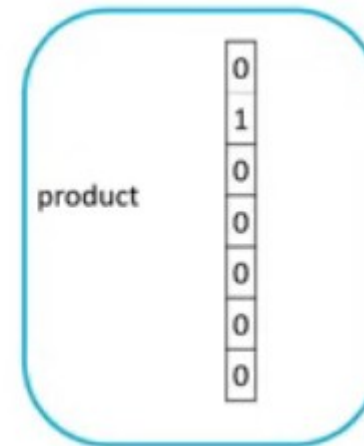
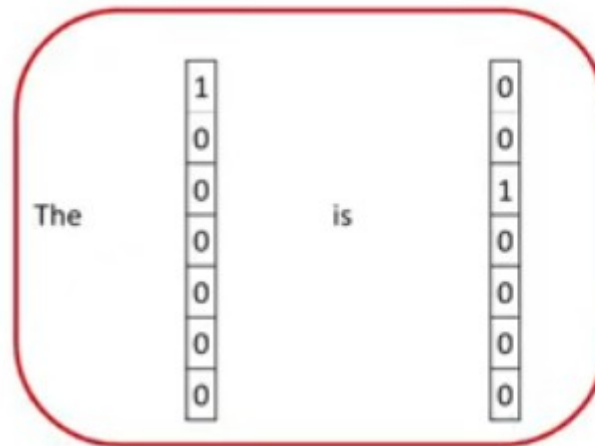
lead

One Hot Encoding

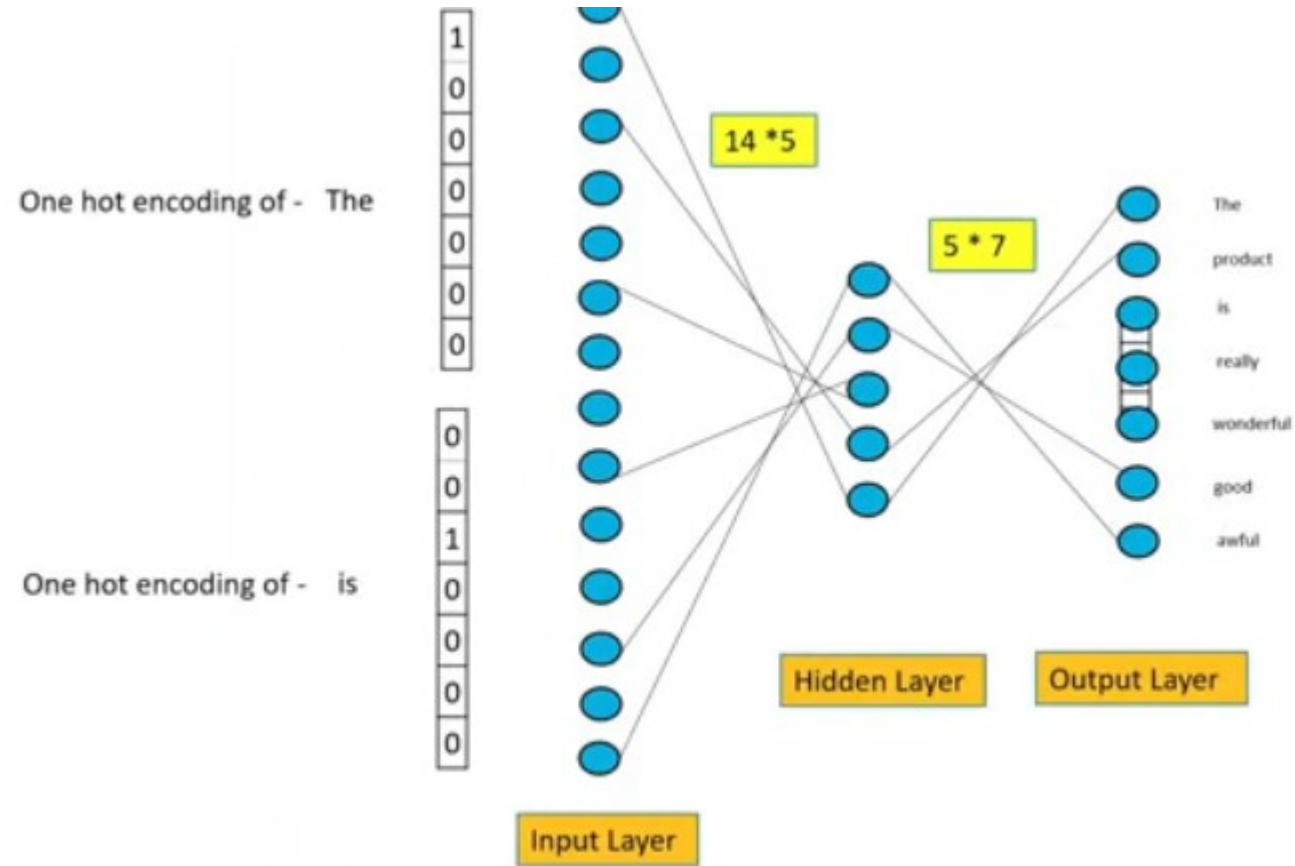
1	The	1	0	0	0	0	0	0
2	product	0	1	0	0	0	0	0
3	is	0	0	1	0	0	0	0
4	really	0	0	0	1	0	0	0
5	wonderful	0	0	0	0	1	0	0
6	good	0	0	0	0	0	1	0
7	awful	0	0	0	0	0	0	1

Training Data

Context Words	Target Word
The,is	product
product,really	is
Is,good	really
Good,product	the



Contd..

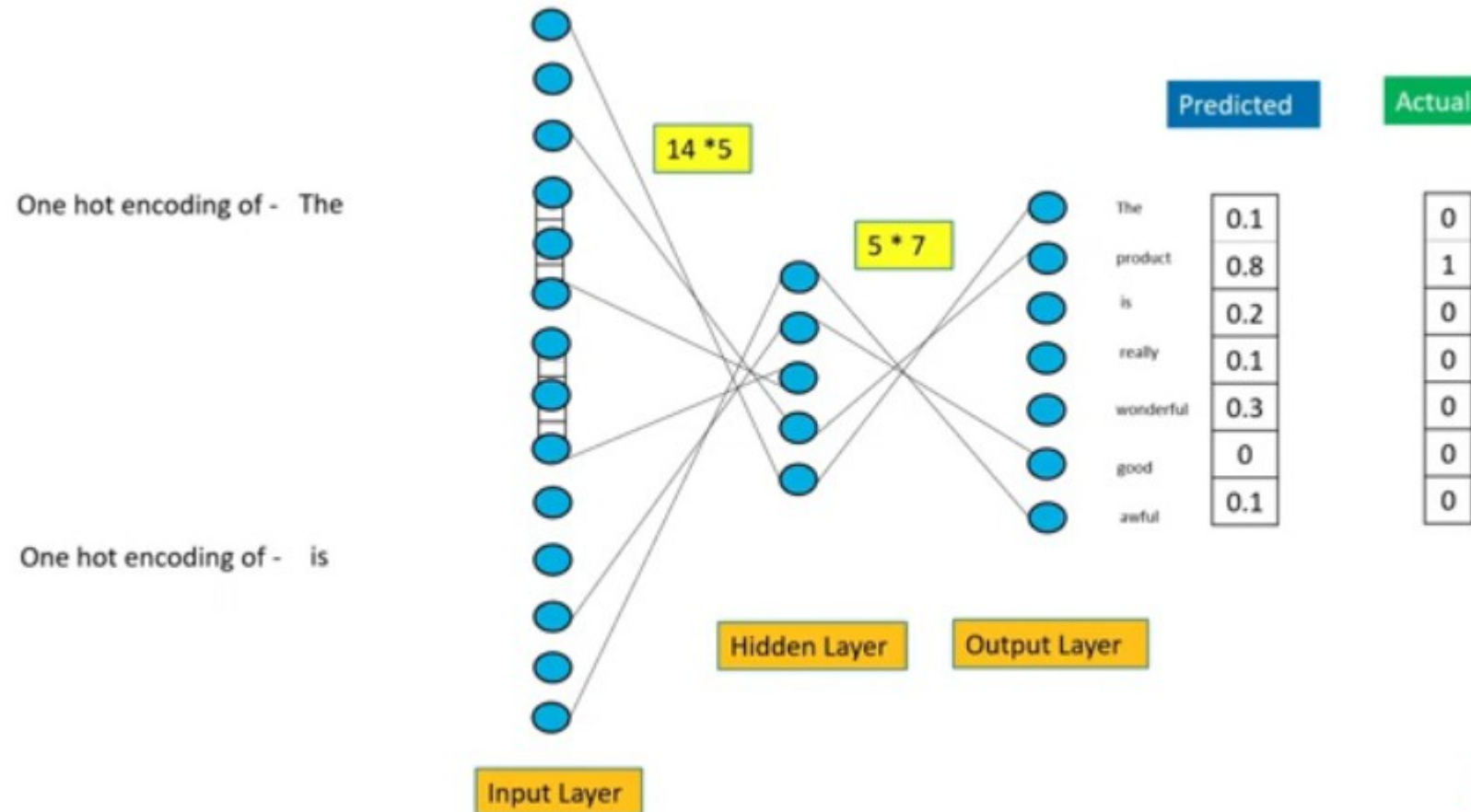


Cond..

innovate

achieve

lead



Example



Corpus = "Ned Stark is the most honourable man"

Current word-context pair = ("Ned", "Stark")

Current negative words = "pimples", "zebra", "idiot"

	one hot vector				
ned	1	0	0	0	0
stark	0	1	0	0	0
pimples	0	0	1	0	0
zebra	0	0	0	1	0
idiot	0	0	0	0	1

initial embedding matrix			
ned	-0.018	0.404	-0.317
stark	0.204	-0.007	-0.733
pimples	-0.706	0.745	0.002
zebra	-0.492	-0.709	0.133
idiot	-0.628	-0.073	0.502

initial context matrix			
ned	-0.572	-0.588	-0.501
stark	0.116	0.723	-0.689
pimples	-0.94	0.601	0.146
zebra	-0.622	0.811	0.64
idiot	-0.077	-0.375	-0.056

Initial embeddings



Current word-context pair = ("**Ned**", "**Stark**")
 Current negative words = "**pimples**", "**zebra**", "**idiot**"

#	Token	x
0	honorable	0
1	is	0
2	man	0
3	most	0
4	ned	1
5	stark	0
6	the	0
Neg.		
	pimples	0
	zebra	0
	idiot	0
	coins	0
	donkey	0
	machine	0

 $\times$

W_{input}		
-0.014	0.135	0.109
-0.665	0.669	0.309
0.702	0.621	-0.981
0.214	-0.813	-0.561
-0.018	0.404	-0.317
0.204	-0.007	-0.733
-0.652	-0.097	0.499
-0.706	0.745	0.002
-0.492	-0.709	0.133
-0.628	-0.073	0.502
0.456	0.767	-0.629
0.348	0.544	-0.740
-0.627	0.487	0.466

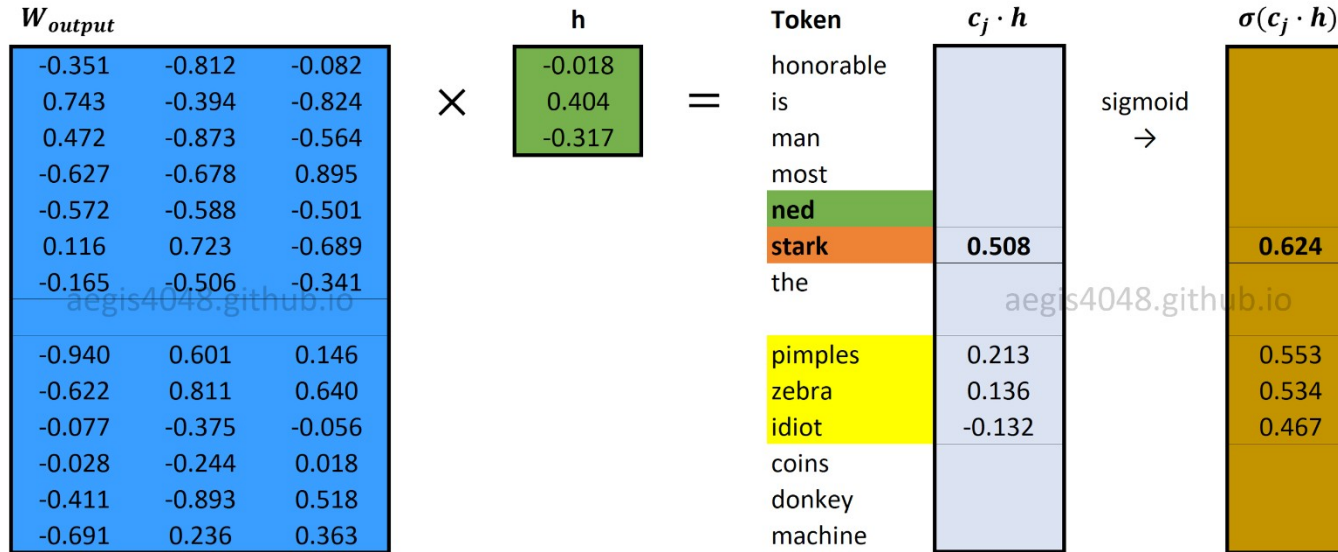
 $=$

h
-0.018
0.404
-0.317

	input embeddings initial		
ned	-0.018	0.404	-0.317
stark	0.204	-0.007	-0.733
pimples	-0.706	0.745	0.002
zebra	-0.492	-0.709	0.133
idiot	-0.628	-0.073	0.502

	context embeddings initial		
ned	-0.572	-0.588	-0.501
stark	0.116	0.723	-0.689
pimples	-0.94	0.601	0.146
zebra	-0.622	0.811	0.64
idiot	-0.077	-0.375	-0.056

Forward Propagation: Sigmoid output layer



$$P(+|w, c) = \sigma(c \cdot w) = \frac{1}{1 + \exp(-c \cdot w)}$$

old context embeddings				old i/p word embedding	dot product	sigmoid()
stark	0.116	0.723	-0.689	-0.018	0.508	0.624
pimples	-0.94	0.601	0.146	0.404	0.213	0.553
zebra	-0.622	0.811	0.64	-0.317	0.136	0.534
idiot	-0.077	-0.375	-0.056		-0.132	0.467

Backward Propagation: Prediction Error



our current positive word is Stark, $t_j=1$ for **Stark** and $t_j=0$ for other negative words (pimples, zebra, idiot).

Token	$\sigma(c_j \cdot h)$	t_j	$\sigma(c_j \cdot h) - t_j$
honorable			
is			
man			
most			
ned			
stark	0.624	1	-0.376
the			
pimples	0.553	0	0.553
zebra	0.534	0	0.534
idiot	0.467	0	0.467
coins			
donkey			
machine			

Sigmoid()	t	sig-t (pred error)
0.624	1	-0.376
0.553	0	0.553
0.534	0	0.534
0.467	0	0.467

Derivative of loss w.r.t input word embeddings



Token	$\sigma(c_j \cdot h) - t_j$	$\{c_{pos}\} \cup W_{neg} \in W_{output}$
honorable		
is		
man		
most		
ned		
stark	-0.376	0.116 0.723 -0.689
the		
pimples	0.553	-0.94 0.601 0.146
zebra	0.534	-0.622 0.811 0.64
idiot	0.467	-0.077 -0.375 -0.056
coins		
donkey		
machine		

Token	∇W_{input}
honorable	
is	
man	
most	
ned	-0.932 0.319 0.655
stark	
the	
pimples	
zebra	
idiot	
coins	
donkey	
machine	

$$\frac{\partial L_{CE}}{\partial w} = [\sigma(c_{pos} \cdot w) - 1]c_{pos} + \sum_{i=1}^k [\sigma(c_{neg_i} \cdot w)]c_{neg_i}$$

	context embeddings initial		
ned	-0.572	-0.588	-0.501
stark	0.116	0.723	-0.689
pimples	-0.94	0.601	0.146
zebra	-0.622	0.811	0.64
idiot	-0.077	-0.375	-0.056

sig-t(pred error)	C*(sig-t)			derivative of loss w.r.t input word embeddings		
-0.376	-0.04362	-0.271848	0.259064	-0.93154	0.318454	0.65541
0.553	-0.51982	0.332353	0.080738			
0.534	-0.33215	0.433074	0.34176			
0.467	-0.03596	-0.175125	-0.02615			

Input word embedding update



$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \left[[\sigma(\mathbf{c}_{pos} \cdot \mathbf{w}^t) - 1] \mathbf{c}_{pos} + \sum_{i=1}^k [\sigma(\mathbf{c}_{neg_i} \cdot \mathbf{w}^t)] \mathbf{c}_{neg_i} \right]$$

Token	$\mathbf{W}_{input}^{(old)}$	η	$\nabla \mathbf{W}_{input}$	$\mathbf{W}_{input}^{(new)}$
honorable	-0.014 0.135 0.109	0.05		-0.014 0.135 0.109
is	-0.665 0.669 0.309			-0.665 0.669 0.309
man	0.702 0.621 -0.981			0.702 0.621 -0.981
most	0.214 -0.813 -0.561			0.214 -0.813 -0.561
ned	-0.018 0.404 -0.317		-0.932 0.319 0.655	0.029 0.388 -0.350
stark	0.204 -0.007 -0.733			0.204 -0.007 -0.733
the	-0.652 -0.097 0.499			-0.652 -0.097 0.499
pimples	-0.706 0.745 0.002			-0.706 0.745 0.002
zebra	-0.492 -0.709 0.133			-0.492 -0.709 0.133
idiot	-0.628 -0.073 0.502			-0.628 -0.073 0.502
coins	0.456 0.767 -0.629			0.456 0.767 -0.629
donkey	0.348 0.544 -0.740			0.348 0.544 -0.740
machine	-0.627 0.487 0.466			-0.627 0.487 0.466

word	old input word embedding			LR	LR* derivative w.r.t input			new i/p word embedding		
ned	-0.018	0.404	-0.317	0.05	-0.04658	0.015923	0.032771	0.029	0.388	-0.35

Derivative of loss w.r.t context word embeddings



$$\frac{\partial L_{CE}}{\partial c_{pos}} = [\sigma(c_{pos} \cdot w) - 1]w$$

$$\frac{\partial L_{CE}}{\partial c_{neg}} = [\sigma(c_{neg} \cdot w)]w$$

the	aegis4048.github.io
pimples	0.553
zebra	0.534
idiot	0.467
coins	
donkey	
machine	

the	aegis4048.github.io		
pimples	-0.010	0.223	-0.175
zebra	-0.010	0.216	-0.169
idiot	-0.008	0.189	-0.148
coins			
donkey			
machine			

sigmoid-t (pred error)	i/p word embedding	derivative of loss w.r.t context word embeddings		
-0.376	-0.018	0.007	-0.152	0.119
0.553	0.404	-0.01	0.223	-0.175
0.534	-0.317	-0.01	0.216	-0.169
0.467		-0.008	0.189	-0.148

Context word embedding update



$$c_{pos}^{t+1} = c_{pos}^t - \eta [\sigma(c_{pos}^t \cdot w^t) - 1] w^t$$

$$c_{neg}^{t+1} = c_{neg}^t - \eta [\sigma(c_{neg}^t \cdot w^t)] w^t$$

Token	$W_{output}^{(old)}$			η	$\times$	∇W_{output}	$=$	$W_{output}^{(new)}$				
honorable	-0.351	-0.812	-0.082	0.05	$\times$		=	-0.351	-0.812	-0.082		
is	0.743	-0.394	-0.824					0.743	-0.394	-0.824		
man	0.472	-0.873	-0.564					0.472	-0.873	-0.564		
most	-0.627	-0.678	0.895					-0.627	-0.678	0.895		
ned	-0.572	-0.588	-0.501							-0.572	-0.588	-0.501
stark	0.116	0.723	-0.689			0.007	-0.152	0.119		0.116	0.731	-0.695
the	-0.165	-0.506	-0.341							-0.165	-0.506	-0.341
aegis4048.github.io												
pimples	-0.94	0.601	0.146			-0.010	0.223	-0.175		-0.940	0.590	0.155
zebra	-0.622	0.811	0.64			-0.010	0.216	-0.169		-0.622	0.800	0.648
idiot	-0.077	-0.375	-0.056			-0.008	0.189	-0.148		-0.077	-0.384	-0.049
coins	-0.028	-0.244	0.018							-0.028	-0.244	0.018
donkey	-0.411	-0.893	0.518							-0.411	-0.893	0.518
machine	-0.691	0.236	0.363							-0.691	0.236	0.363

word	old context word embedding			LR	LR* derivative w.r.t context			new context word embedd		
stark	0.116	0.723	-0.689	0.05	0.00035	-0.0076	0.00595	0.116	0.731	-0.695
pimples	-0.94	0.601	0.146	0.05	-0.0005	0.01115	-0.00875	-0.94	0.59	0.155
zebra	-0.622	0.811	0.64	0.05	-0.0005	0.0108	-0.00845	-0.622	0.8	0.648
idiot	-0.077	-0.375	-0.056	0.05	-0.0004	0.00945	-0.0074	-0.077	-0.384	-0.049

References



- https://aegis4048.github.io/optimize_computational_efficiency_of_skip-gram_with_negative_sampling#pred_error
- Ch-6 : Speech and Language Processing Daniel Jurafsky and James H. Martin

CBOW vs Skip-gram

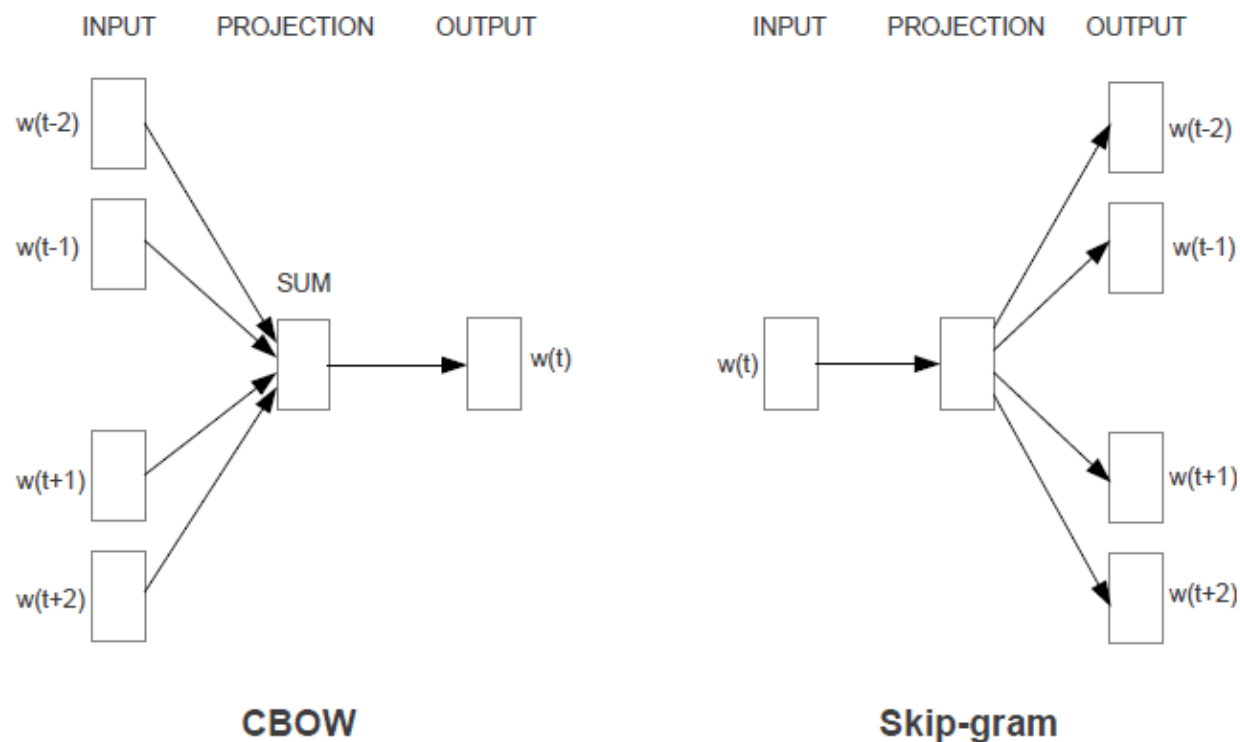


Figure 1: New model architectures. The CBOW architecture predicts the current word based on the context, and the Skip-gram predicts surrounding words given the current word.

- GloVe stands for global vectors for word representation.
- Unsupervised learning algorithm developed by Stanford for generating word embeddings
- GloVe captures both global statistics and local statistics of a corpus, in order to come up with word vectors
- Aggregates global word-word co-occurrence matrix from a corpus
- Idea is learning should be with ratios of co-occurrence probabilities rather than the probabilities themselves.
- The advantage of GloVe is that, unlike Word2vec, GloVe does not rely just on local statistics (local context information of words), but incorporates global statistics (word co-occurrence) to obtain word vectors.

Matrix of word-word co-occurrence counts

- I like deep learning.
- I like NLP.
- I enjoy flying.

counts	I	like	enjoy	deep	learning	NLP	flying	.
I	0	2	1	0	0	0	0	0
like	2	0	0	1	0	1	0	0
enjoy	1	0	0	0	0	0	1	0
deep	0	1	0	0	1	0	0	0
learning	0	0	0	1	0	0	0	1
NLP	0	1	0	0	0	0	0	1
flying	0	0	1	0	0	0	0	1
.	0	0	0	0	1	1	1	0

- let the matrix of word-word co-occurrence counts be denoted by X ,
- whose entries X_{ij} tabulate the number of times word j occurs in the context of word i
- let $X_i = \sum_k X_{ik}$ be the number of times any word appears in the context of word i
- let $P_{ij} = P(i|j) = X_{ij}/X_i$ be the probability that word i appear in the context of word j

Intuition behind GloVe

ratio of conditional probabilities represents the word meanings

Probability and Ratio	$k = \text{solid}$	$k = \text{gas}$	$k = \text{water}$	$k = \text{fashion}$
$P(k \text{ice})$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \text{steam})$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k \text{ice})/P(k \text{steam})$	8.9	8.5×10^{-2}	1.36	0.96

Very small or large:

solid is related to ice but not steam, or
gas is related to steam but not ice

close to 1:

water is highly related to ice and steam, or
fashion is not related to ice or steam.

- Given a prob word, the ratio can be small, large or equal to 1 depends on their correlations.
- For example, if the ratio is large, the probe word is related to w_i but not w_j .
- This ratio gives us hints on the relations between three different words

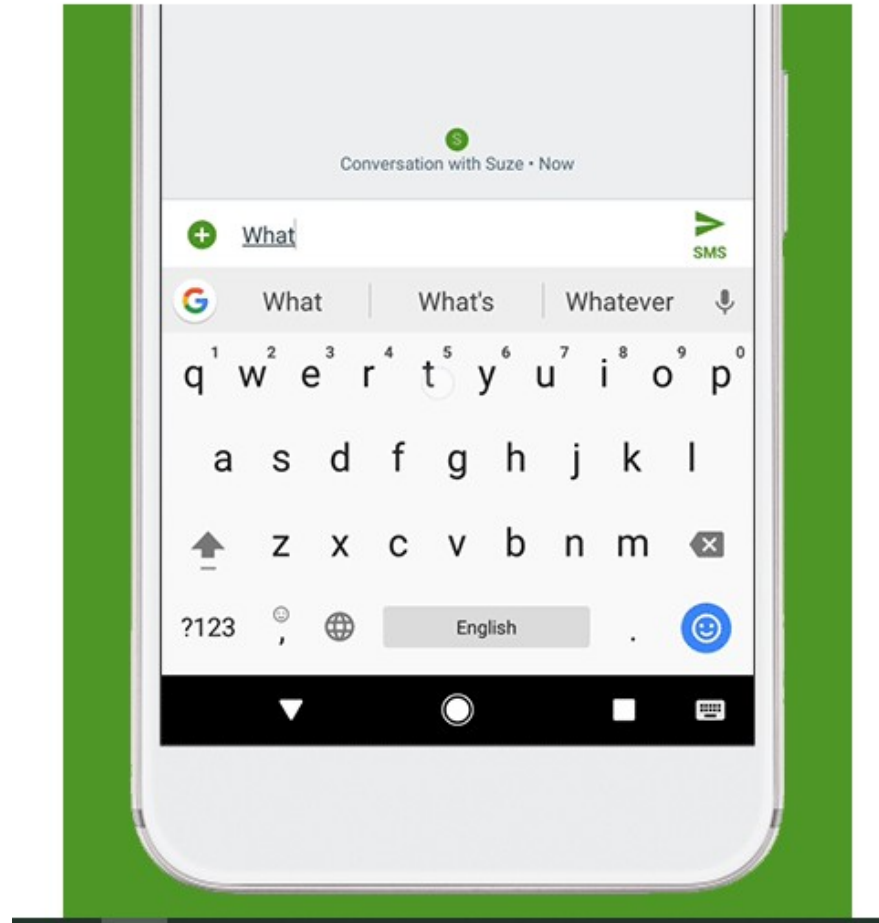
Session#2 – N gram Language model

- Human word Prediction
- Language Models
 - What is language model
 - Why language models
- N-gram language models
 - Uni-gram , bi-gram and N-gram
- Evaluation of Language models
 - Perplexity
- Smoothing
 - Laplace smoothing
 - Interpolation and Back off

Next word Prediction

- We may have ability to predict future words in an utterance .
- How?
 - ❖ Based on domain knowledge
red blood
 - ❖ Based on syntactic knowledge
the <adj/noun>
 - ❖ Based on Lexical knowledge
baked <potato>

Example



Language modelling



A model that computes either of these:

Probability of a sentence ($P(W)$) or Probability of an upcoming word ($P(w_n | w_1, w_2 \dots w_{n-1})$) is called a **language model**.

Simply we can say that

A language model learns to predict the probability of a sequence of words.

Language Models



1. Machine Translation:

Machine translation system is used to translate one language to another language .For example Chinese to English or German to English etc.

I eat lunch

我在吃”



I am eating

Me am eating

Eating am I

3.Speech Recognition

Speech recognition is the ability of a machine or program to identify words and phrases in spoken language and convert them to a machine-readable format.



Sentiment analysis

Sentiment analysis using a language model involves determining the emotional tone or polarity (e.g., positive, negative, or neutral) of a given piece of text.

- **Use Case in Real Life**
- **E-commerce:** Analyzing customer reviews for feedback.
- **Social Media Monitoring:** Understanding public sentiment about a brand.
- **Healthcare:** Detecting emotional distress in patients' messages.

How to build a language model



- Recall the definition of conditional probabilities

$$p(B|A) = P(A,B)/P(A) \text{ Rewriting: } P(A,B) = P(A)P(B|A)$$

- More variables:

$$P(A,B,C,D) = P(A)P(B|A)P(C|A,B)P(D|A,B,C)$$

- The **Chain Rule** in General

$$P(x_1, x_2, x_3, \dots, x_n) = P(x_1)P(x_2|x_1)P(x_3|x_1, x_2) \dots P(x_n|x_{n-1}, x_{n-2}, \dots, x_1)$$

Example

In a factory there are 100 units of a certain product, 5 of which are defective. We pick three units from the 100 units at random. What is the probability that none of them are defective?

Let A_i as the event and $i=1,2,3$

$$P(A_1) = \frac{95}{100}.$$

Given that the first chosen item was good, the second item will be chosen from 94 good units and 5 defective units, thus

$$P(A_2|A_1) = \frac{94}{99}.$$

Given that the first and second chosen items were okay, the third item will be chosen from 93 good units and 5 defective units, thus

$$P(A_3|A_2, A_1) = \frac{93}{98}.$$

$$\begin{aligned} P(A_1 \cap A_2 \cap A_3) &= P(A_1)P(A_2|A_1)P(A_3|A_2, A_1) \\ &= \frac{95}{100} \frac{94}{99} \frac{93}{98} \\ &= 0.8560 \end{aligned}$$

The Chain Rule applied to compute joint probability of words in sentence



-)

- For ex:

$P(\text{"its water is so transparent"}) =$

$P(\text{its}) \times P(\text{water} | \text{its}) \times P(\text{is} | \text{its water})$

$\times P(\text{so} | \text{its water is}) \times P(\text{transparent} | \text{its water is so})$

- **$P(\text{most biologists and specialist believe that in fact the mythical unicorn horns derived from the narwhal})$**

Markov Assumption



- Simplifying assumption:

limit history of fixed number of words $N-1$



Andrei Markov

$P(\text{the l its water is so transparent that}) \approx P(\text{the l that})$



or

$P(\text{the l its water is so transparent that}) \approx P(\text{the l transparent that})$



Markov Assumption

$$P(w_1 w_2 \dots w_n) \approx \prod_i P(w_i \mid w_{i-k} \dots w_{i-1})$$

In other words, we approximate each component in the product

$$P(w_i \mid w_1 w_2 \dots w_{i-1}) \approx P(w_i \mid w_{i-k} \dots w_{i-1})$$

N -Gram Language models

N-gram Language models

- N gram is a sequence of tokens(words)
- Unigram language model(N=1)

$$P(w_1 w_2 \dots w_n) \approx \prod_i P(w_i)$$

Example:

**$P(\text{I want to eat Chinese food}) \approx P(\text{I})P(\text{want})$
 $P(\text{to})P(\text{eat})P(\text{Chinese})P(\text{food})$**

Bigram model



N=2

$$P(w_i | w_1 w_2 \dots w_{i-1}) \approx P(w_i | w_{i-1})$$

Example:

$P(\text{I want to eat Chinese food}) \approx P(\text{I} | \text{<start>}) P(\text{want} | \text{I}) P(\text{to} | \text{want})$
 $P(\text{eat} | \text{to}) P(\text{Chinese} | \text{eat}) P(\text{food} | \text{Chinese})$
 $P(\text{<end>} | \text{food})$

N-gram models

- We can extend to trigrams, 4-grams, 5-grams

Advantages

- no human supervision, easy to extend to more data, allows querying about open-class relations,

Disadvantage:

- In general this is an insufficient model of language
 - because language has **long-distance dependencies**:
“The computer(s) which I had just put into the machine room on the fifth floor is (are) crashing.”

Estimating bigram probabilities

- Estimating the probability as the relative frequency is the *maximum likelihood estimate* (or *MLE*), because this value makes the observed data maximally likely
- The Maximum Likelihood Estimate

$$P(w_i | w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}$$

$$P(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

An example

$$P(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like green eggs and ham </s>

$$P(\text{I} | \text{<s>}) = \frac{2}{3} = .67$$

$$P(\text{Sam} | \text{<s>}) = \frac{1}{3} = .33$$

$$P(\text{am} | \text{I}) = \frac{2}{3} = .67$$

$$P(\text{</s>} | \text{Sam}) = \frac{1}{2} = 0.5$$

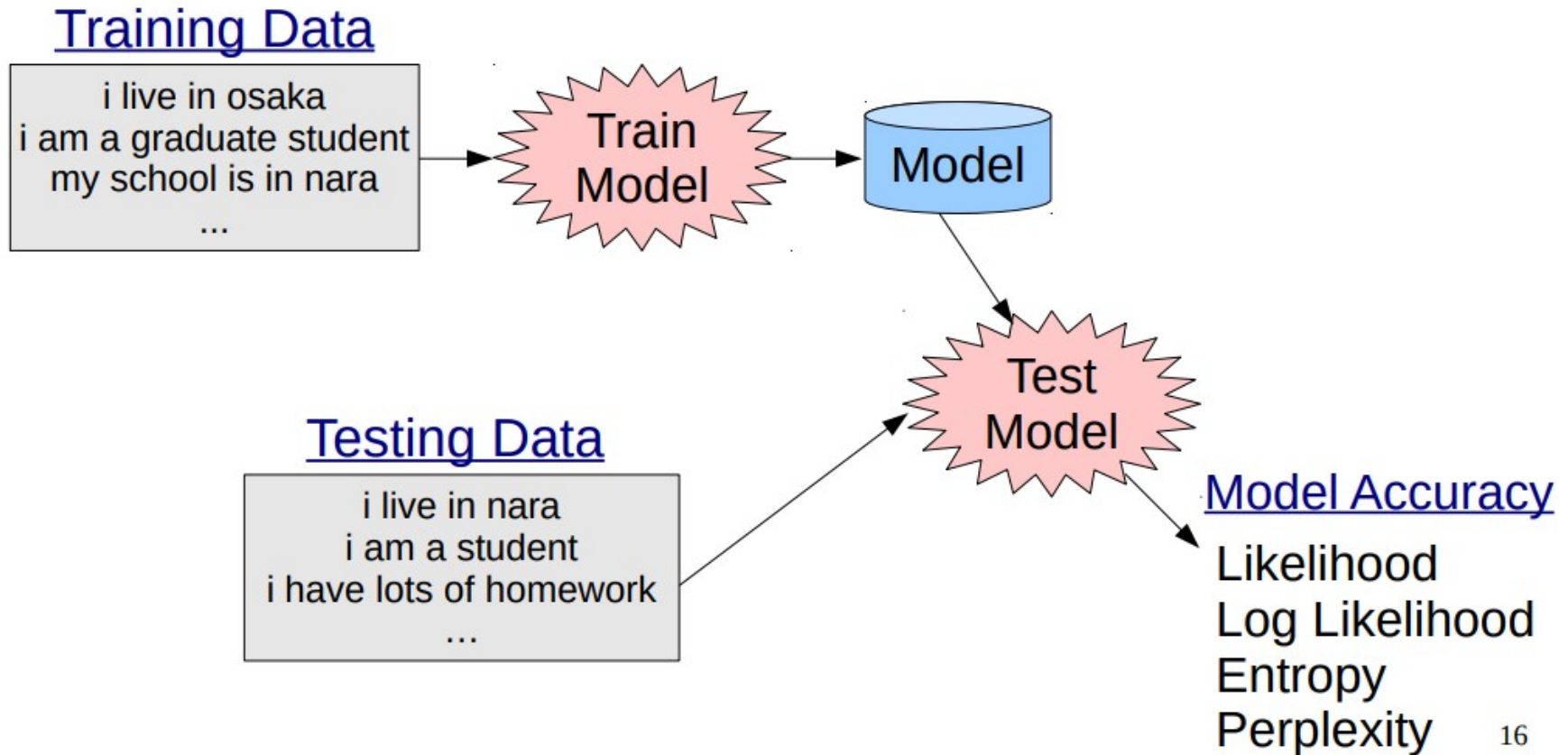
$$P(\text{Sam} | \text{am}) = \frac{1}{2} = .5$$

$$P(\text{do} | \text{I}) = \frac{1}{3} = .33$$

How good is our model?

- Does our language model prefer good sentences to bad ones?
 - Assign higher probability to “real” or “frequently observed” sentences
 - Than “ungrammatical” or “rarely observed” sentences?
- We train parameters of our model on a **training set**.
- We test the model’s performance on data we haven’t seen.
 - A **test set** is an unseen dataset that is different from our training set, totally unused.
 - An **evaluation metric** tells us how well our model does on the test set.

Experimental setup



Example1:

Training data

There is a big house

I buy a house

They buy the new house

Model

$p(\text{big} | a) = 0.5$
 $p(\text{is} | \text{there}) = 1$
 $p(\text{buy} | \text{they}) = 1$
 $p(\text{house} | a) = 0.5$
 $p(\text{buy} | i) = 1$
 $p(a | \text{buy}) = 0.5$
 $p(\text{new} | \text{the}) = 1$
 $p(\text{house} | \text{big}) = 1$
 $p(\text{the} | \text{buy}) = 0.5$
 $p(a | \text{is}) = 1$
 $p(\text{house} | \text{new}) = 1$
 $p(\text{they} | <s>) = .333$

Test data

S1:

they buy a big house

$$P(S1) = 0.333 * 1 * 0.5 * 0.5 * 1$$

$$P(S1) = 0.0833$$

S2:

they buy **a new** house

$$P(S2) = ?$$

$$P(w_1, w_2, \dots, w_T) = \prod_{t=1}^T P(w_t | w_{t-1})$$

Extrinsic evaluation of N-gram models



- Best evaluation for comparing models A and B
 - Put each model in a task
 - spelling corrector, speech recognizer, MT system
 - Run the task, get an accuracy for A and for B
 - How many misspelled words corrected properly
 - How many words translated correctly
 - Compare accuracy for A and B

Difficulty of extrinsic (in-vivo) evaluation of N-gram models



- Extrinsic evaluation
 - Time-consuming; can take days or weeks
 - Bad approximation
 - unless the test data looks **just** like the training data
 - So **generally only useful in pilot experiments**
- So
 - Sometimes use **intrinsic** evaluation: **perplexity**

Intuition of Perplexity



- The Shannon Game:
 - How well can we predict the next word?

I always order pizza with cheese and....

The president of India is.....

I wrote a

mushrooms 0.1
pepperoni 0.1
anchovies 0.01
....
fried rice 0.0001
....
and 1e-100

- Unigrams are terrible at this game. (Why?)
- A better model of a text
 - is one which assigns a higher probability to the word that actually occurs

Perplexity



The best language model is one that best predicts an unseen test set

- Gives the highest $P(\text{sentence})$

Perplexity is the inverse probability of the test set, normalized by the number of words:

$$\begin{aligned} PP(W) &= P(w_1 w_2 \dots w_N)^{-\frac{1}{N}} \\ &= \sqrt[N]{\frac{1}{P(w_1 w_2 \dots w_N)}} \end{aligned}$$

Chain rule:

$$PP(W) = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i | w_1 \dots w_{i-1})}}$$

For bigrams:

$$PP(W) = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i | w_{i-1})}}$$

Minimizing perplexity is the same as maximizing probability

Example1

<S> I am Henry </S>
 <S> I like college </S>
 <S> Do Henry like college </S>
 <S> Henry I am </S>
 <S> Do I like Henry </S>
 <S> Do I like college </S>
 <S> I do like Henry </S>

Word	Frequency
<S>	7
</S>	7
I	6
am	2
Henry	5
like	5
college	3
do	4

Calculate the perplexity for the bigram language model for the below test sentence

<S> I like college </S>

Soln:

(<S>, I)(I, like)(like, college)(college, </S>)

$$\begin{aligned}
 P(w) &= P(I | <S>) \times P(\text{like} | I) \times P(\text{college} | \text{like}) \times P(</S> | \text{college}) \\
 &= \frac{3}{7} \times \frac{3}{6} \times \frac{2}{5} \times \frac{2}{3} = \frac{3 \times 3 \times 2 \times 2}{7 \times 6 \times 5 \times 3} = \frac{36}{630} = 0.0571
 \end{aligned}$$

$$PP(W) = (P(W))^{-\frac{1}{5}} = (0.0571)^{-\frac{1}{5}} \approx 3.71$$

Examples of corpora (in chronological order)

Focusing on English; most released by the **Linguistic Data Consortium** (LDC):

Brown: 500 texts, 1M words in 15 genres. POS-tagged. **SemCor** subset (234K words) labelled with WordNet word senses.

WSJ: 6 years of *Wall Street Journal*; subsequently used to create Penn Treebank, PropBank, and more! Translated into Czech for the **Prague Czech-English Dependency Treebank**.

ECI: European Corpus Initiative, multilingual.

BNC: 100M words; balanced selection of written and spoken genres.

Redwoods: Treebank aligned to wide-coverage grammar; several genres.

Gigaword: 1B words of news text.

AMI: Multimedia (video, audio, synchronised transcripts).

Google Books N-grams: 5M books, 500B words (361B English).

Flickr 8K: images with NL captions

English Visual Genome: Images, bounding boxes $\Rightarrow$ NL descriptions

Lower perplexity = better model

- Training 38 million words, test 1.5 million words, WSJ

N-gram Order	Unigram	Bigram	Trigram
Perplexity	962	170	109

Generalization and Zeros

The perils of overfitting

- N-grams only work well for word prediction if the test corpus looks like the training corpus
 - In real life, it often doesn't
 - We need to train robust models that generalize!
 - One kind of generalization: Zeros!
 - Things that don't ever occur in the training set
 - But occur in the test set

Problems with simple MLE estimations

- Training set:
 - ... denied the allegations
 - ... denied the reports
 - ... denied the claims
 - ... denied the request
- Test set
 - ... denied the offer
 - ... denied the loan

$$P(\text{"offer"} \mid \text{denied the}) = 0$$

Zero probability bigrams

- Bigrams with zero probability
 - mean that we will assign 0 probability to the test set!
- And hence we cannot compute perplexity (can't divide by 0)!

Laplace Smoothing(Add 1 smoothing)

- Pretend we saw each word one more time than we did
- Just add one to all the counts!

- MLE estimate:

$$P_{MLE}(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

- Add-1 estimate:

$$P_{Add-1}(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i) + 1}{c(w_{i-1}) + V}$$

Example

JOHN READ MOBY DICK
MARY READ A DIFFERENT BOOK
SHE READ A BOOK BY CHER

$p(\text{JOHN READ A BOOK})$

$$\begin{aligned}
 &= p(\text{JOHN}|\bullet) \quad p(\text{READ}|\text{JOHN}) \quad p(\text{A}|\text{READ}) \quad p(\text{BOOK}|\text{A}) \quad p(\bullet|\text{BOOK}) \\
 &= \frac{c(\bullet \text{ JOHN})}{\sum_w c(\bullet w)} \quad \frac{c(\text{JOHN READ})}{\sum_w c(\text{JOHN } w)} \quad \frac{c(\text{READ A})}{\sum_w c(\text{READ } w)} \quad \frac{c(\text{A BOOK})}{\sum_w c(\text{A } w)} \quad \frac{c(\text{BOOK } \bullet)}{\sum_w c(\text{BOOK } w)} \\
 &= \quad \frac{1}{3} \quad \quad \quad \frac{1}{1} \quad \quad \quad \frac{2}{3} \quad \quad \quad \frac{1}{2} \quad \quad \quad \frac{1}{2} \\
 &\approx 0.06
 \end{aligned}$$

JOHN READ MOBY DICK
 MARY READ A DIFFERENT BOOK
 SHE READ A BOOK BY CHER

$$p(\text{CHER READ A BOOK})$$

$$= p(\text{CHER}|\bullet) \ p(\text{READ}|\text{CHER}) \ p(\text{A}|\text{READ}) \ p(\text{BOOK}|\text{A}) \ p(\bullet|\text{BOOK})$$

$$= \frac{c(\bullet \text{ CHER})}{\sum_w c(\bullet w)} \frac{c(\text{CHER READ})}{\sum_w c(\text{CHER } w)} \frac{c(\text{READ A})}{\sum_w c(\text{READ } w)} \frac{c(\text{A BOOK})}{\sum_w c(\text{A } w)} \frac{c(\text{BOOK } \bullet)}{\sum_w c(\text{BOOK } w)}$$

$$= \frac{0}{3} \quad \frac{0}{1} \quad \frac{2}{3} \quad \frac{1}{2} \quad \frac{1}{2}$$

$$= 0$$

After Add-1 smoothing

JOHN READ MOBY DICK
MARY READ A DIFFERENT BOOK
SHE READ A BOOK BY CHER

$$p(\text{JOHN READ A BOOK})$$

$$= \frac{1+1}{11+3} \frac{1+1}{11+1} \frac{1+2}{11+3} \frac{1+1}{11+2} \frac{1+1}{11+2}$$

$$\approx 0.0001$$

$$p(\text{CHER READ A BOOK})$$

$$= \frac{1+0}{11+3} \frac{1+0}{11+1} \frac{1+2}{11+3} \frac{1+1}{11+2} \frac{1+1}{11+2}$$

$$\approx 0.00003$$

Berkeley Restaurant Project corpus

- Let's compute simple N -gram models of speech queries about restaurants in Berkeley California.
 - E.g.,
 - *can you tell me about any good cantonese restaurants close by*
 - *mid priced thai food is what i'm looking for*
 - *tell me about chez panisse*
 - *can you give me a listing of the kinds of food that are available*
 - *i'm looking for a good place to eat breakfast*
 - *when is caffe venezia open during the day*

Raw Bigram Counts

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Converting to probabilities

- Obtain likelihoods by dividing bigram counts by unigram counts.

Unigram counts:

I	want	to	eat	Chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

$P(w_t w_{t-1})$	I	want	to	eat	Chinese	food	lunch	spend
I	0.002	0.33	0	0.0036	0	0	0	0.00079

$$P(\text{want}|I) \approx \frac{\text{Count}(I \text{ want})}{\text{Count}(I)} = \frac{827}{2533} \approx 0.33$$

$$P(\text{spend}|I) \approx \frac{\text{Count}(I \text{ spend})}{\text{Count}(I)} = \frac{2}{2533} \approx 7.9 \times 10^{-4}$$

Contd...



- Obtain likelihoods by dividing bigram counts by unigram counts.

Unigram counts:

I	want	to	eat	Chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

$P(w_t w_{t-1})$	I	want	to	eat	Chinese	food	lunch	spend
I	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
Chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Berkeley Restaurant Corpus: Laplace smoothed bigram



- Out of 9222 sentences in Berkeley restaurant corpus,
 - e.g., “I want” occurred 827 times so Laplace gives 828

Count(w_{t-1}, w_t)		w_t							
		I	want	to	eat	Chinese	food	lunch	spend
w_{t-1}	I	5+1	827+1	1	9+1	1	1	1	2+1
	want	2+1	1	608+1	1+1	6+1	6+1	5+1	1+1
	to	2+1	1	4+1	686+1	2+1	1	6+1	211+1
	eat	1	1	2+1	1	16+1	2+1	42+1	1
	Chinese	1+1	1	1	1	1	82+1	1+1	1
	food	15+1	1	15+1	1	1+1	4+1	1	1
	lunch	2+1	1	1	1	1	1+1	1	1
	spend	1+1	1	1+1	1	1	1	1	1

Laplace-smoothed bigrams

$$P_{Lap}(w_t|w_{t-1}) = \frac{C(w_{t-1}w_t) + 1}{C(w_{t-1}) + \|\mathcal{V}\|}$$

$P(w_t w_{t-1})$	I	want	to	eat	Chinese	food	lunch	spend
I	0.0015	0.21	0.00025	0.0025	0.00025	0.00025	0.00025	0.00075
want	0.0013	0.00042	0.26	0.00084	0.0029	0.0029	0.0025	0.00084
to	0.00083	0.00026	0.0013	0.18	0.00078	0.00026	0.0018	0.055
eat	0.00046	0.00046	0.0014	0.00046	0.0078	0.0014	0.02	0.00046
Chinese	0.0012	0.00062	0.00062	0.00062	0.00062	0.052	0.0012	0.00062
food	0.0063	0.00039	0.0063	0.00039	0.00079	0.002	0.00039	0.00039
lunch	0.0017	0.00056	0.00056	0.00056	0.00056	0.0011	0.00056	0.00056
spend	0.0012	0.00058	0.0012	0.00058	0.00058	0.00058	0.00058	0.00058

V=1446 in the Berkeley restaurant project corpus

Reconstituted counts



$$c^*(w_{n-1}w_n) = \frac{[C(w_{n-1}w_n) + 1] \times C(w_{n-1})}{C(w_{n-1}) + V}$$

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527	0.64	6.4	0.64	0.64	0.64	1.9
want	1.2	0.39	238	0.78	2.7	2.7	2.3	0.78
to	1.9	0.63	3.1	430	1.9	0.63	4.4	133
eat	0.34	0.34	1	0.34	5.8	1	15	0.34
chinese	0.2	0.098	0.098	0.098	0.098	8.2	0.2	0.098
food	6.9	0.43	6.9	0.43	0.86	2.2	0.43	0.43
lunch	0.57	0.19	0.19	0.19	0.19	0.38	0.19	0.19
spend	0.32	0.16	0.32	0.16	0.16	0.16	0.16	0.16

Compare with raw bigram counts



	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527	0.64	6.4	0.64	0.64	0.64	1.9
want	1.2	0.39	238	0.78	2.7	2.7	2.3	0.78
to	1.9	0.63	3.1	430	1.9	0.63	4.4	133
eat	0.34	0.34	1	0.34	5.8	1	15	0.34
chinese	0.2	0.098	0.098	0.098	0.098	8.2	0.2	0.098
food	6.9	0.43	6.9	0.43	0.86	2.2	0.43	0.43
lunch	0.57	0.19	0.19	0.19	0.19	0.38	0.19	0.19
spend	0.32	0.16	0.32	0.16	0.16	0.16	0.16	0.16

Add-1 estimation is a blunt instrument



- So add-1 isn't used for N-grams:
 - We'll see better methods
- But add-1 is used to smooth other NLP models
 - For text classification
 - In domains where the number of zeros isn't so huge.

Backoff and Interpolation

- Sometimes it helps to use **less** context
 - Condition on less context for contexts you haven't learned much about
- **Backoff:**
 - use trigram if you have good evidence,
otherwise bigram,
otherwise unigram
- **Interpolation:**
 - mix unigram, bigram, trigram
- Interpolation works better

Linear Interpolation

- Involves using different pieces of information to derive a probability

Simple interpolation

$$\begin{aligned}\hat{P}(w_n|w_{n-2}w_{n-1}) &= \lambda_1 P(w_n|w_{n-2}w_{n-1}) \\ &\quad + \lambda_2 P(w_n|w_{n-1}) \\ &\quad + \lambda_3 P(w_n)\end{aligned}$$

$$\sum_i \lambda_i = 1$$

Conditional interpolation

$$\begin{aligned}\hat{P}(w_n|w_{n-2}w_{n-1}) &= \lambda_1 (w_{n-2}^{n-1}) P(w_n|w_{n-2}w_{n-1}) \\ &\quad + \lambda_2 (w_{n-1}^{n-1}) P(w_n|w_{n-1}) \\ &\quad + \lambda_3 (w_n^{n-1}) P(w_n)\end{aligned}$$

How to set the lambdas?

- Use a **held-out** corpus



- Choose λ s to maximize the probability of held-out data:
 - Fix the N-gram probabilities (on the training data)
 - Then search for λ s that give largest probability to held-out set:

Unknown words: Open versus closed vocabulary tasks



- If we know all the words in advanced
 - Vocabulary V is fixed
 - Closed vocabulary task
- Often we don't know this
 - **Out Of Vocabulary** = OOV words
 - Open vocabulary task
- Instead: create an unknown word token <UNK>
 - Training of <UNK> probabilities
 - Create a fixed lexicon L of size V
 - At text normalization phase, any training word not in L changed to <UNK>
 - Now we train its probabilities like a normal word
 - At decoding time
 - If text input: Use UNK probabilities for any word not in training

Huge web-scale n-grams

- How to deal with, e.g., Google N-gram corpus
- Pruning
 - Entropy-based pruning
 - Only store N-grams with count > threshold.
- Efficiency
 - Efficient data structures like tries
 - Bloom filters: approximate language models
 - Store words as indexes, not strings
 - Use Huffman coding to fit large numbers of words into two bytes

Smoothing for Web-scale N-grams

- “Stupid backoff” (Brants *et al.* 2007)

$$S(w_i | w_{i-k+1}^{i-1}) = \begin{cases} \frac{\text{count}(w_{i-k+1}^i)}{\text{count}(w_{i-k+1}^{i-1})} & \text{if } \text{count}(w_{i-k+1}^i) > 0 \\ 0.4S(w_i | w_{i-k+2}^{i-1}) & \text{otherwise} \end{cases}$$

$$S(w_i) = \frac{\text{count}(w_i)}{N}$$

Exercise 1

What is the most probable next word predicted by the model for the following word sequence?

Given Corpus

<S> I am Henry </S>

<S> I like college </S>

<S> Do Henry like college </S>

<S> Henry I am </S>

<S> Do I like Henry </S>

<S> Do I like college </S>

<S> I do like Henry </S>

Word	Frequency
<S>	7
</S>	7
I	6
am	2
Henry	5
like	5
college	3
do	4

- 1.<s> do? Two gram
- 2.<s>I like Hendry?
Two gram
- 3.<s> Do I like? Use
trigram model
- 4.<s>Do I like
college?
Four grams

Exercise2

Which of the following sentence is better. Find out using bigram model

Given Corpus

<S> I am Henry </S>

<S> I like college </S>

<S> Do Henry like college </S>

<S> Henry I am </S>

<S> Do I like Henry </S>

<S> Do I like college </S>

<S> I do like Henry </S>

Word	Frequency
<S>	7
</S>	7
I	6
am	2
Henry	5
like	5
college	3
do	4

1.<S>I like college<S>

2.<S>Do I like
Hendry<S>

Neural Language Model

Types of language model



- Statistical model
- Neural language model

Probabilistic Language Models



- Probability of a sequence of words: $P(W) = P(w_1, w_2, \dots, w_{t-1}, w_T)$
- **Conditional probability** of an upcoming word: $P(w_T | w_1, w_2, \dots, w_{t-1})$
- Chain rule of probability: $P(w_1, w_2, \dots, w_{t-1}, w_T) = P(w_1)P(w_2 | w_1)P(w_3 | w_1, w_2) \dots P(w_T | w_1, w_2, \dots, w_{t-1})$
- $(n-1)^{\text{th}}$ order Markov assumption $P(w_1, w_2, \dots, w_{t-1}, w_T) = \prod_{t=1}^T P(w_t | w_1, w_2, \dots, w_{t-1})$
- Each $p(w_i | w_{i-4}, w_{i-3}, w_{i-2}, w_{i-1})$ may not have enough statistics to estimate
 - we back off to $p(w_i | w_{i-3}, w_{i-2}, w_{i-1})$, $p(w_i | w_{i-2}, w_{i-1})$, etc., all the way to $p(w_i)$

Example



Have a good ...

day

weather

terrible

time

4-gram language model:

$$p(\text{day} \mid \text{have a good}) = \frac{c(\text{have a good day})}{c(\text{have a good})}$$

Curse of dimensionality

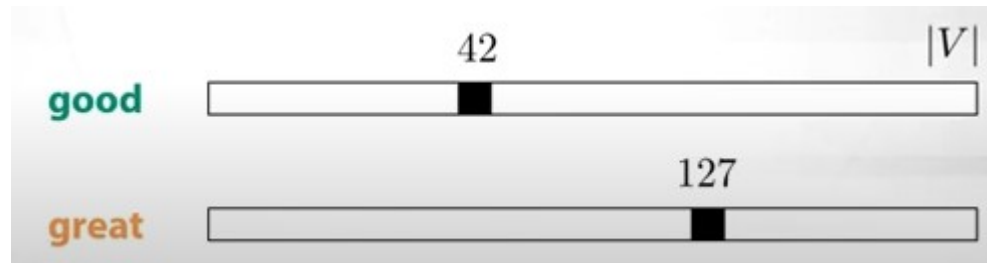


Consider this sentences

Have a good day

Have a great day

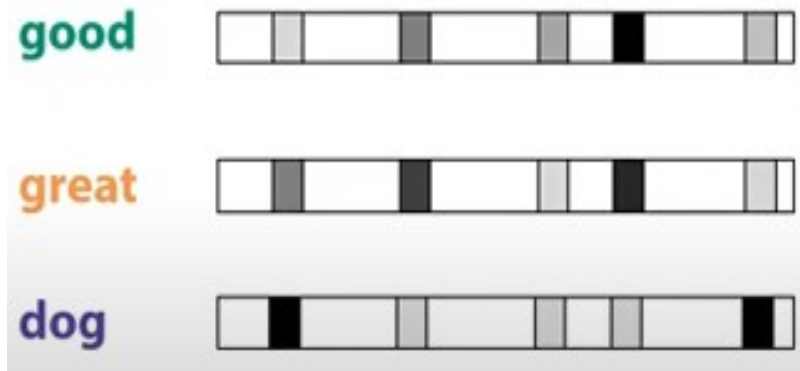
What happens in smoothing ?



How to generalize better



- Learn distributed representation of words
- What is distributed representation?
 - also known as embedding.
 - Eg:



Neural Probabilistic Language Models



- Instead of treating words as tokens, exploit semantic similarity
 - Learn a distributed representation of words that will allow sentences like these to be seen as similar

The cat is walking in the bedroom.
A dog was walking in the room.
The cat is running in a room.
The dog was running in the bedroom.
etc.
 - Use a neural net to represent the conditional probability function
 - Learn the word representation and the probability function simultaneously

Simple feedforward Neural Language Models



- **Task:** predict next word w_t
given prior words $w_{t-1}, w_{t-2}, w_{t-3}, \dots$
- **Output :** probability distribution over possible next words.

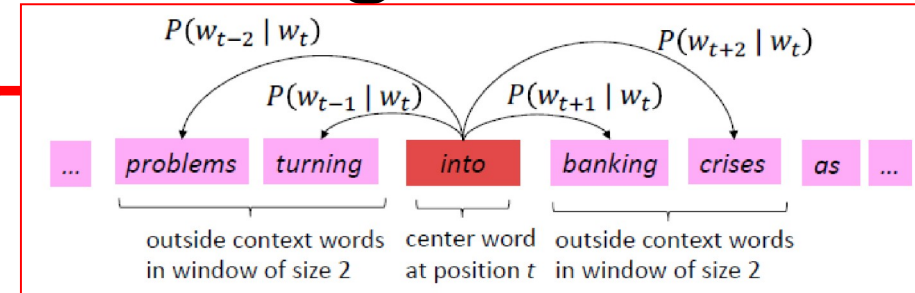
$$P(w_t | w_1, \dots, w_{t-1}) \approx P(w_t | w_{t-N+1}, \dots, w_{t-1})$$

- **Problem:** Now we're dealing with sequences of arbitrary length.
- **Solution:** Sliding windows (of fixed length)

Skip gram –word embedding



- skip-gram with negative sampling (SGNS)
- $N = 2$, $C=4$ (Here C is the context window)



Thou shalt not make a machine in the likeness of a human mind

thou	shalt	not	make	a	machine	in	the	...
thou	shalt	not	make	a	machine	in	the	...
thou	shalt	not	make	a	machine	in	the	...
thou	shalt	not	make	a	machine	in	the	...
thou	shalt	not	make	a	machine	in	the	...

input word	target word
not	thou
not	shalt
not	make
not	a
make	shalt
make	not
make	a
make	machine
a	not
a	make
a	machine
a	in
machine	make
machine	a
machine	in
machine	the
in	a
in	machine
in	the
in	likeness

Source Credit : <https://jalammar.github.io/illustrated-word2vec/>

Contd..

- skip-gram with negative sampling (**SGNS**)
- $N = 2$, $C=4$ (Here C is the context window)

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

input word	target word
not	thou
not	shalt
not	make
not	a
make	shalt
make	not
make	a
make	machine

input word	output word	target
not	thou	1
not	shalt	1
not	make	1
not	a	1
make	shalt	1
make	not	1
make	a	1
make	machine	1

input word	output word	target
not	thou	1
not		0
not		0
not	shalt	1
not	make	1

➤ Negative examples

Word Embeddings



- one-hot vector representation , e.g., dog = (0,0,0,0,1,0,0,0,0,...), cat = (0,0,0,0,0,0,0,0,1,0,...)
- Represent each of the N previous words as a one-hot vector of length $|V|$ one-hot vector , i.e., with one dimension for each word in the vocabulary
- word “toothpaste”, supposing it is V_5 , i.e., index 5 in the vocabulary, $x_5 = 1$, and $x_i = 0 \text{ } i \neq 5$,

$[0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 0 \ \dots \ 0 \ 0 \ 0 \ 0]$
 $1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ \dots \ \dots \ |V|$

Embedding matrix

One-hot vector

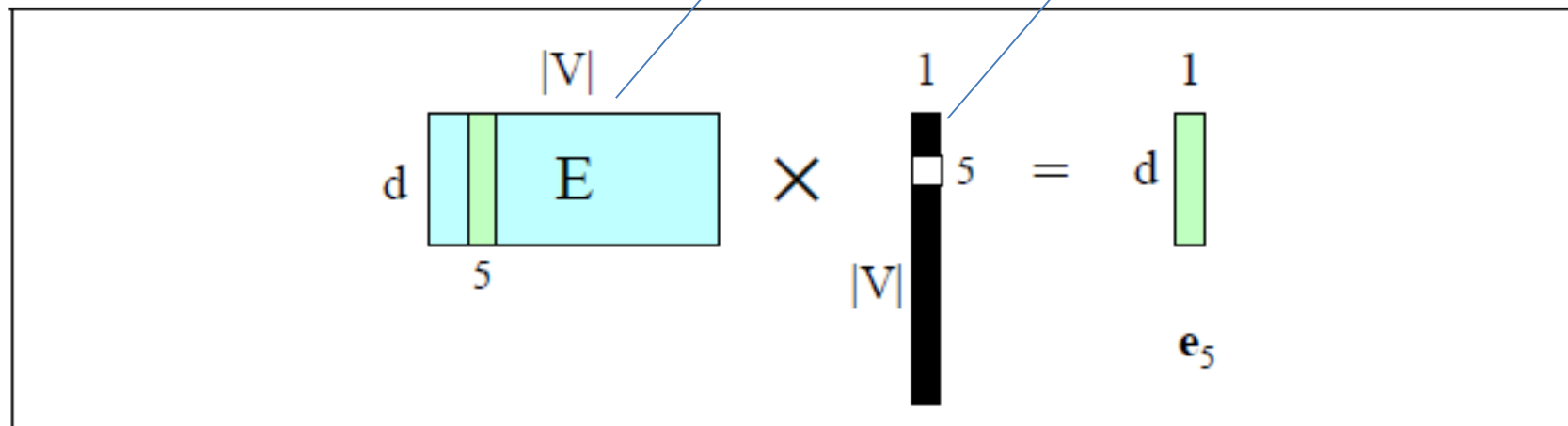


Figure 7.12 Selecting the embedding vector for word V_5 by multiplying the embedding matrix E with a one-hot vector with a 1 in index 5.

Why Neural LMs work better than N-gram LMs



embeddings

- Neural language models represent words in this prior context by their embeddings, rather than just by their word identity as used in n-gram language models.
- Using embeddings allows neural language models to generalize better to unseen data.

Training data:

- We've seen: I have to make sure that the cat gets fed.
- Never seen: dog gets fed

Test data:

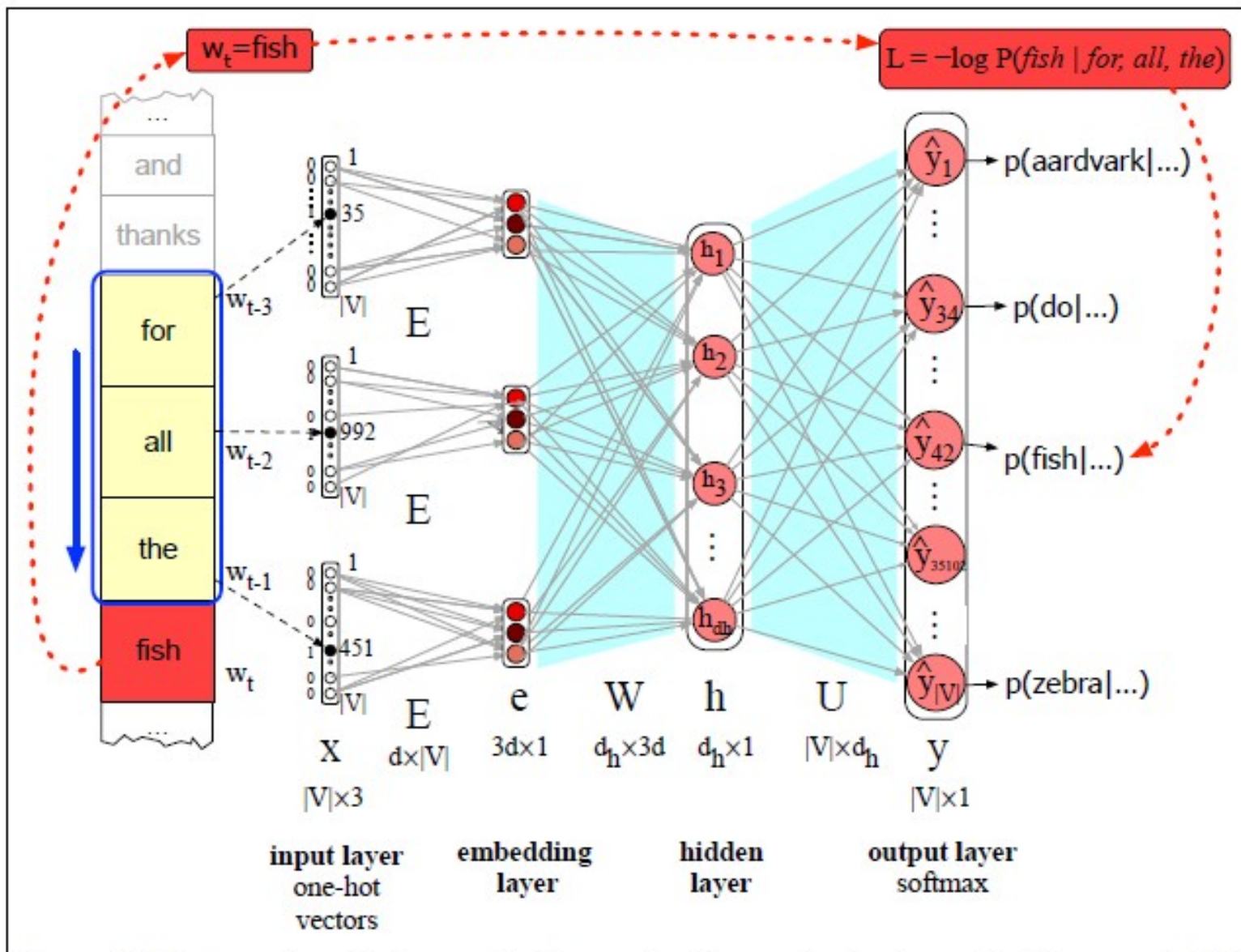
- I forgot to make sure that the dog gets ____
- N-gram LM can't predict "fed"!
- Neural LM can use similarity of "cat" and "dog" embeddings to generalize and predict "fed" after dog

Training the neural language model



- Two ways:
- Freeze the embedding layer E with initial word2vec values.
 - Freezing means we use **word2vec** or **some other pretraining algorithm** to compute the initial embedding matrix E, and then hold it constant while we only Modify W, U, and b, i.e., we don't update E during language model training
- Learn the embeddings simultaneously with training the network.
 - Useful when the task the network is designed for (like sentiment classification, translation, or parsing) places strong constraints on what makes a good representation for words.

Training the neural language model



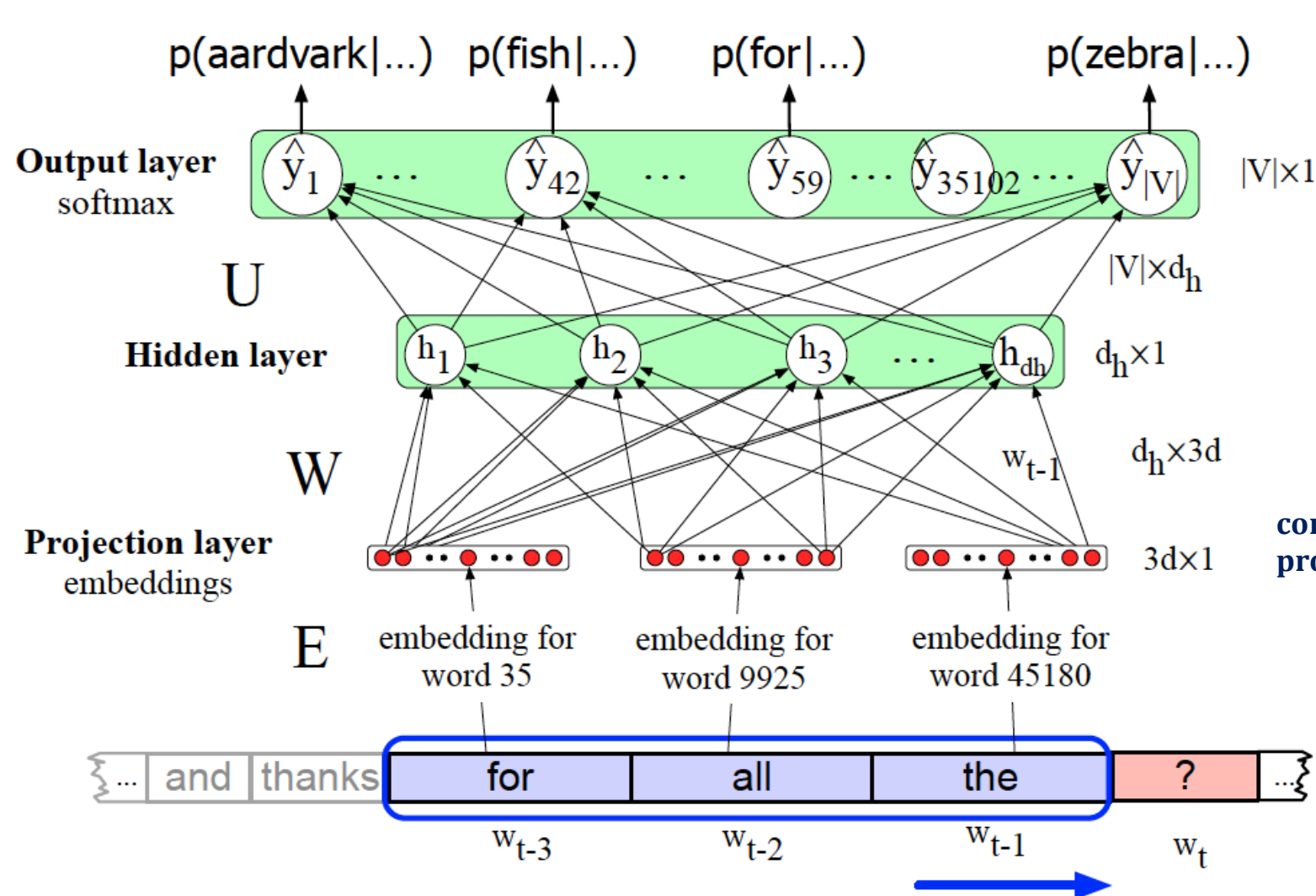
$$\theta = E, W, U, b.$$

one single embedding dictionary E that's shared among one-hot vectors

Training the parameters to minimize loss will result both in an algorithm for **language modeling (a word predictor)** but also a **new set of embeddings E** that can be used as word representations for other tasks.

Figure 7.18 Learning all the way back to embeddings. Again, the embedding matrix E is

Neural Language Model



Equations:

$$e = [E_{x_{t-3}}; E_{x_{t-2}}; E_{x_{t-1}}]$$

$$h = \sigma(We + b)$$

$$z = Uh$$

$$\hat{y} = \text{softmax}(z)$$

concatenate 3 embeddings for the 3 context words to produce the embedding layer e

Training the neural language model



- Input a very long text, concatenating all the sentences
- Starting with random weights
- Iteratively moving through the text predicting each word w_t
- For language modeling, the classes are the words in the vocabulary,
 $y_i - \text{hat} =$
- At each word w_t , we use the cross-entropy (negative log likelihood) loss

$$L_{CE}(\hat{y}, y) = -\log \hat{y}_i, \quad (\text{where } i \text{ is the correct class}) \quad \Rightarrow \quad L_{CE} = -\log p(w_t | w_{t-1}, \dots, w_{t-n+1})$$

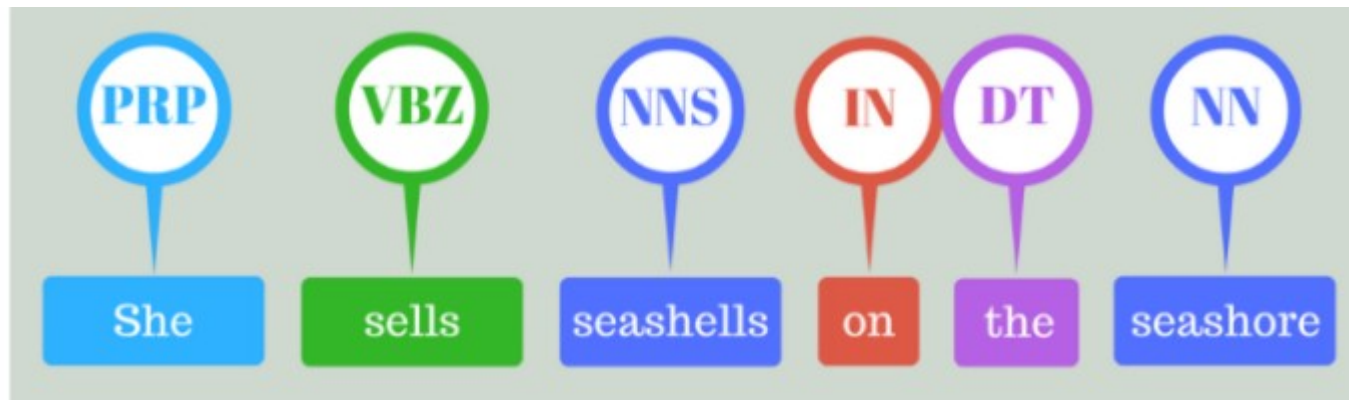
- Gradient Descent update: $\theta^{s+1} = \theta^s - \eta \frac{\partial [-\log p(w_t | w_{t-1}, \dots, w_{t-n+1})]}{\partial \theta}$
- This gradient can be computed in any standard neural network framework which will then backpropagate through $\theta = E, W, U, b$.

:POS tagging

- ❖ What is Part of speech tagging
- ❖ Why POS tagging is required
- ❖ Application
- ❖ (Mostly) English Word Classes
- ❖ Tag set
 - Penn tree bank
- ❖ Part-of-Speech Tagging
- ❖ Different approaches
 - Rule based
 - Stochastic based
 - ❑ HMM Part-of-Speech Tagging
 - Hybrid system

What is POS Tagging

- The process of assigning a part-of-speech or lexical class marker to each word in a sentence (and all sentences in a collection).



Process



❖ List of all possible tag for each word in sentence

❖ Eg:

"People jump high".

People : Noun/Verb

jump : Noun/Verb

high : Noun/Verb/Adjective

❖ Choose best suitable tag sequence

start with probabilities.

Why POS?

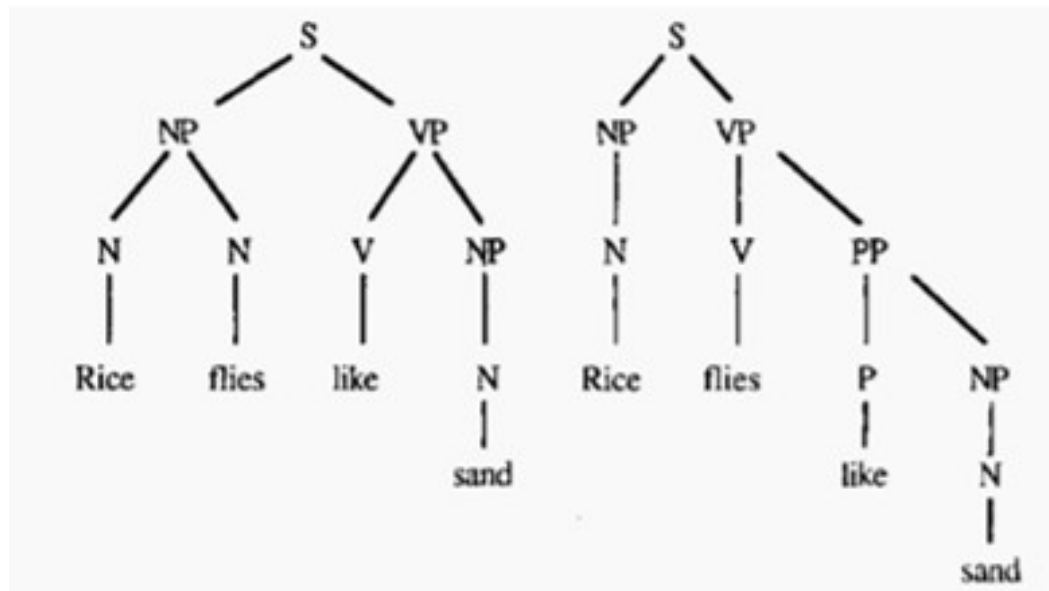
- ❖ First step in many applications
- ❖ POS tell us a lot about word
- ❖ Pronunciation depends on POS
- ❖ To find named entities
- ❖ Stemming

Application of POS

❖ Parsing

Recovering syntactic structures requires correct POS tags

Eg:



Information retrieval

❖ Word Disambiguation

-ability to determine which meaning of word is activated by the use of word in a particular context.

Eg:

- I can hear **bass** sound.
- He likes to eat grilled **bass**.

Question answering system

- ❖ automatically answer questions posed by humans in a natural language.
- ❖ Eg: Does he eat bass?

Why POS tagging is hard?

❖ Ambiguity

Eg:

- He will race/VB the car.
 - When will the race/NOUN end?
 - The boat floated/VBD ... down the river sank
- ❖ Average of ~2 parts of speech for each word

Parts of Speech

- 8 (ish) traditional parts of speech
 - Noun, verb, adjective, preposition, adverb, article, interjection, pronoun, conjunction, etc
 - Called: parts-of-speech, lexical categories, word classes, morphological classes, lexical tags...
 - Lots of debate within linguistics about the number, nature, and universality of these
 - We'll completely ignore this debate.

POS examples

- N noun *chair, bandwidth, pacing*
- V verb *study, debate, munch*
- ADJ adjective *purple, tall, ridiculous*
- ADV adverb *unfortunately, slowly*
- P preposition *of, by, to*
- PRO pronoun *I, me, mine*
- DET determiner *the, a, that, those*

POS Tagging

- Assigning label to something or someone
- The process of assigning a part-of-speech to each word in a collection.

<u>WORD</u>	<u>tag</u>
the	DET
koala	N
put	V
the	DET
keys	N
on	P
the	DET
table	N

Open and Closed Classes

- Open class: new ones can be created all the time
 - English has 4: Nouns, Verbs, Adjectives, Adverbs
 - Many languages have these 4, but not all!
- Closed class: a small fixed membership
 - Prepositions: of, in, by, ...
 - Auxiliaries: may, can, will had, been, ...
 - Pronouns: I, you, she, mine, his, them, ...
 - Usually **function words** (short common words which play a role in grammar)

Closed Class Words

Examples:

- prepositions: *on, under, over, ...*
- particles: *up, down, on, off, ...*
- determiners: *a, an, the, ...*
- pronouns: *she, who, I, ..*
- conjunctions: *and, but, or, ...*
- auxiliary verbs: *can, may should, ...*
- numerals: *one, two, three, third, ...*

Tagsets

- A set of all POS tags used in a corpus is called a tag set
- There are various **standard tag sets** to choose from; some have a lot more tags than others
- The choice of tagset is based on the application
- Accurate tagging can be done with even large tag sets

Penn tree bank

- ❖ Background
 - From the early 90s
 - Developed at the University of Pennsylvania
 - (Marcus, Santorin and Marcinkiewicz 1993)
- ❖ Size
 - 40000 training sentences
 - 2400 test sentences
 - Genre
 - Mostly wall street journal news stories and some spoken conversations
- ❖ Importance
 - Helped launch modern automatic parsing methods.

Penn TreeBank POS Tagset



Tag	Description	Example	Tag	Description	Example
CC	coordin. conjunction	<i>and, but, or</i>	SYM	symbol	<i>+, %, &</i>
CD	cardinal number	<i>one, two, three</i>	TO	“to”	<i>to</i>
DT	determiner	<i>a, the</i>	UH	interjection	<i>ah, oops</i>
EX	existential ‘there’	<i>there</i>	VB	verb, base form	<i>eat</i>
FW	foreign word	<i>mea culpa</i>	VBD	verb, past tense	<i>ate</i>
IN	preposition/sub-conj	<i>of, in, by</i>	VBG	verb, gerund	<i>eating</i>
JJ	adjective	<i>yellow</i>	VCN	verb, past participle	<i>eaten</i>
JJR	adj., comparative	<i>bigger</i>	VBP	verb, non-3sg pres	<i>eat</i>
JJS	adj., superlative	<i>wildest</i>	VBZ	verb, 3sg pres	<i>eats</i>
LS	list item marker	<i>1, 2, One</i>	WDT	wh-determiner	<i>which, that</i>
MD	modal	<i>can, should</i>	WP	wh-pronoun	<i>what, who</i>
NN	noun, sing. or mass	<i>llama</i>	WP\$	possessive wh-	<i>whose</i>
NNS	noun, plural	<i>llamas</i>	WRB	wh-adverb	<i>how, where</i>
NNP	proper noun, singular	<i>IBM</i>	\$	dollar sign	<i>\$</i>
NNPS	proper noun, plural	<i>Carolinas</i>	#	pound sign	<i>#</i>
PDT	predeterminer	<i>all, both</i>	“	left quote	<i>‘ or “</i>
POS	possessive ending	<i>’s</i>	”	right quote	<i>’ or ”</i>
PRP	personal pronoun	<i>I, you, he</i>	(	left parenthesis	<i>[, (, {, <</i>
PRP\$	possessive pronoun	<i>your, one’s</i>	)	right parenthesis	<i>],), }, ></i>
RB	adverb	<i>quickly, never</i>	,	comma	<i>,</i>
RBR	adverb, comparative	<i>faster</i>	.	sentence-final punc	<i>. ! ?</i>
RBS	adverb, superlative	<i>fastest</i>	:	mid-sentence punc	<i>: ; ... --</i>
RP	particle	<i>up, off</i>			

Just for Fun...



- Using Penn Treebank tags, tag the following sentence from the Brown Corpus:
- The grand jury commented on a number of other topics.

Just for Fun...



- Using Penn Treebank tags, tag the following sentence from the Brown Corpus:
- The/DT grand/JJ jury/NN commented/VBD on/IN a/DT number/NN of/IN other/JJ topics/NNS ./.

POS Tagging-

Choosing a Tag set

- ❖ There are so many parts of speech, potential distinctions we can draw
- ❖ To do POS tagging, we need to choose a standard set of tags to work with
- ❖ Could pick very coarse tagsets
 - ❖ N, V, Adj, Adv.
- ❖ More commonly used set is finer grained, the “Penn TreeBank tagset”, 45 tags
 - ❖ PRP\$, WRB, WP\$, VBG

Approaches to POS Tagging

❖ Rule-based Approach

- Uses handcrafted sets of rules to tag input sentences

❖ Statistical approaches

- Use training corpus to compute probability of a tag in a context-HMM tagger

❖ Hybrid systems (e.g. Brill's transformation-based learning)

Rule based POS tagging

- ❖ **First stage** – In the first stage, it uses a dictionary to assign each word a list of potential parts-of-speech.
- ❖ **Second stage** – In the second stage, it uses large lists of hand-written disambiguation rules to sort down the list to a single part-of-speech for each word.

Session Content

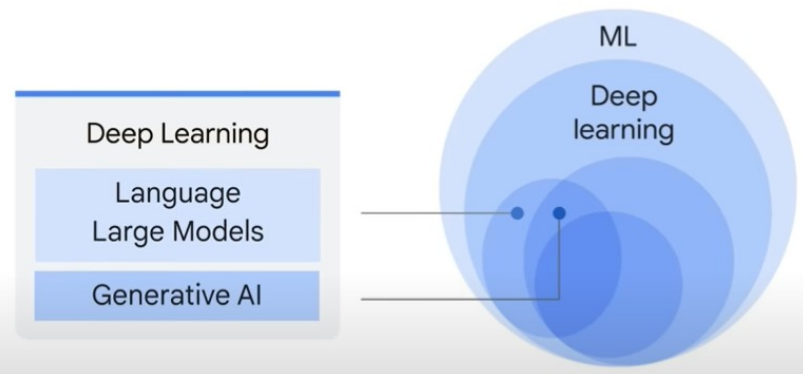


- Introduction to LLM
- Prompt engineering
- Generative AI
- Large language model (LLM) enhancement

Large Language Models



Large Language Models (LLMs) also intersects with **Generative AI**



Large Language Models



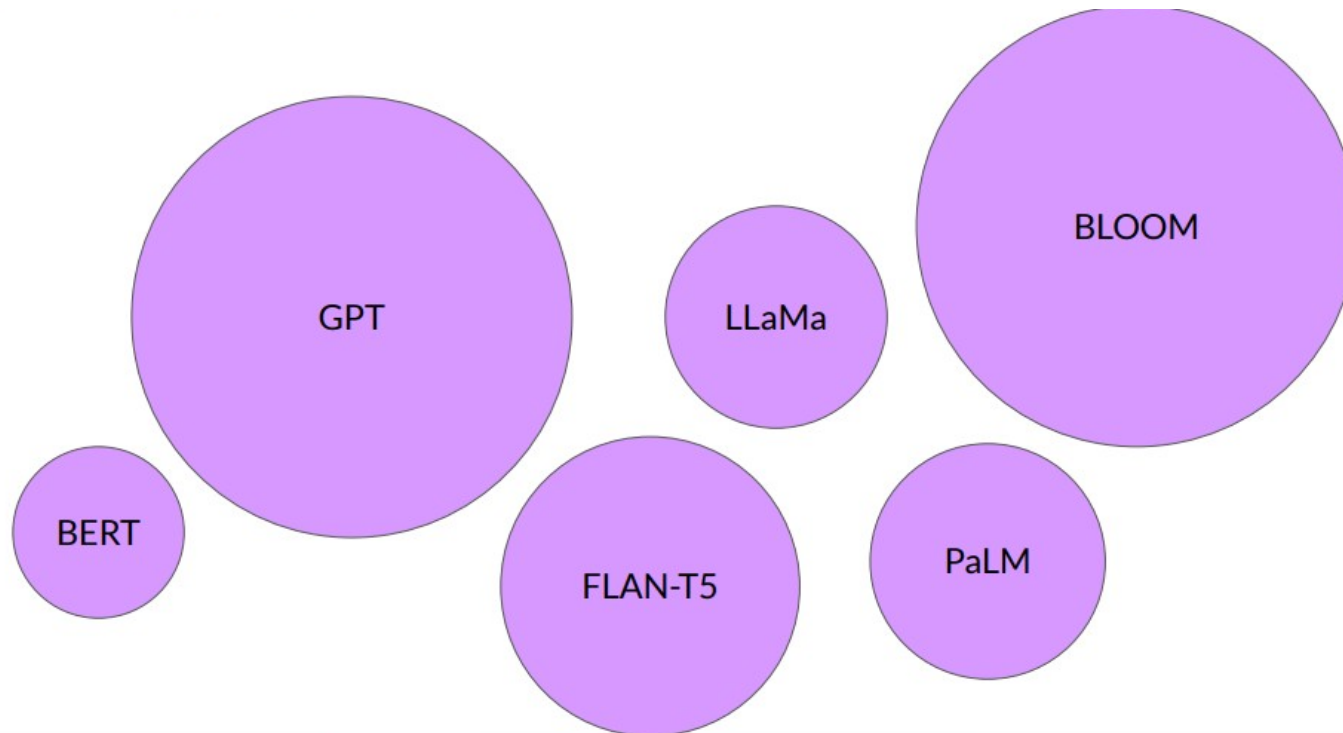
Large, general-purpose language models can be pre-trained and then fine-tuned for specific purposes

Large Language Models



- ✓ Large
 - Large training dataset
 - Large number of parameters
- ✓ General purpose
 - Commonality of human languages
 - Resource restriction
- ✓ Pre-trained and fine-tuned

Large Language Models

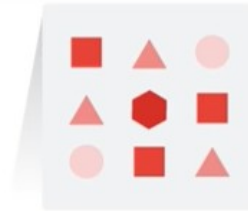


Benefits of using Large Language Models



01

A single model can be used for different tasks



02

The fine-tune process requires minimal filed data



03

The performance is continuously growing with more data and parameters

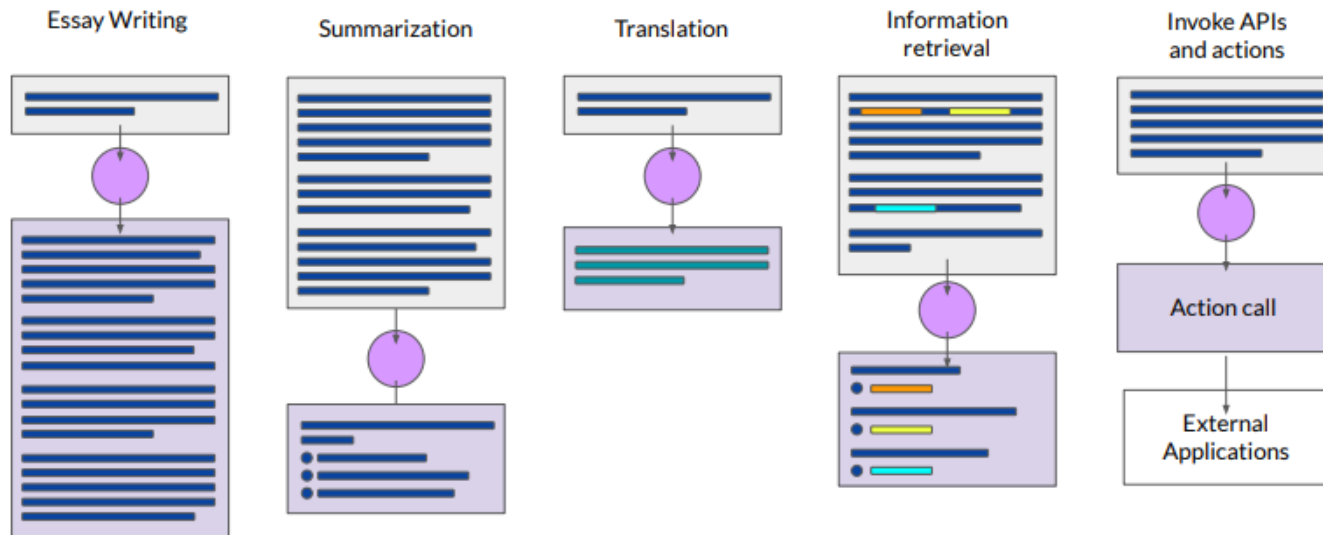
LLM Development (using pre-trained APIs)

- NO ML expertise needed
- NO training examples
- NO need to train a model
- Thinks about prompt design

Traditional ML Development

- YES ML expertise needed
- YES training examples
- YES need to train a model
- YES compute time +
+ hardware
- Thinks about minimizing
a loss function

LLM Use Cases



What is prompt engineering



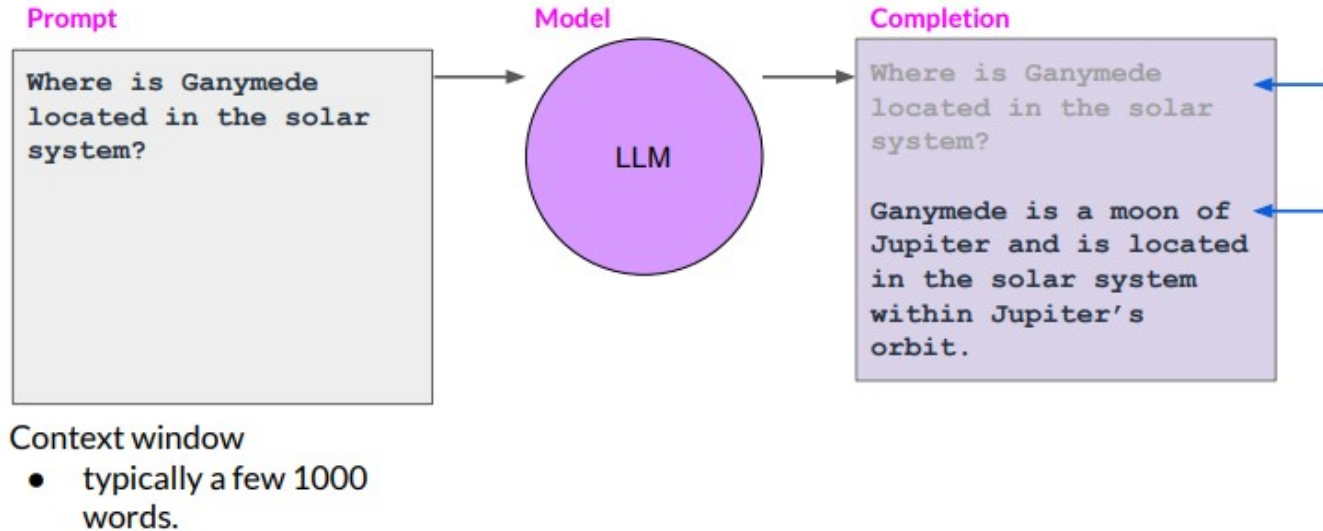
What is a prompt?

A prompt refers to the input or query provided to the AI model to generate a response. It can be a sentence, a question, or a combination of multiple sentences. The prompt sets the context and guides the model in understanding the desired task or the information it needs to generate its response.

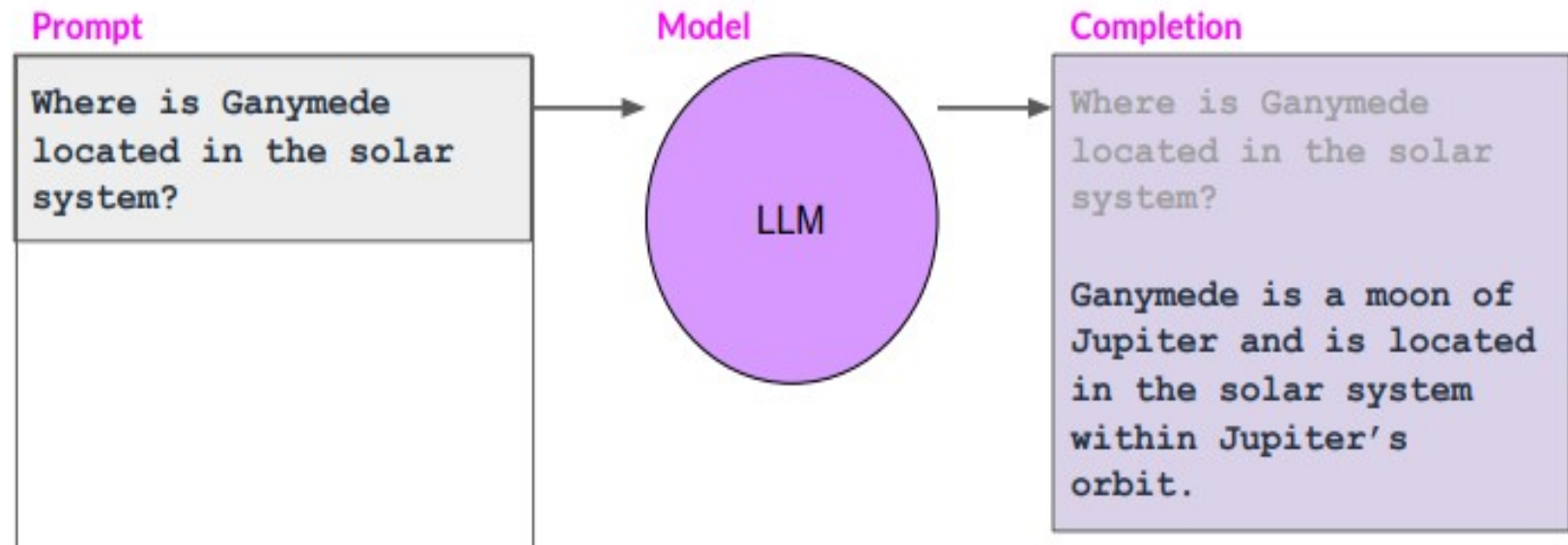
What is prompt engineering?

Prompt engineering refers to the practice of crafting prompts or input text to LLMs in a way that optimizes their performance and output. It involves carefully selecting and formulating input text to elicit the desired response from the LLM.

Prompts and Completions



Prompting and Prompt Engineering



Context window: typically a few thousand words

In context learning and zero shot inference



Prompt

Classify this review:
I loved this movie!
Sentiment:

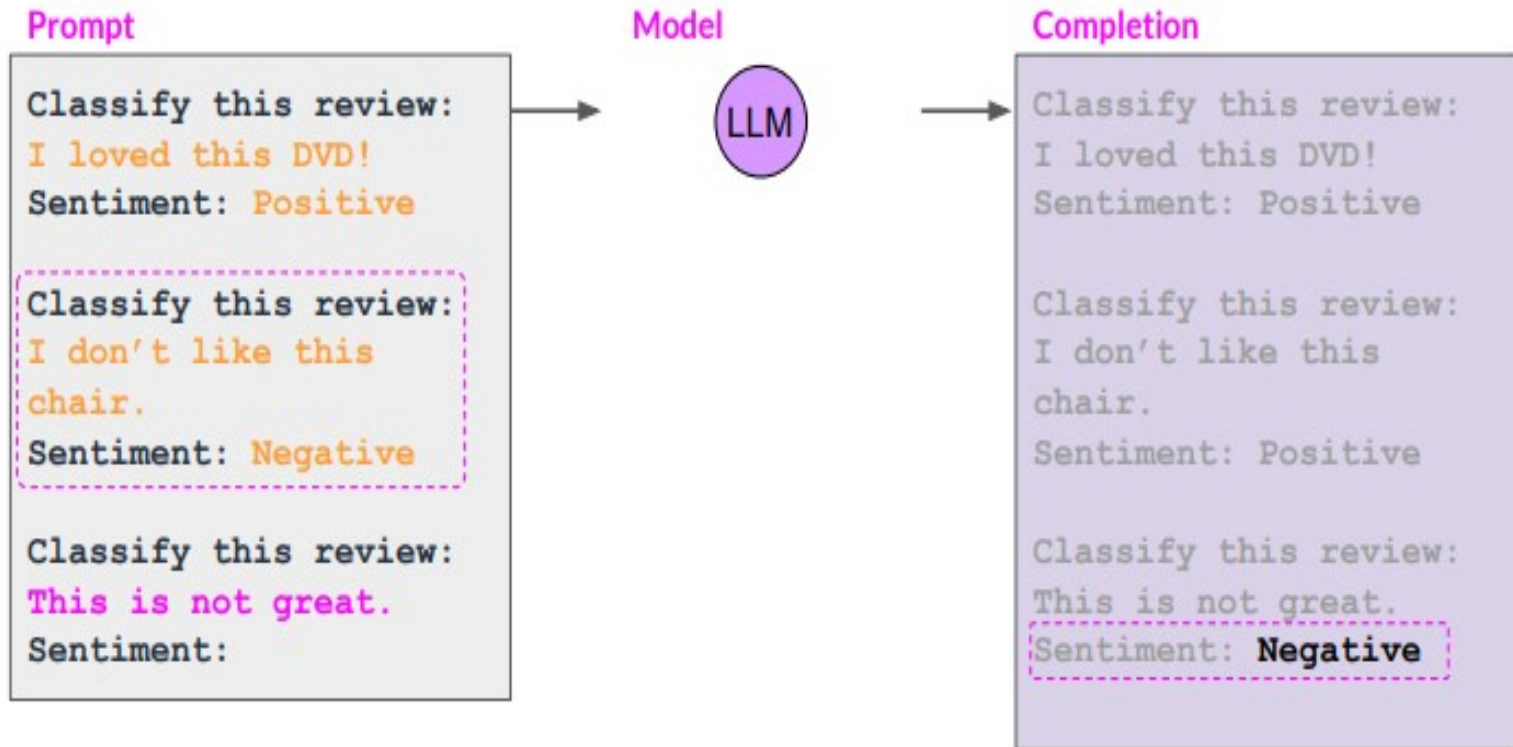
Model



Completion

Classify this review:
I loved this movie!
Sentiment: eived a
very nice book review

In context learning and Few shot inference



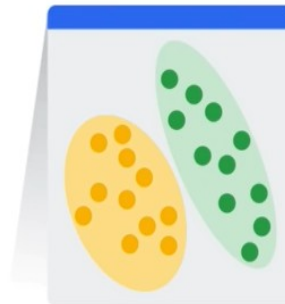
DL Model Types

Deep Learning Model Types



Discriminative

- Used to classify or predict
- Typically trained on a dataset of labeled data
- Learns the relationship between the features of the data points and the labels



Generative

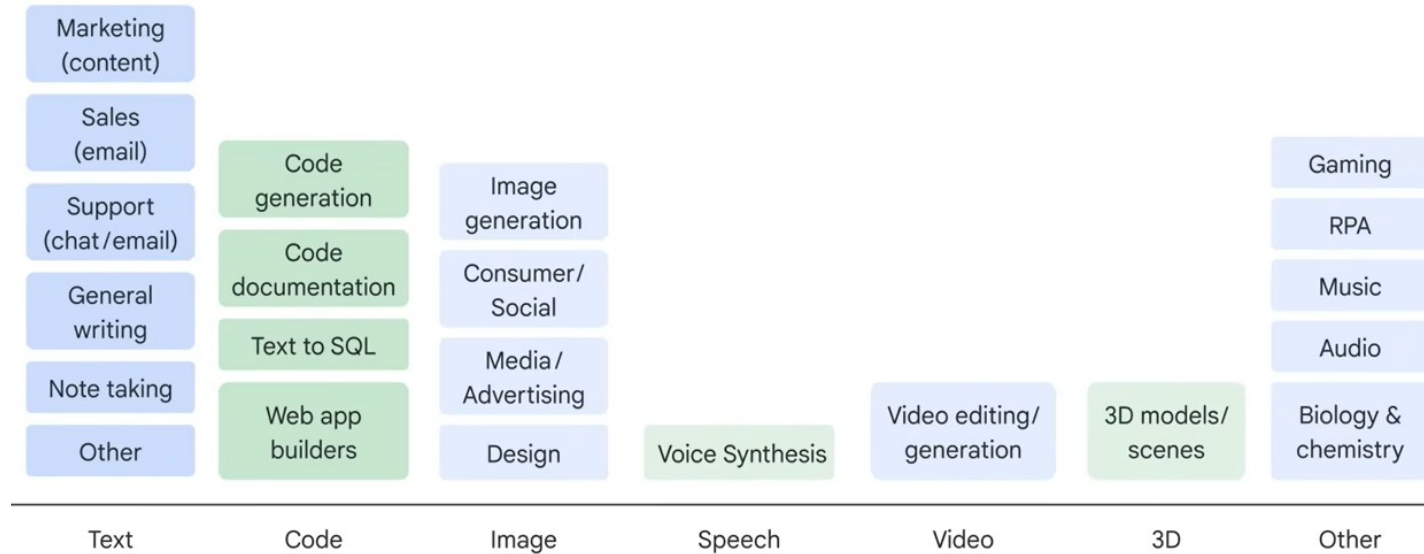
- Generates new data that is similar to data it was trained on
- Understands distribution of data and how likely a given example is
- Predict next word in a sequence

Generative AI



- GenAI is a type of Artificial Intelligence that creates new content based on what it has learned from existing content.
- The process of learning from existing content is called training and results in the creation of a statistical model.
- When given a prompt, GenAI uses this statistical model to predict what an expected response might be—and this generates new content.

Generative AI Applications



Generative AI- AI Assistants

Notification Assistant



Hi there - just a friendly reminder that your insurance policy expires in a month. Make sure to renew it in our member portal.

FAQ Assistant

I need to renew my renters insurance. How much will it be?



You can calculate your renewal price on our website here: xyz.com/renew

Contextual Assistant

I need to renew my renters insurance. How much will it be?



I'd be happy to check for you. Firstly, are you still living in the same apartment?

Yes



Great - so just confirming it's 980 sq ft?

Yes



Thanks! Your new rate from September 1st onwards would be \$10 / month.



Would you like me to renew your policy for you right now?

Sure



Great. I've sent you a confirmation to your email.

Generative AI- AI Assistants

Personalized Assistant

- Assistant knows you much more in detail
- Quickly checks a few final things before giving you a quote tailored to your actual situation.



I can see your details are almost the same, except now you might want coverage for your new laptop. Additional coverage is only \$4 a month more for full coverage. Sound ok?

Sounds good!



Autonomous Organization of Assistants

- Group of AI assistants that know every customer personally
- Eventually run large parts of company operations—from lead generation over marketing, sales, HR, or finance



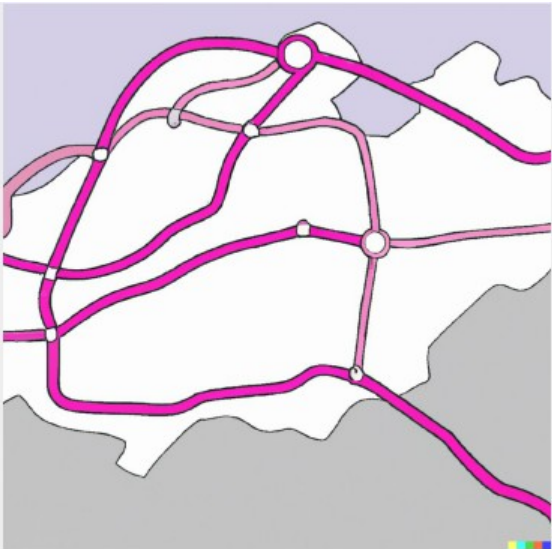
Generative AI- PaintBox

What do you want to create?

**An imaginary subway map
in a coastal city.**

Image dimensions: by (Max 2048)

Generate



Generative AI- CodeAid

```
1 def binary_search(arr, x, l, r):-  
2     if r >= l:  
3         mid = l + (r - l) // 2  
4         if arr[mid] == x:  
5             return mid  
6         elif arr[mid] > x:  
7             return binary_search(arr, x, l, mid - 1)  
8         else:  
9             return binary_search(arr, x, mid + 1, r)  
     else:  
         return -1
```

< 1/2 > Accept

Tab

Real-life challenges in NLP tasks

- Deep learning methods are data-hungry
- >50K data items needed for training
- The distributions of the source and target data must be the same
- Labeled data in the target domain may be limited
- This problem is typically addressed with **transfer learning**

Transfer Learning



- Using a pre-trained model as a starting point for a new task or domain.
- Leverage the knowledge acquired by the pre-trained model on a large dataset and apply it to a related task with a smaller dataset.
- We can benefit from the general features and patterns learned by the pre-trained model, saving time and computational resources.
- Transfer learning involves freezing the pre-trained model's weights and only training the new layers

Ex: image classification, knowledge gained while learning to recognize cars could be applied when trying to recognize trucks

Transductive vs Inductive Transfer Learning

- **Transductive** transfer
 - No labeled target domain data available
 - Focus of most transfer research in NLP
 - E.g. Domain adaptation
- **Inductive** transfer
 - Labeled target domain data available
 - Goal: improve performance on the target task by training on other task(s)
 - Jointly training on >1 task (multi-task learning)
 - Pre-training (e.g. word embeddings)

Applications of Transfer Learning



- Image Classification
- Names Entity Recognition
- Sentiment Analysis
- Cross Lingual Learning
- Gaming
- Image Recognition
- Speech Recognition

Fine Tuning



- Fine-tuning takes it a step further by allowing the pre-trained layers to be updated.
- Beneficial when the new dataset is large enough and similar to the original dataset on which the pre-trained model was trained



Natural Language Processing AIML ZG530



BITS Pilani
Pilani Campus

Prof. Vijayalakshmi Anand
BITS-Pilani

POS tagging(Viterbi ,Maximum entropy model)



- Likelihood Computation:
 - The Forward Algorithm
- Decoding: The Viterbi Algorithm
- Bidirectionality
- Maximum entropy model

Three Problems

Given this framework there are 3 problems that we can pose to an HMM

1. Given an observation sequence, what is the probability of that sequence given a model?
2. Given an observation sequence and a model, what is the most likely state sequence?
3. Given an observation sequence, find the best model parameters for a partially specified model

Problem 1:



Observation Likelihood

- The probability of a observation sequence given a model and state sequence
- Evaluation problem

Problem 2:



- Most probable state sequence given a model and an observation sequence
- Decoding problem

Problem 3:



- Infer the best model parameters, given a partial model and an observation sequence...
 - That is, fill in the A and B tables with the right numbers --
 - the numbers that make the observation sequence most likely
- This is to learn the probabilities!

Solutions

Problem 1: Forward (learn observation sequence)

Problem 2: Viterbi (learn state sequence)

Problem 3: Forward-Backward (learn probabilities)

- An instance of EM (Expectation Maximization)

Example :HMMs for Ice Cream

You are a climatologist in the year 2799 studying global warming

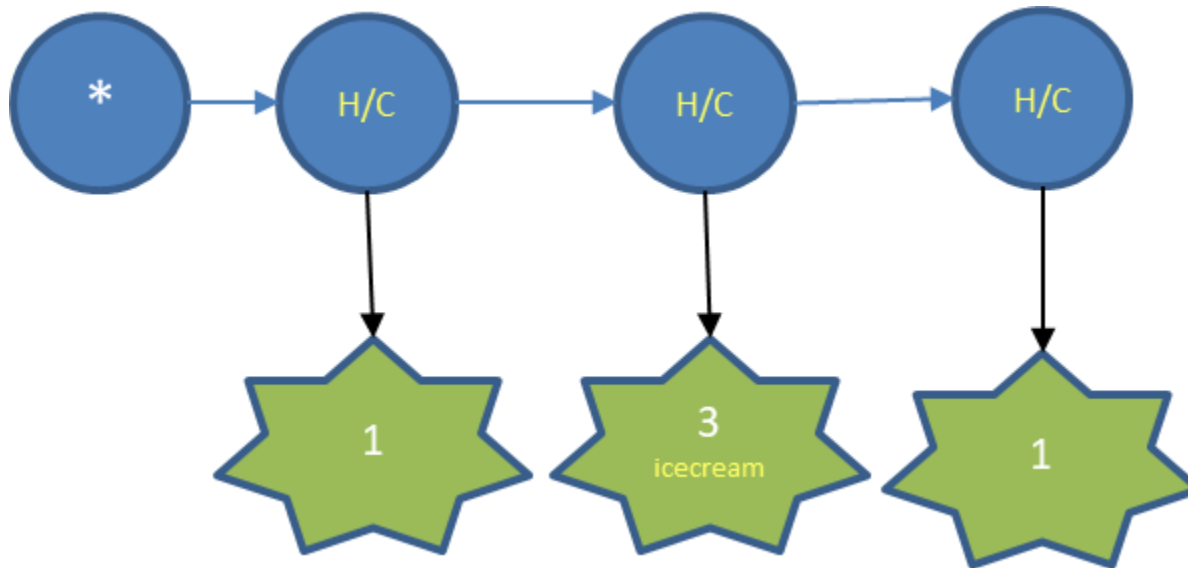
You can't find any records of the weather in Baltimore for summer of 2007

But you find Jason Eisner's diary which lists how many ice creams Jason ate every day that summer



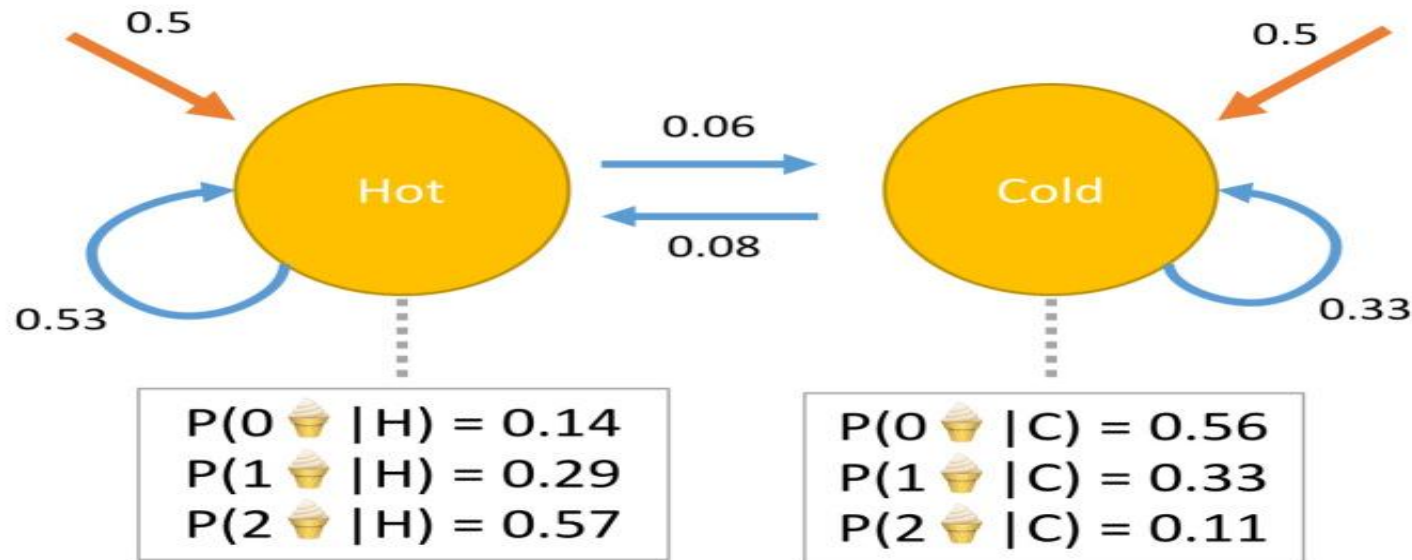
Your job: figure out how hot it was each day

Problem 1



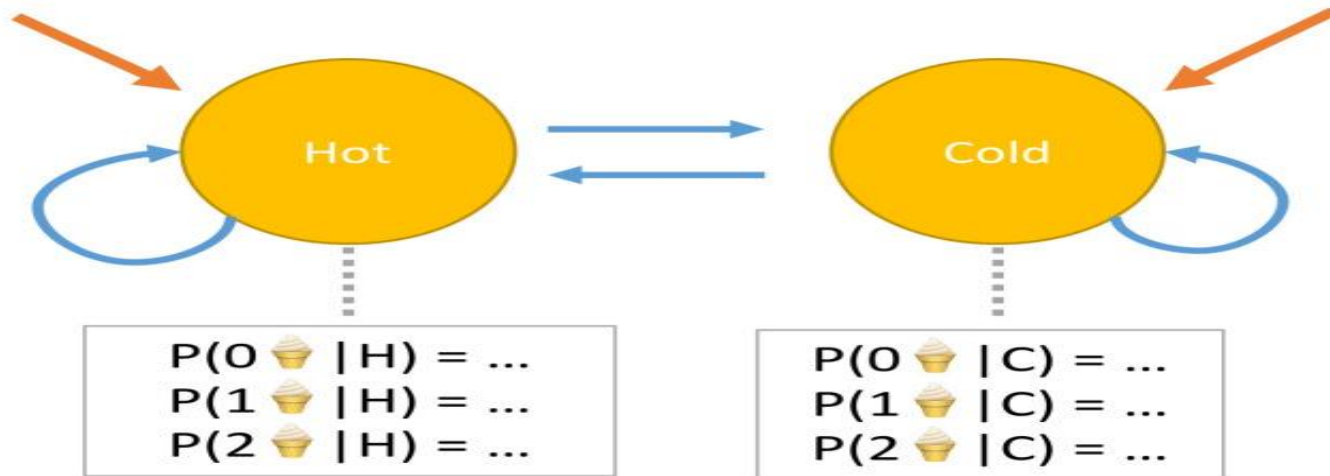
- ❖ 1. Consider all possible 3-day weather sequences [H, H, H], [H, H, C], [H, C, H],
- 2. For each 3-day weather sequence, consider the probability of the ice cream Consumption sequences [131,]
- 3. Add all the probability
- ❖ Not efficient
- ❖ Forward algorithm

Problem2



- 1. Consider all possible 3-day weather sequences $[H, H, H]$, $[H, H, C]$, $[H, C, H]$,
 - 2. For each 3-day weather sequence, consider the probability of the ice cream Consumption sequences $[0, 1, 2]$
 - 3. Pick out the sequence that has the highest probability from step #2.
- Not efficient
 - Viterbi algorithm

Problem3



- **Find :**
 - The start probabilities
 - The transition probabilities
 - Emission probabilities
- Forward –backward algorithm

Viterbi intuition

- We want to compute the joint probability of the observation sequence together with the best state sequence

$$v_t(j) = \max_{q_0, q_1, \dots, q_{t-1}} P(q_0, q_1 \dots q_{t-1}, o_1, o_2 \dots o_t, q_t = j | \lambda)$$

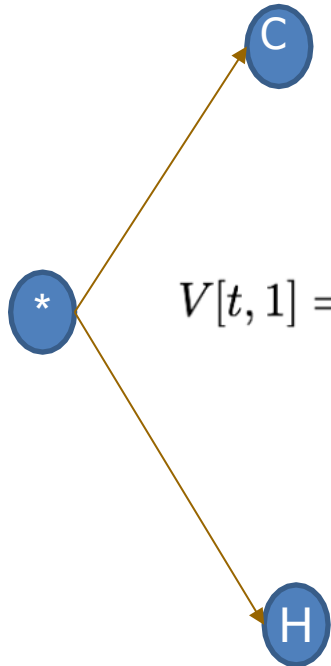
$$v_t(j) = \max_{i=1}^N v_{t-1}(i) a_{ij} b_j(o_t)$$

Hidden Markov Model



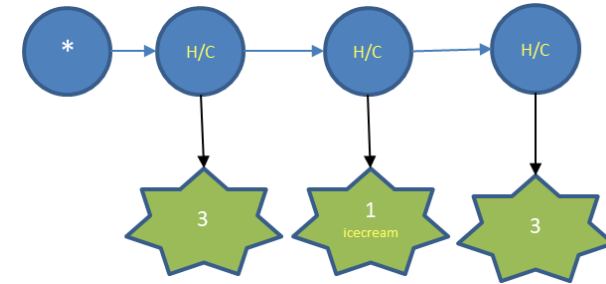
Example -1 – Viterbi Algorithm - Initialization

$$P(C) * P(3|C) = 0.2 * 0.1 = 0.02$$



$$V[t, 1] = P(t|start)P(w_1|t)$$

$$P(H) * P(3|H) = 0.8 * 0.4 = 0.32$$



	Hot	Cold
< S >	0.8	0.2

A	Hot	Cold
Hot	0.6	0.4
Cold	0.5	0.5

B	1	2	3
Hot	.2	.4	.4
Cold	.5	.4	.1

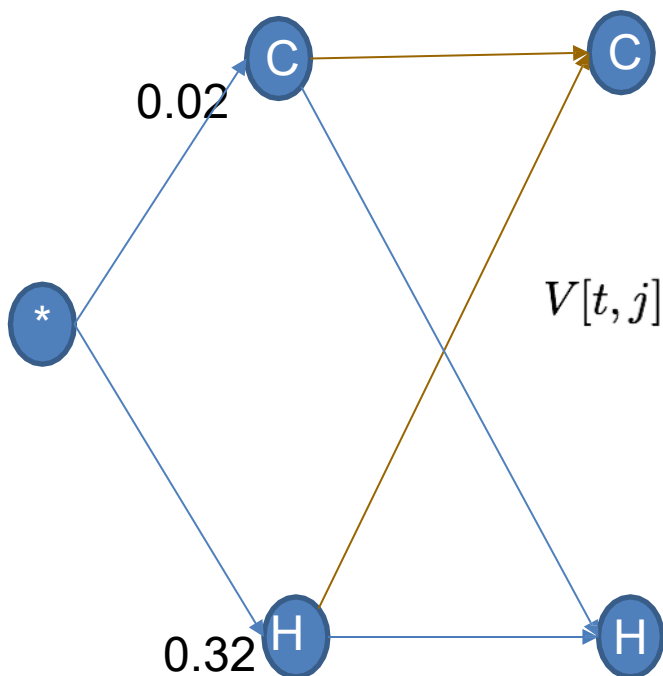
Hidden Markov Model



Example -1 – Viterbi Algorithm - Recursion

$$P(C)*P(C|C)*P(1|C) = 0.02*0.5*0.5 = 0.005$$

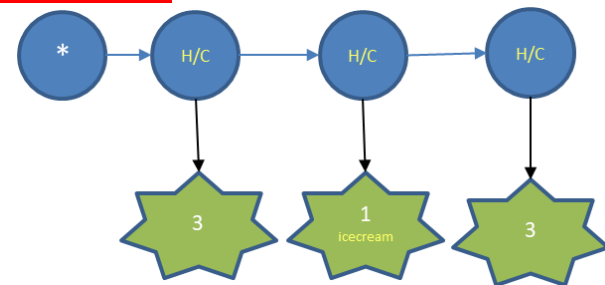
$$P(H)*P(C|H)*P(1|C) = 0.32*0.4*0.5 = \mathbf{0.064}$$



$$V[t, j] = \max_{t'} V[t', j - 1] P(t|t') P(w_j|t)$$

$$P(C)*P(H|C)*P(1|H) = 0.02*0.5*0.2 = 0.002$$

$$P(H)*P(H|H)*P(1|H) = 0.32*0.6*0.2 = \mathbf{0.0384}$$



	Hot	Cold	
<S	0.8	0.2	
>			
A	Hot	Cold	
Hot	0.6	0.4	
Cold	0.5	0.5	
B	1	2	3
Hot	.2	.4	.4
Cold	.5	.4	.1

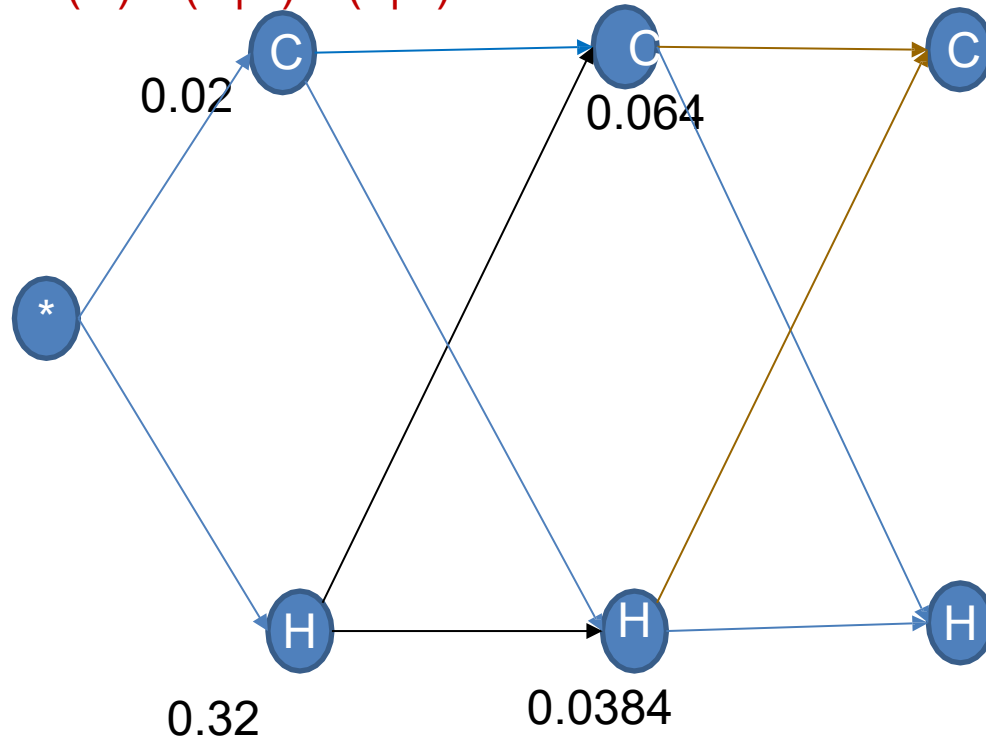
Hidden Markov Model



Example -1 – Viterbi Algorithm – Termination through Back Trace

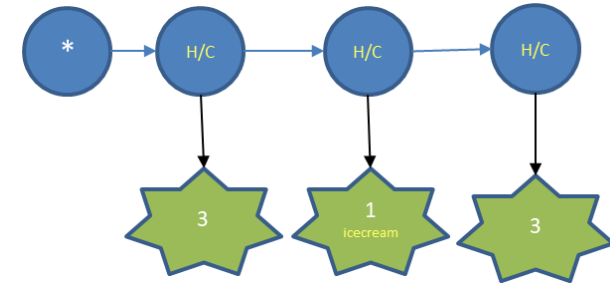
$$P(C)*P(C|C)*P(3|C) = 0.064*0.5*0.1 = \mathbf{0.032}$$

$$P(H)*P(C|H)*P(3|C) = 0.0384*0.4*0.1 = \mathbf{0.0015}$$



$$P(C)*P(H|C)*P(3|H) = 0.064 * 0.5 * 0.2 = \mathbf{0.0064}$$

$$P(H)*P(H|H)*P(3|H) = 0.0384 * 0.6 * 0.2 = \mathbf{0.0046}$$



Best Sequence:
Hot ☐ Cold ☐

	Hot	Cold	
<S	0.8	0.2	
>			
A	Hot	Cold	
Hot	0.6	0.4	
Cold	0.5	0.5	
B	1	2	3
Hot	.2	.4	.4
Cold	.5	.4	.1
d			

Hidden Markov Model



Example -1 – Viterbi Algorithm

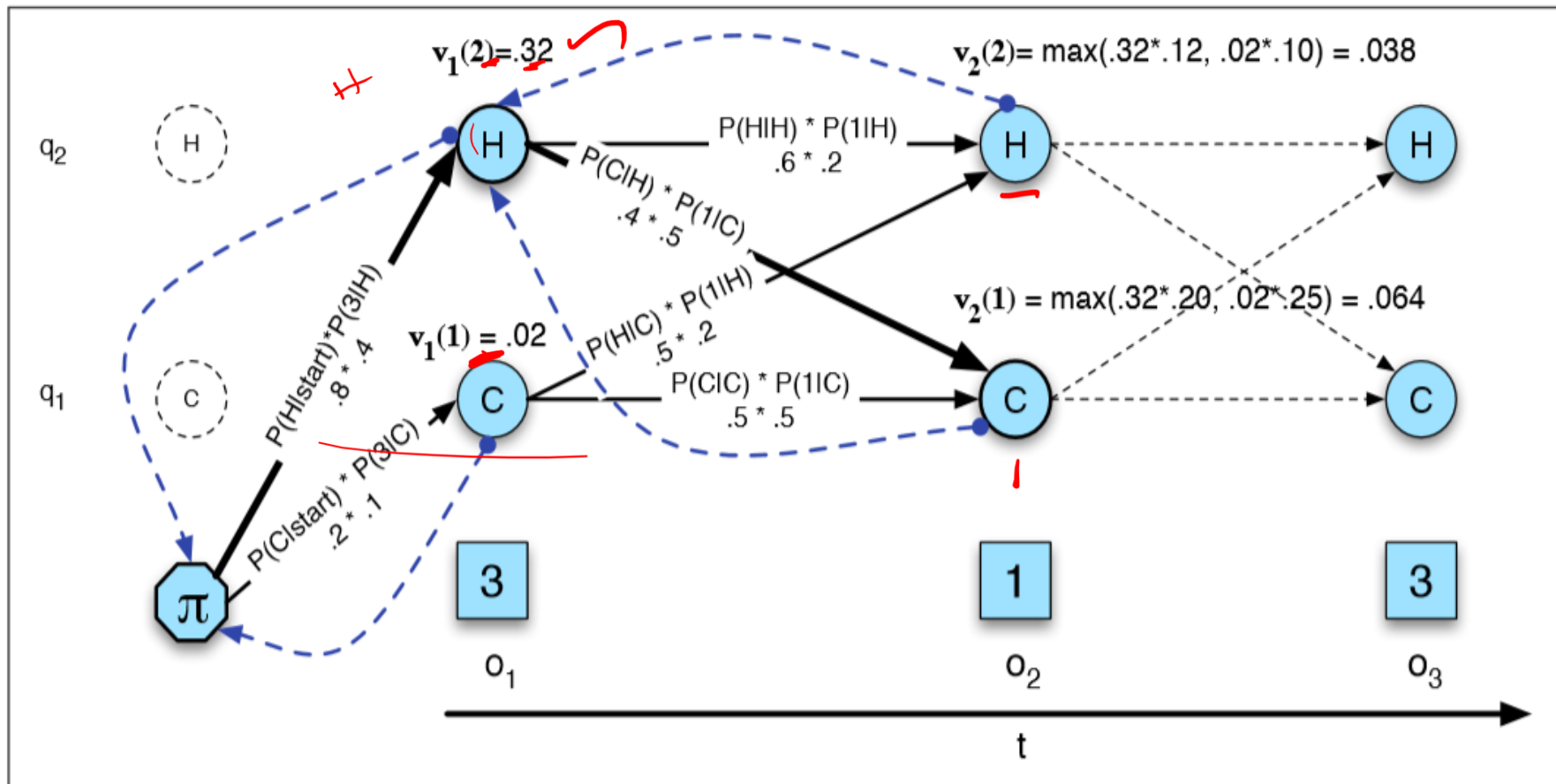


Figure A.10 The Viterbi backtrace. As we extend each path to a new state account for the next observation, we keep a backpointer (shown with broken lines) to the best path that led us to this state.

Source Credit : Speech and Language Processing - Jurafsky and Martin

Hidden Markov Model



POS Tagging – Viterbi Algorithm

function VITERBI(*observations* of len T , *state-graph* of len N) **returns** *best-path*

create a path probability matrix $viterbi[N+2, T]$

for each state s **from** 1 **to** N **do** ; initialization step

$viterbi[s, 1] \leftarrow a_{0,s} * b_s(o_1)$

$backpointer[s, 1] \leftarrow 0$

for each time step t **from** 2 **to** T **do** ; recursion step

for each state s **from** 1 **to** N **do**

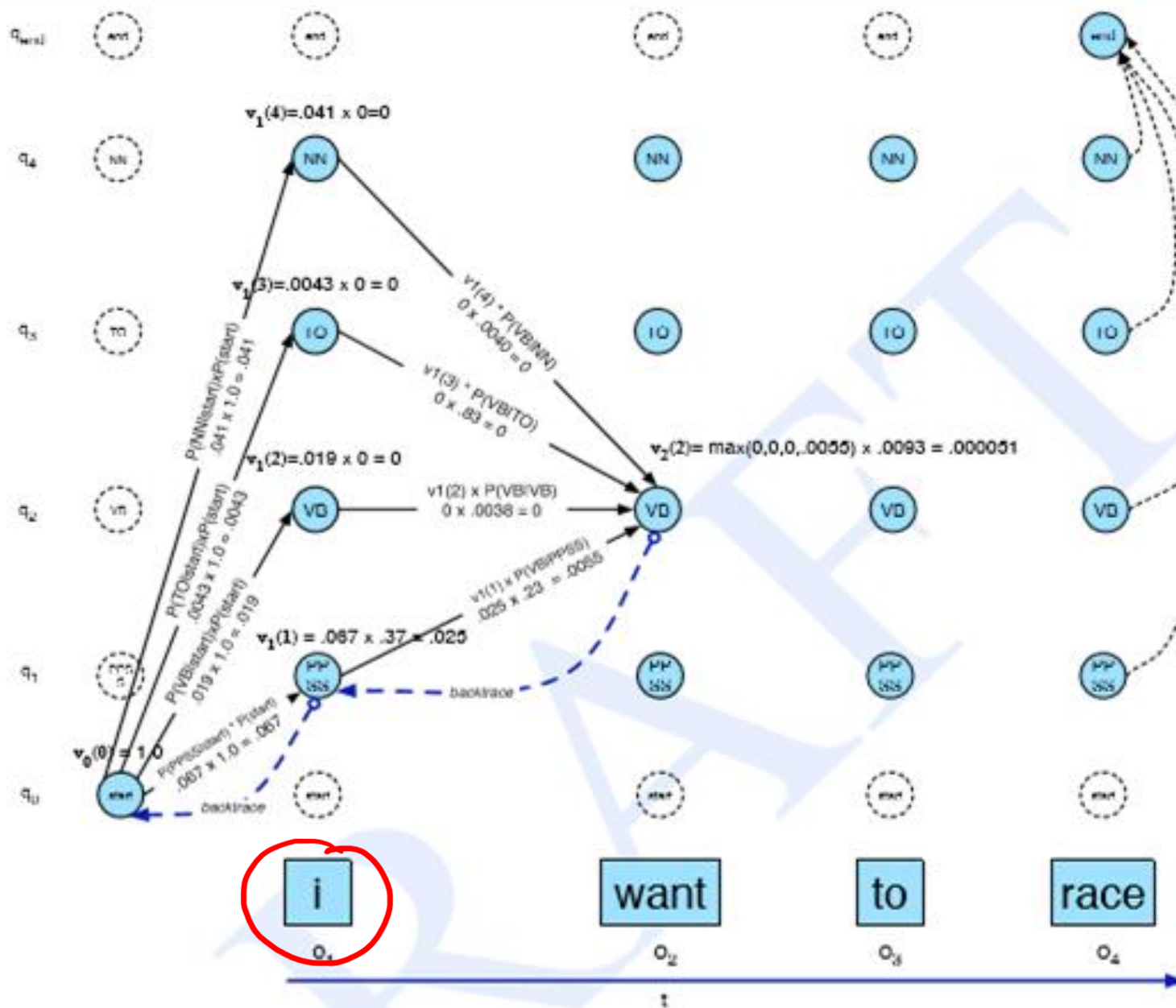
$viterbi[s, t] \leftarrow \max_{s'=1}^N viterbi[s', t-1] * a_{s',s} * b_s(o_t)$

$backpointer[s, t] \leftarrow \operatorname{argmax}_{s'=1}^N viterbi[s', t-1] * a_{s',s}$

$viterbi[q_F, T] \leftarrow \max_{s=1}^N viterbi[s, T] * a_{s,q_F}$; termination step

$backpointer[q_F, T] \leftarrow \operatorname{argmax}_{s=1}^N viterbi[s, T] * a_{s,q_F}$; termination step

return the backtrace path by following backpointers to states back in time from $backpointer[q_F, T]$



Source Credit : Speech and Language Processing - Jurafsky and Martin

Hidden Markov Model



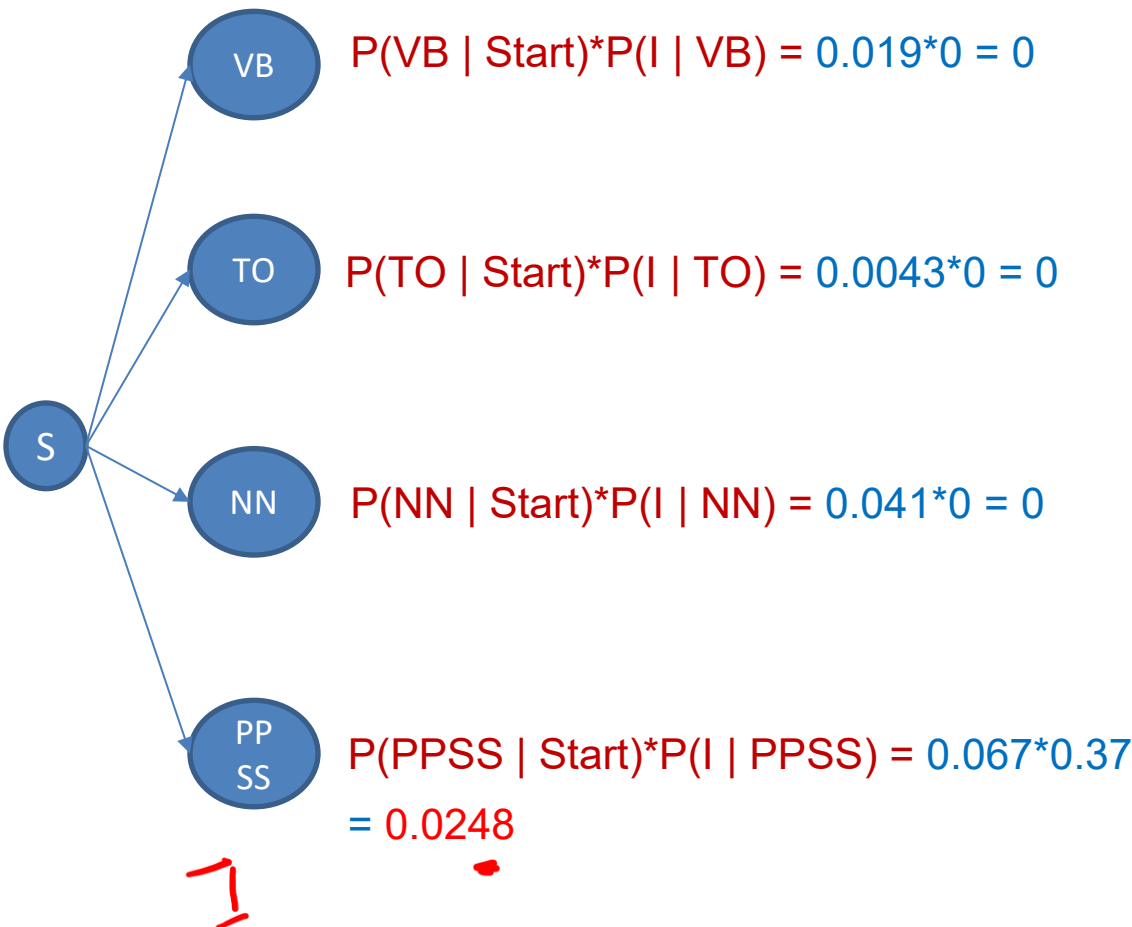
POS Tagging – Example -2 – Viterbi Algorithm - Initialization

$$V[t, 1] = P(t|start)P(w_1|t)$$

	VB	TO	NN	PPSS
<S>	0.019 3	0.004	0.041	0.067

A	VB	TO	NN	PPSS
VB	0.003 8	0.035	0.047	0.0070
TO	0.83	0	0.0004 7	0
NN	0.004 0	0.016	0.087	0.0045
PPS S	0.23	0.0007 9	0.0012	0.00014

B	I	want	to	race
VB	0	0.0093	0	0.00012
TO	0	0	0.99	0
NN	0	0.000054	0	0.00057
PPS S	0.37	0	0	0

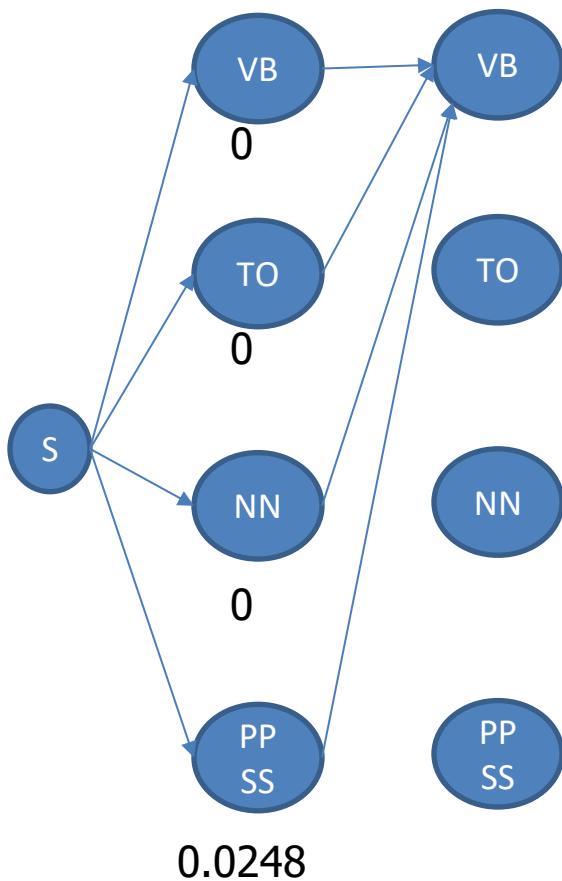


Hidden Markov Model



POS Tagging – Example -2 – Viterbi Algorithm - Recursion

$$V[t, j] = \max_{t'} V[t', j - 1] P(t|t') P(w_j|t)$$



	VB	TO	NN	PPSS
<S>	0.019	0.004 3	0.041	0.067

A	VB	TO	NN	PPSS
VB	0.0038	0.035	0.047	0.0070
TO	0.83	0	0.00047	0
NN	0.0040	0.016	0.087	0.0045
PPSS	0.23	0.00079	0.0012	0.00014

B	I	want	to	race
VB	0	0.0093	0	0.00012
TO	0	0	0.99	0
NN	0	0.00005 4	0	0.00057
PPS	0.37	0	0	0
S				

$$P(\text{VB}) * P(\text{VB} | \text{VB}) * P(\text{want} | \text{VB}) = 0 * 0.0038 * 0.0093 = 0$$

$$P(\text{TO}) * P(\text{VB} | \text{TO}) * P(\text{want} | \text{VB}) = 0 * 0.83 * 0.0093 = 0$$

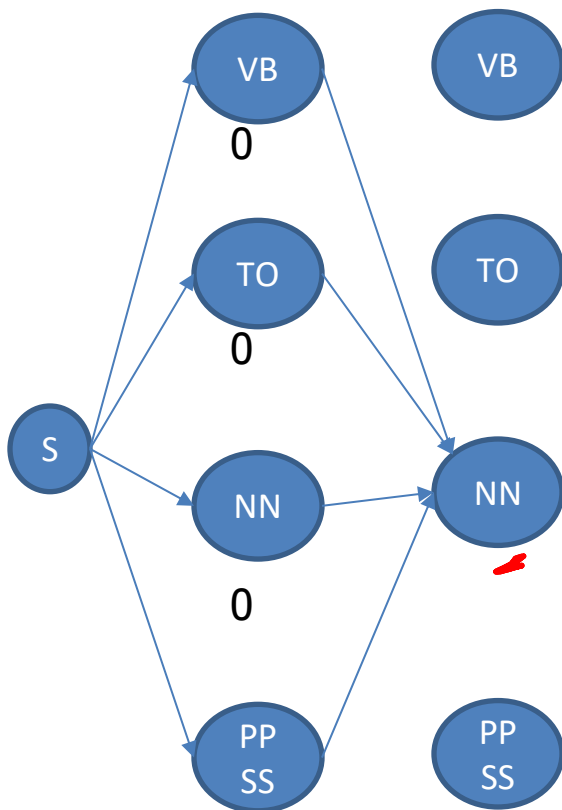
$$P(\text{NN}) * P(\text{VB} | \text{NN}) * P(\text{want} | \text{VB}) = 0 * 0.004 * 0.0093 = 0$$

$$P(\text{PPSS}) * P(\text{VB} | \text{PPSS}) * P(\text{want} | \text{VB}) = 0.0248 * 0.23 * 0.0093 = 0.0005$$

Hidden Markov Model



POS Tagging – Example -2 – Viterbi Algorithm - Recursion



0.0248

$$P(VB) * P(NN | VB) * P(want | NN) = 0 * 0.047 * 0.000054 = 0$$

$$P(TO) * P(NN | TO) * P(want | NN) = 0 * 0.0047 * 0.000054 = 0$$

$$P(NN) * P(NN | NN) * P(want | NN) = 0 * 0.087 * 0.000054 = 0$$

$$P(PPSS) * P(NN | PPSS) * P(want | NN) = 0.0012 * 0.23 * 0.000054 = 0.16 * 10^{-7}$$

	VB	TO	NN	PPSS
<S>	0.019	0.004 3	0.041	0.067

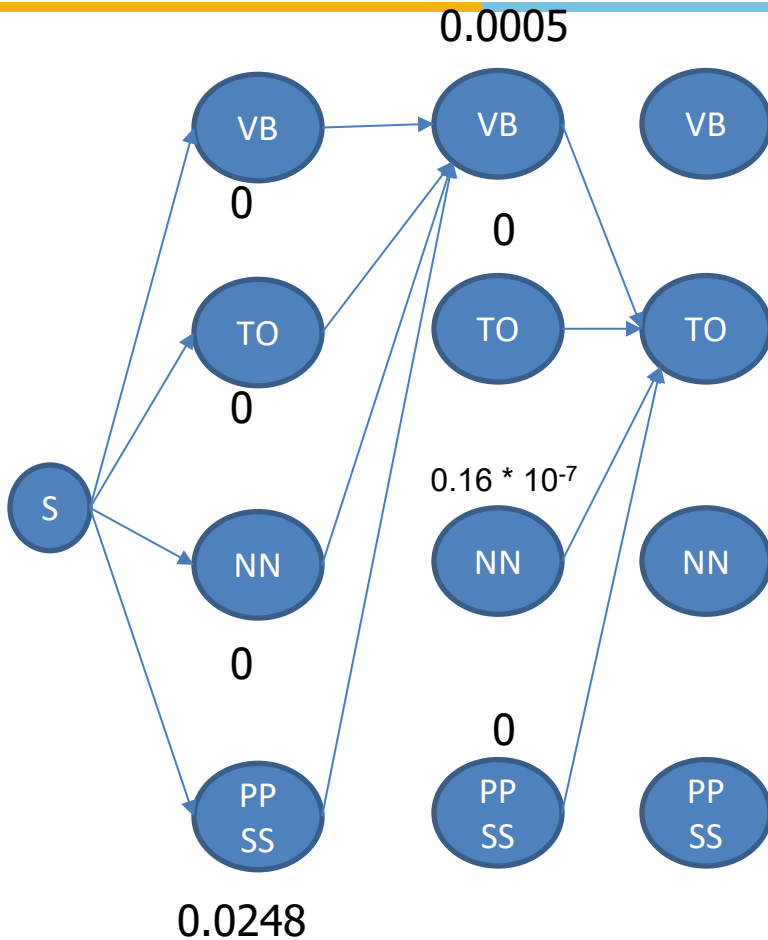
A	VB	TO	NN	PPSS
VB	0.003 8	0.035	0.047	0.0070
TO	0.83	0	0.0004 7	0
NN	0.004 0	0.016	0.087	0.0045
PPS S	0.23	0.0007 9	0.0012	0.00014

B	I	want	to	race
VB	0	0.0093	0	0.00012
TO	0	0	0.99	0
NN	0	0.00005 4	0	0.00057
PPS S	0.37	0	0	0

Hidden Markov Model



POS Tagging – Example -2 – Viterbi Algorithm - Recursion



	VB	TO	NN	PPSS
<S>	0.019	0.004 3	0.041	0.067

A	VB	TO	NN	PPSS
VB	0.0038	0.035	0.047	0.0070
TO	0.83	0	0.00047	0
NN	0.0040	0.016	0.087	0.0045
PPSS	0.23	0.00079	0.0012	0.00014

B	I	want	to	race
VB	0	0.0093	0	0.00012
TO	0	0	0.99	0
NN	0	0.00005 4	0	0.00057
PPS S	0.37	0	0	0

$$P(\text{VB}) * P(\text{TO} | \text{VB}) * P(\text{to} | \text{TO}) = 0.0005 * 0.035 * 0.99 = 0.17 * 10^{-4}$$

$$P(\text{TO}) * P(\text{TO} | \text{TO}) * P(\text{to} | \text{TO}) = 0 * 0 * 0.99 = 0$$

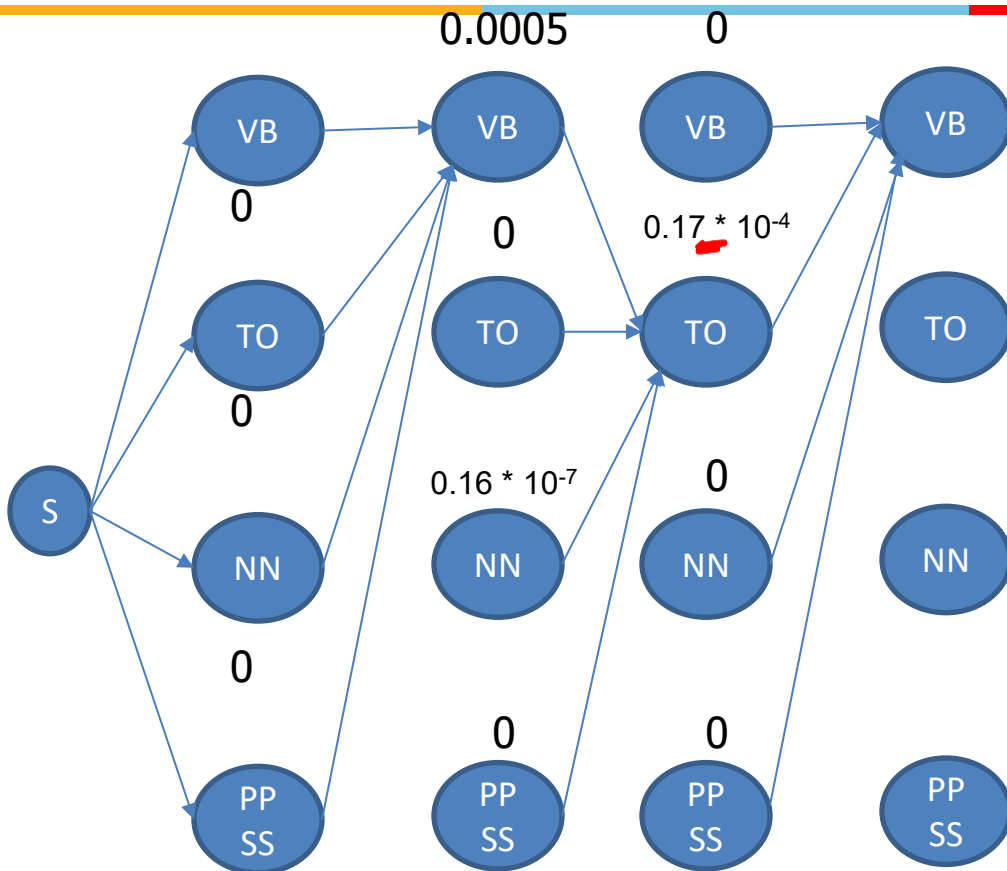
$$P(\text{NN}) * P(\text{TO} | \text{NN}) * P(\text{to} | \text{TO}) = 0.16 * 10^{-7} * 0.016 * 0.99 = 0.13 * 10^{-10}$$

$$P(\text{PPSS}) * P(\text{TO} | \text{PPSS}) * P(\text{to} | \text{TO}) = 0 * 0.00079 * 0.99 = 0$$

Hidden Markov Model



POS Tagging – Example -2 – Viterbi Algorithm - Recursion



0.0248

$$P(\text{VB}) * P(\text{VB} | \text{VB}) * P(\text{race} | \text{VB}) = 0 * 0.0038 * 0.00012 = 0$$

$$P(\text{TO}) * P(\text{VB} | \text{TO}) * P(\text{race} | \text{VB}) = 0.17 * 10^{-4} * 0.83 * 0.00012$$

$$= 0.17 * 10^{-8}$$

$$P(\text{NN}) * P(\text{VB} | \text{NN}) * P(\text{race} | \text{VB}) = 0 * 0.0040 * 0.00012 = 0$$

$$P(\text{PPSS}) * P(\text{VB} | \text{PPSS}) * P(\text{race} | \text{VB}) = 0 * 0.23 * 0.00012 = 0$$

	VB	TO	NN	PPSS
<S>	0.019	0.0043	0.041	0.067

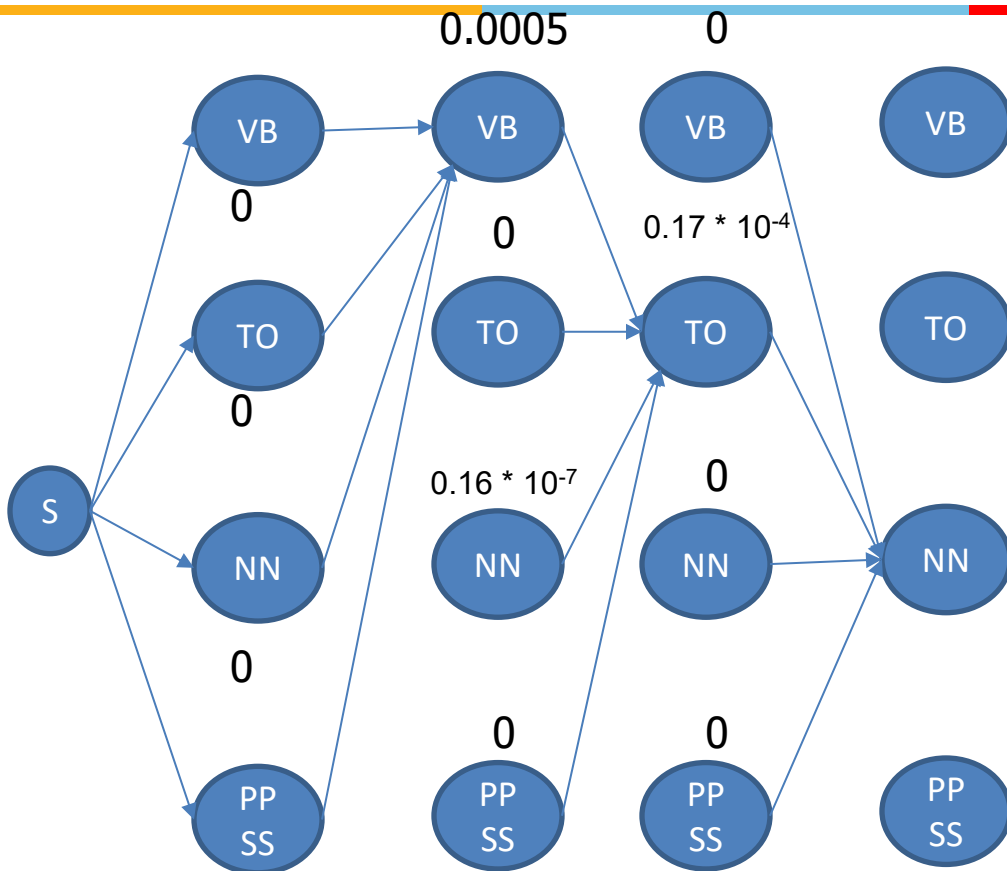
A	VB	TO	NN	PPSS
VB	0.0038	0.035	0.047	0.0070
TO	0.83	0	0.00047	0
NN	0.0040	0.016	0.087	0.0045
PPS	0.23	0.00079	0.0012	0.00014

B	I	want	to	race
VB	0	0.0093	0	0.00012
TO	0	0	0.99	0
NN	0	0.000054	0	0.00057
PPS	0.37	0	0	0
S				

Hidden Markov Model



POS Tagging – Example -2 – Viterbi Algorithm - Recursion



0.0248

$$P(VB) * P(NN | VB) * P(race | NN) = 0 * 0.047 * 0.00057$$

$$P(TO) * P(NN | TO) * P(race | NN) = 0.17 * 10^{-4} * 0.00047 * 0.00057$$

$$= 0.46 * 10^{-11}$$

$$P(NN) * P(NN | NN) * P(race | NN) = 0 * 0.087 * 0.00057 = 0$$

$$P(PPSS) * P(NN | PPSS) * P(race | NN) = 0 * 0.0012 * 0.00057 = 0$$

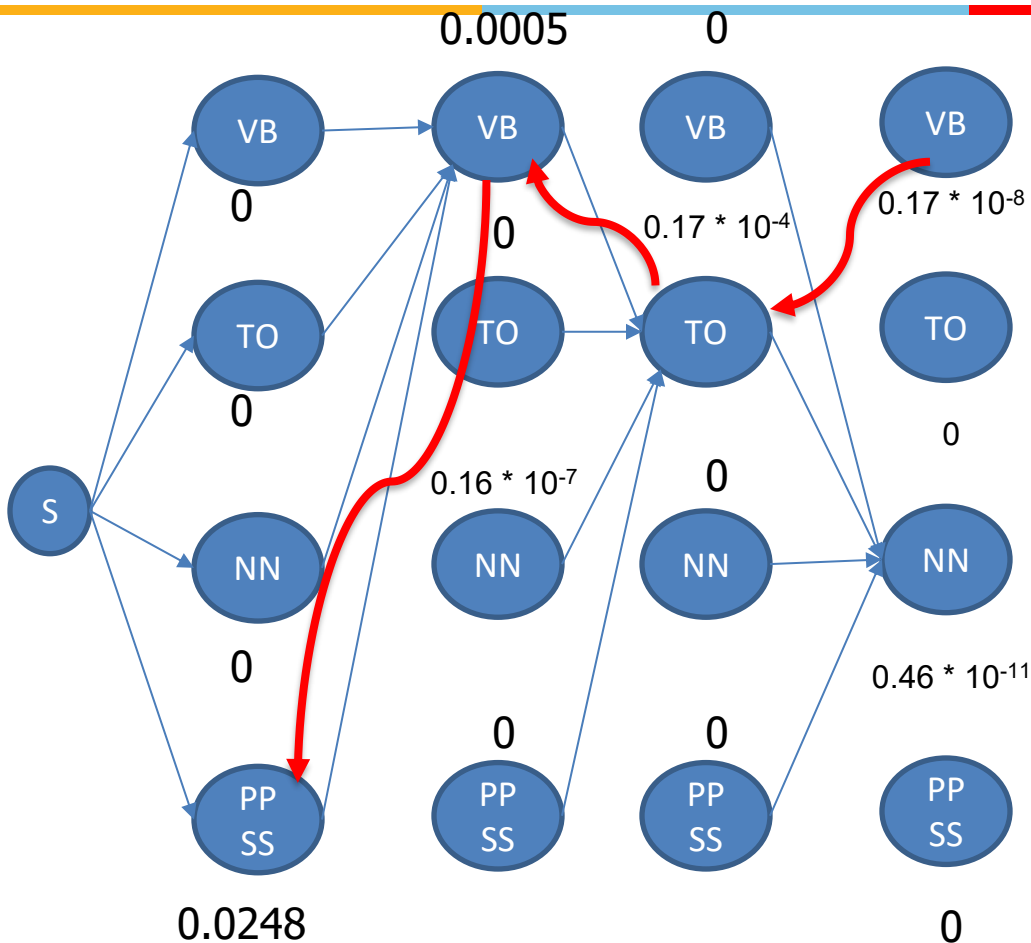
$$V[t, j] = \max_{t'} V[t', j - 1] P(t|t') P(w_j|t)$$

	VB	TO	NN	PPSS
<S>	0.019	0.004 3	0.041	0.067
A	VB	TO	NN	PPSS
VB	0.003 8	0.035	0.047	0.0070
TO	0.83	0	0.0004 7	0
NN	0.004 0	0.016	0.087	0.0045
PPSS	0.231	0.0007 9	0.0012	0.00014
S	VB	0	0.0093	0
	TO	0	0	0.99
	NN	0	0.00005 4	0
	PPS	0.37	0	0
	S			

Hidden Markov Model



POS Tagging – Example -2 – Viterbi Algorithm - Termination



$$V[t, j] = \max_{t'} V[t', j - 1] P(t|t') P(w_j|t)$$

	VB	TO	NN	PPSS
<S>	0.019	0.004 3	0.041	0.067
A	VB	TO	NN	PPSS
VB	0.003 8	0.035	0.047	0.0070
TO	0.83	0	0.0004 7	0
NN	0.004 0	0.016	0.087	0.0045
PPSS	0.231	0.0007 9	0.0012	0.00014
S	VB	TO	NN	PPSS
VB	0	0.0093	0	0.00012
TO	0	0	0.99	0
NN	0	0.00005 4	0	0.00057
PPSS	0.37	0	0	0
S				

Hidden Markov Model



POS Tagging – Example -3 – Viterbi Algorithm – Practice

Transition matrix: $P(t_i|t_{i-1})$

	NOUN	Verb	Det	Prep	ADV	STOP
<S>	.3	.1	.3	.2	.1	0
Noun	.2	.4	.01	.3	.04	.05
Verb	.3	.05	.3	.2	.1	.05
Det	.9	.01	.01	.01	.07	0
Prep	.4	.05	.4	.1	.05	0
Adv	.1	.5	.1	.1	.1	.1

Emission matrix: $P(w_i|t_i)$

	a	cat	doctor	in	is	the	very
Noun	0	.5	.4	0	0.1	0	0
Verb	0	0	.1	0	.9	0	0
Det	.3	0	0	0	0	.7	0
Prep	0	0	0	1.0	0	0	0
Adv	0	0	0	.1	0	0	.9

$$V[t, 1] = P(t|start)P(w_1|t)$$

	w1= the	w2=docto r	w3=i s	w4=in	STOP
Noun	0				
Verb	0				
Det	.21				
Prep	0				
Adv	0				

$$V(\text{Noun}, \text{the}) = P(\text{Noun}|\text{<S>})P(\text{the}|\text{Noun}) = .3 \times 0 = 0$$

$$V(\text{Verb}, \text{the}) = P(\text{Verb}|\text{<S>})P(\text{the}|\text{Verb}) = .1 \times 0 = 0$$

$$V(\text{Det}, \text{the}) = P(\text{Det}|\text{<S>})P(\text{the}|\text{Det}) = .3 \times .7 = .21$$

$$V(\text{Prep}, \text{the}) = P(\text{Prep}|\text{<S>})P(\text{the}|\text{Prep}) = .2 \times .0 = 0$$

$$V(\text{Adv}, \text{the}) = P(\text{Adv}|\text{<S>})P(\text{the}|\text{Adv}) = .2 \times .0 = 0$$

Hidden Markov Model



POS Tagging – Example -3 – Viterbi Algorithm – Practice

$$\begin{aligned}
 V(\text{Noun}, \text{doctor}) &= \max_{t'} V(t', \text{the})XP(\text{Noun}|t')X P(\text{doctor}|\text{Noun}) \\
 &= \max \{0, 0, .21 (.9 \times .4), 0, 0\} = .0756
 \end{aligned}$$

$$\begin{aligned}
 V(\text{Verb}, \text{doctor}) &= \max_{t'} V(t', \text{the})XP(\text{Verb}|t')X P(\text{doctor}|\text{Verb}) \\
 &= \max \{0, 0, .21(.01 \times .1), 0, 0\} = .00021
 \end{aligned}$$

	w1= the	w2=docto r	w3=i s	w4=in	STOP
Noun	0	.0756			
Verb	0	.00021			
Det	.21	0			
Prep	0	0			
Adv	0	0			

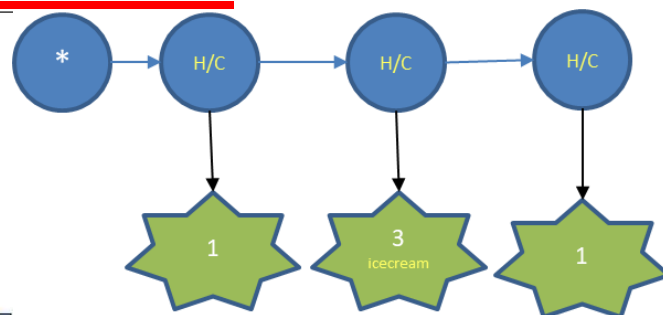
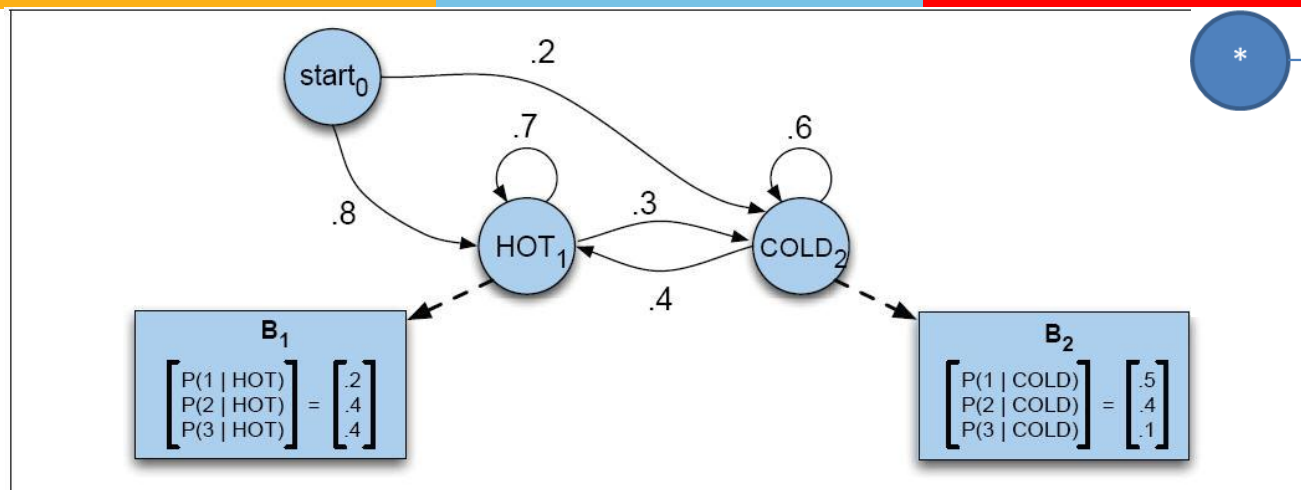
	w1= the	w2=docto r	w3=is	w4=in	STOP
Noun	0	.0756	.00151 2	0	
Verb	0	.00021	.02721 6	0	.000027 2
Det	.21	0	0	0	
Prep	0	0	0	0	
Adv	0	0	0	0	

Det Noun Verb Prep

Hidden Markov Model



Example -4 – Naïve Search



HHH	$P(H) * P(1 H) * P(H H) * P(3 H) * P(H H) * P(1 H)$	
HHC		
HCC		
CCC		
CHC	$P(C) * P(1 C) * P(H C) * P(3 H) * P(C H) * P(1 C)$ $= 0.2 * 0.5 * 0.4 * 0.4 * 0.3 * 0.5$	0.0024
CCH		
CHH		
HCH		

	Hot	Cold
<S>	0.8	0.2

A	Hot	Cold
Hot	0.7	0.3
Cold	0.4	0.6

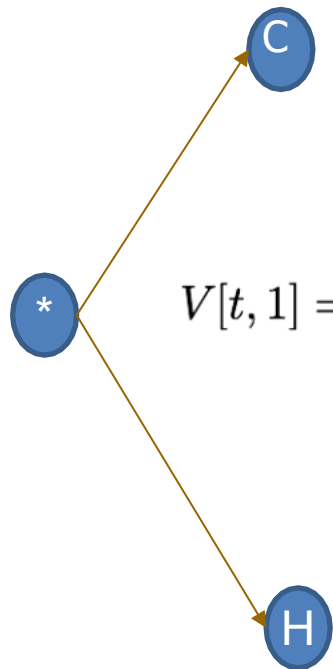
B	1	2	3
Hot	.2	.4	.4
Cold	.5	.4	.1

Hidden Markov Model

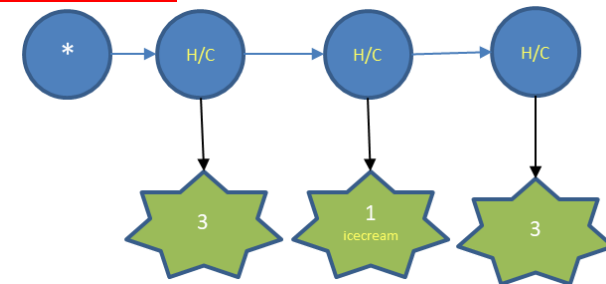


Example -5 – Observation Likelihood by Forward Propagation

$$P(C) * P(3|C) = 0.2 * 0.1 = 0.02$$



$$V[t, 1] = P(t|start)P(w_1|t)$$



Efficiently computes the probability of an observed sequence given a model

$$P(\text{sequence} | \text{model})$$

Identical to Viterbi; **replace the MAX with a SUM**

	Hot	Cold
<S	0.8	0.2
>		

A	Hot	Cold
Hot	0.6	0.4
Cold	0.5	0.5

$$P(H) * P(3|H) = 0.8 * 0.4 = 0.32$$

For our particular case, we would sum over the eight 3-event sequences *cold cold cold*, *cold, cold cold hot*, that is,

$$P(3 \ 1 \ 3) = P(3 \ 1 \ 3, \text{cold cold cold}) + P(3 \ 1 \ 3, \text{cold cold hot}) + P(3 \ 1 \ 3, \text{hot hot cold}) + \dots$$

B	1	2	3
Hot	.2	.4	.4
Cold	.5	.4	.1

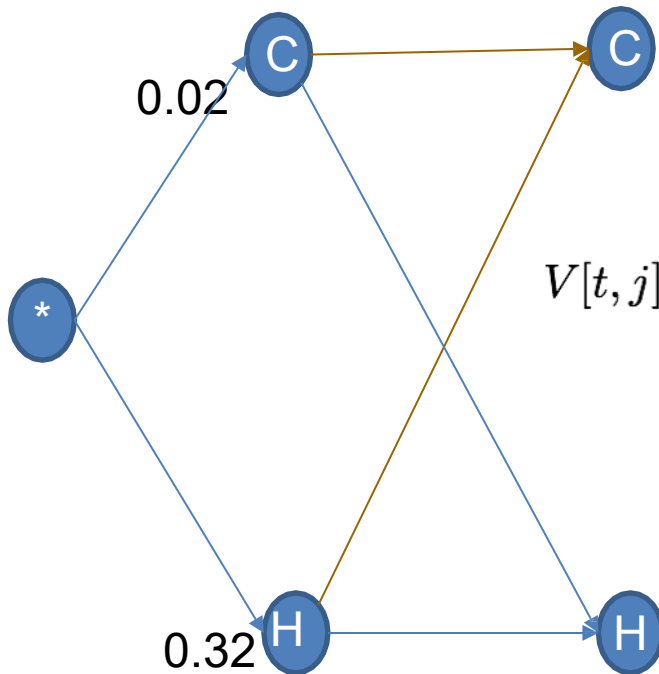
Hidden Markov Model



Example -5 – Observation Likelihood by Forward Propagation - Recursion

$$P(C)*P(C|C)*P(1|C) = 0.02*0.5*0.5 = 0.005$$

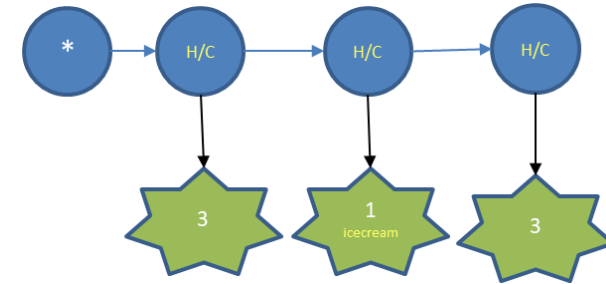
$$P(H)*P(C|H)*P(1|C) = 0.32*0.4*0.5 = 0.064$$



$$V[t, j] = \max_{t'} V[t', j - 1] P(t|t') P(w_j|t)$$

$$P(C)*P(H|C)*P(1|H) = 0.02*0.5*0.2 = 0.002$$

$$P(H)*P(H|H)*P(1|H) = 0.32*0.6*0.2 = 0.0384$$



	Hot	Cold	
<S	0.8	0.2	
>			
A	Hot	Cold	
Hot	0.6	0.4	
Cold	0.5	0.5	
B	1	2	3
Hot	.2	.4	.4
Cold	.5	.4	.1
d			

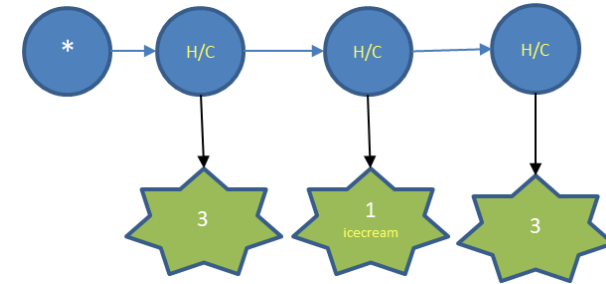
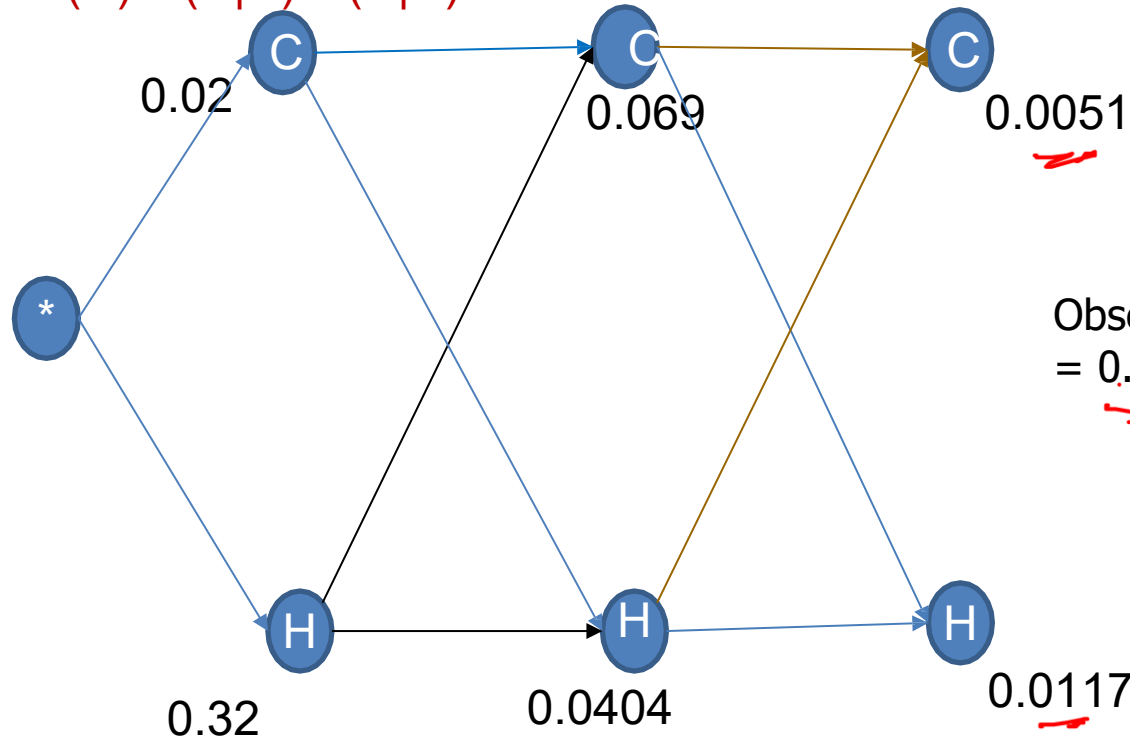
Hidden Markov Model



Example -5 – Observation Likelihood by Forward Propagation

$$P(C)*P(C|C)*P(R|C) = 0.069*0.5*0.1 = \mathbf{0.0035}$$

$$P(H)*P(C|H)*P(R|C) = 0.0404*0.4*0.1 = \mathbf{0.0016}$$



Observation Likelihood
= ~~0.0168~~

$$P(C)*P(H|C)*P(R|H) = 0.069 * 0.5 * 0.2 = \mathbf{0.0069}$$

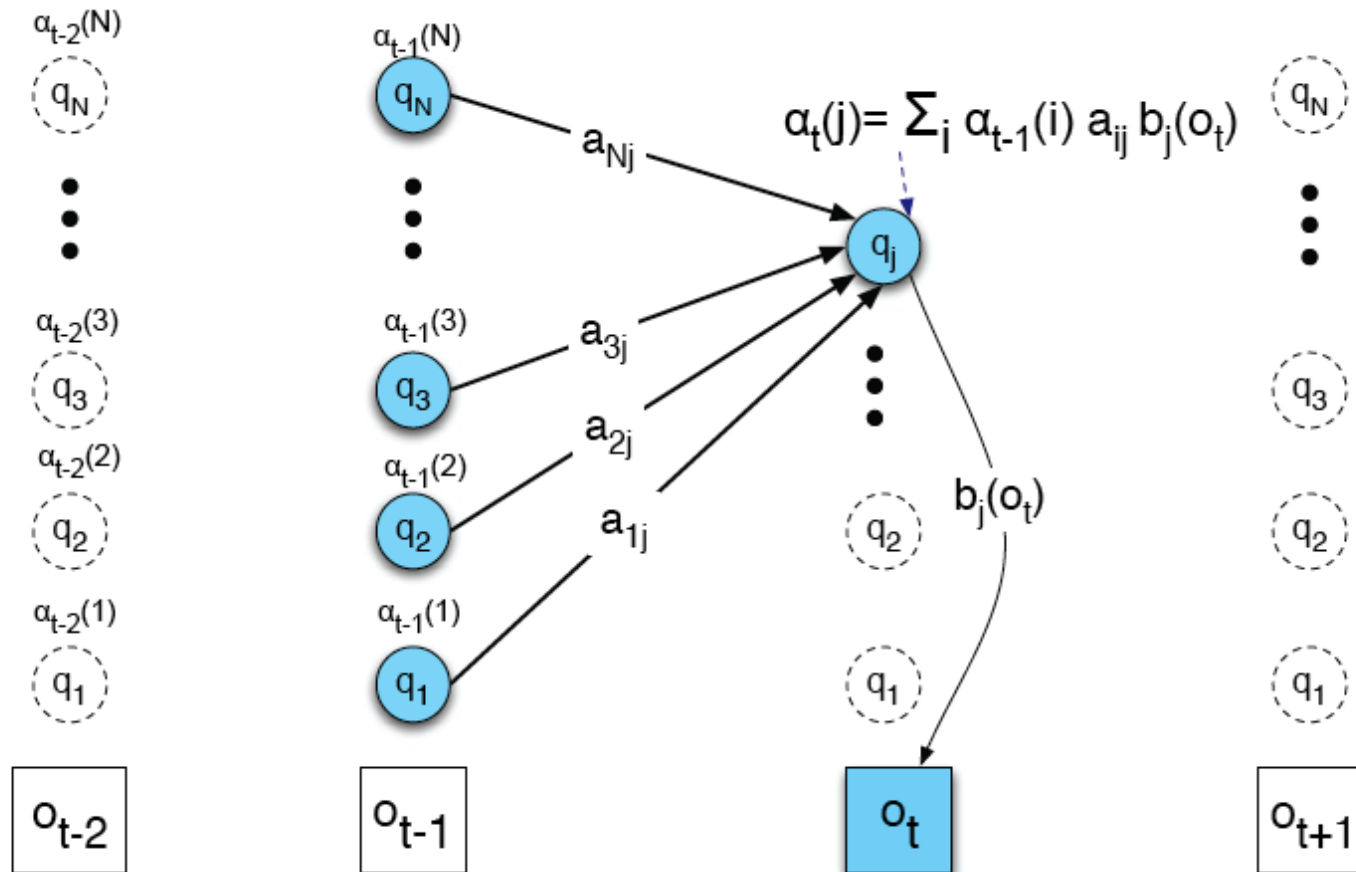
$$P(H)*P(H|H)*P(R|H) = 0.0404 * 0.6 * 0.2 = \mathbf{0.0048}$$

	Hot	Cold	
<S	0.8	0.2	
>			
A	Hot	Cold	
Hot	0.6	0.4	
Cold	0.5	0.5	
B	1	2	3
Hot	.2	.4	.4
Cold	.5	.4	.1
d			

Hidden Markov Model



Observation Likelihood – Forward Propagation



Source Credit : Speech and Language Processing - Jurafsky and Martin

Hidden Markov Model



Observation Likelihood – Forward Propagation

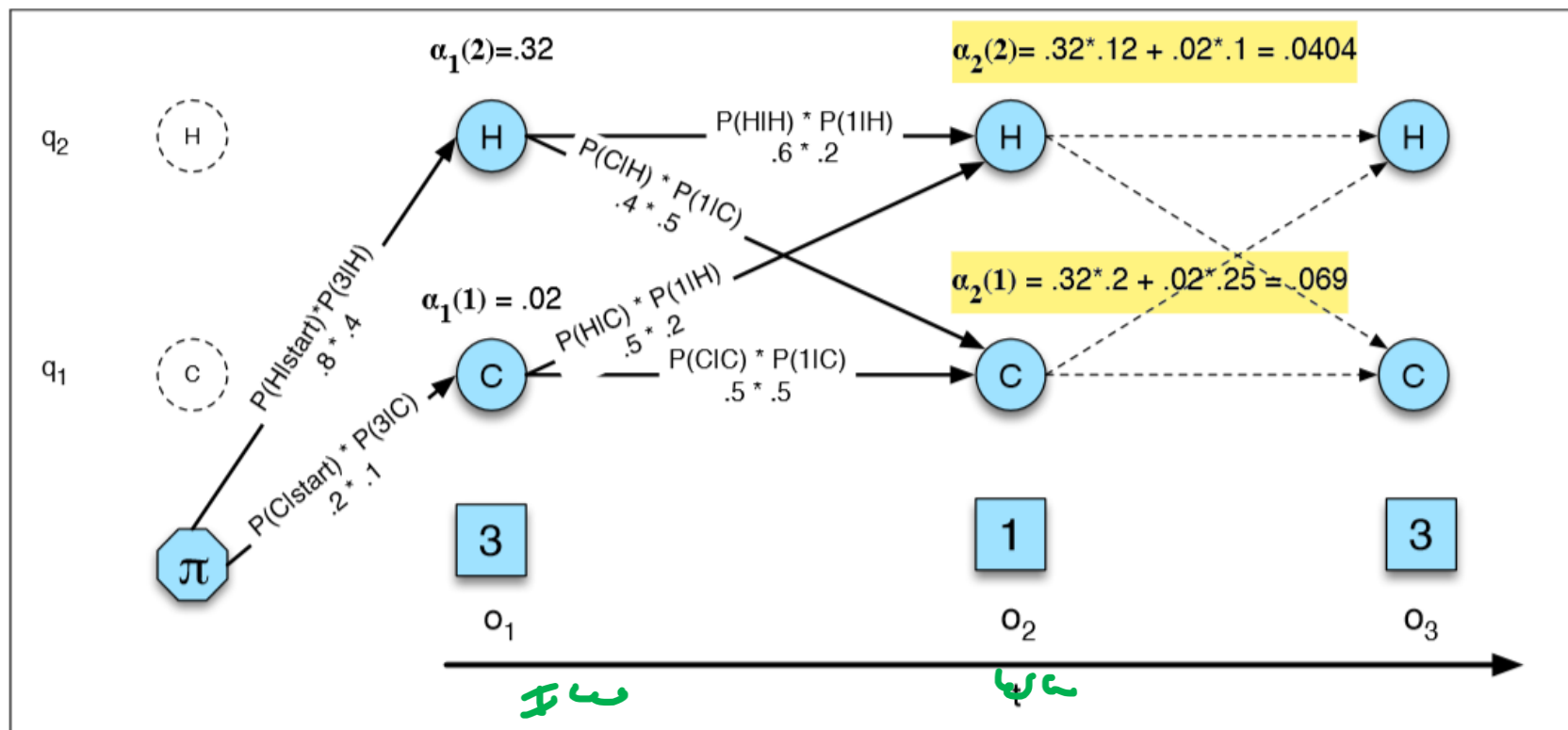


Figure A.5 The forward trellis for computing the total observation likelihood for the ice-cream events 3 1 3. Hidden states are in circles, observations in squares. The figure shows the computation of $\alpha_t(j)$ for two states at two time steps. The computation in each cell follows Eq. A.12: $\alpha_t(j) = \sum_{i=1}^N \alpha_{t-1}(i) a_{ij} b_j(o_t)$. The resulting probability expressed in each cell is Eq. A.11: $\alpha_t(j) = P(o_1, o_2 \dots o_t, q_t = j | \lambda)$.

Source Credit : Speech and Language Processing - Jurafsky and Martin

Forward-Backward

Learning: Given an observation sequence O and the set of possible states in the HMM, learn the HMM parameters A and B .

Is a special case of the EM or Expectation-Maximization algorithm

The algorithm will let us learn the transition probabilities $A = \{a_{ij}\}$ and the emission probabilities $B = \{b_i(o_t)\}$ of the HMM

- One problem with the HMM models as presented is that they are exclusively run left-to-right.
- Viterbi algorithm still allows present decisions to be influenced indirectly by future decisions,
- It would help even more if a decision about word w_i could directly use information about future tags t_{i+1} and t_{i+2} .

Bidirectionality

t1-1



- Any sequence model can be turned into a bidirectional model by using multiple passes.
- For example, the first pass would use only part-of-speech features from already-disambiguated words on the left. In the second pass, tags for all words, including those on the right, can be used.
- Alternately, the tagger can be run twice, once left-to-right and once right-to-left.
- In Viterbi decoding, the classifier chooses the higher scoring of the two sequences (left-to-right or right-to-left).
- Modern taggers are generally run bidirectionally.

Some limitation of HMMS



- Unknown words
- First order HMM

Eg:

Is clearly marked

He clearly marked



Natural Language Processing



BITS Pilani
Pilani Campus

Dr. Chetana Gavankar, Ph.D,
IIT Bombay-Monash University Australia
Chetana.gavankar@pilani.bits-pilani.ac.in



Session 7

These slides are prepared by the instructor, with grateful acknowledgement of Jurafsky and Martin and many others who made their course materials freely available online.

Session Content



- POS Tagging with Bi-LSTM-CRF
- Transformer model
- POS Tagging with Pre-trained Models (BERT, RoBERTa)
- Prompting techniques for POS Tagging
- LLM agents and POS Tagging

Bi-LSTM-CRF Architecture

The Go-To Neural Baseline for Sequence Tagging.

Concept: Combines a powerful feature extractor (Bi-LSTM) with a structured prediction layer (CRF) to ensure valid tag sequences.

Input Layer: Word Embeddings (e.g., Word2Vec, GloVe)
+ **Character Embeddings** (handles unknown words).

Bi-LSTM Layer (Feature Extractor):

- Processes the sentence in **forward and backward** directions.
- Captures **long-range dependencies** and **bidirectional context**.

CRF Layer (Structured Decoder):

- **Crucial for POS:** It learns constraints (e.g., a **Verb** is unlikely to follow a **Determiner**).
- Finds the sequence of tags that has the **highest overall probability** across the entire sentence.

Bi-LSTM-CRF



- LSTM: Solves the Vanishing Gradient problem of simple RNNs using Gates (Input, Forget, Output).
- Bidirectional Advantage: The tag for 'bank' in "I walked to the river bank." depends on both 'river' (left context) and '.' (right context).
- CRF Output: Instead of just picking the highest probability tag at each word (which can lead to impossible sequences),
- CRF layer calculates

$$P(\mathbf{y}|\mathbf{x}) = \frac{\prod_{i=1}^N \Psi_i(y_{i-1}, y_i, \mathbf{x})}{\sum_{\mathbf{y}' \in \mathcal{Y}(\mathbf{x})} \prod_{i=1}^N \Psi_i(y'_{i-1}, y'_i, \mathbf{x})}$$

- Goal: Maximize the numerator (score of the true path) relative to all possible paths.

Performance & Applications



- Typical Accuracy: approx. 97% - 98% on Penn Treebank.
- Pros: Highly accurate, automatically learns features, good for low-resource languages (with transfer learning).
- Cons: Slower training than non-recurrent models, replaced as SOTA by Transformers.

Transformer Model



- Key Idea: Replaces recurrence (RNN/LSTM) with the Self-Attention Mechanism.
- Self-Attention: Allows the model to weigh the relevance of every other word in the sentence simultaneously, regardless of distance.
- Example: When tagging "He," the model instantly links it to "ran" and "ball" equally well.
- Model Architecture: Utilizes multi-layer Encoder blocks consisting of Multi-Head Attention and Feed-Forward Networks.

POS Tagging with Pre-trained Models (BERT, RoBERTa)



Method: Fine-tuning a Pre-trained Language Model (PLM).

Pre-training: The model learns grammar and world knowledge on trillions of words (e.g., predicting masked words).

Fine-Tuning: A simple classification layer is added on top of the PLM's output vectors.

The entire model is then trained for a few epochs on the specific POS tagging corpus.

Output: The model produces a highly contextualized vector for each token, which the final layer maps to the appropriate tag (e.g., CLS --> NN).

Transformer Advantages & Impact



- SOTA Performance: approx. 98.5% - 99.5% accuracy.
- Efficiency: Highly parallelizable (faster training than RNNs).
- Context: Captures the deepest contextual meaning, solving nearly all lexical ambiguity issues.
- Impact: This approach is the standard for POS tagging in modern libraries like spaCy and Hugging Face Transformers.

Prompting



- Concept: Leverage the vast linguistic knowledge and zero-shot capabilities of Large Language Models (LLMs) (e.g., GPT-4, Llama).
- Traditional Fine-Tuning is Replaced by Prompting:
- Prompting: The POS task is framed as a natural language instruction.
- Example Prompt: "Tag the following sentence using the Penn Treebank tagset. Output only the word/tag pairs. Sentence: The concert was great."
- Zero-Shot/Few-Shot Learning: LLM can tag a sentence accurately without being explicitly fine-tuned on a POS corpus, based solely on its general pre-training.

LLM Agents & System Integration



LLM Agent: An LLM that is given the ability to use external tools to complete a task.

The POS Agent Flow:

Input: User gives a complex request (e.g., Analyze the sentiment of all nouns in the user reviews.)

LLM Agent: Breaks the task down.

Tool Use (POS Tagging): The LLM might decide to use a specialized, fast POS tagger (like a fine-tuned BERT model) as a tool for the labeling sub-task.

Integration: The agent feeds the tags back into its context to perform the final task (e.g., sentiment analysis on the words tagged as NN).

Benefit: Combines the LLM's reasoning/planning ability with the speed/accuracy of a specialized POS model.

Conclusion



- Modern POS Tagging is Solved?
- The task has achieved near-human performance thanks to Deep Learning.
- Current SOTA: Fine-tuned Transformer models (BERT/RoBERTa).
Future Trend (LLMs): POS tagging is becoming an internal utility within larger LLM agent systems.
- The focus shifts from how to build the best tagger to how to use tags to power complex downstream AI systems.

References



- <https://www.nltk.org/>
- <https://likegeeks.com/nlp-tutorial-using-python-nltk/>
- <https://www.guru99.com/pos-tagging-chunking-nltk.html>
- <https://medium.com/greyatom/learning-pos-tagging-chunking-in-nlp-85f7f811a8cb>
- <https://nlp.stanford.edu/software/tagger.shtml>
- <https://www.forbes.com/sites/mariyayao/2020/01/22/what-are--important-ai--machine-learning-trends-for-2020/#601ce9623239>
- <https://medium.com/fintechexplained/nlp-text-processing-in-data-science-projects-f083009d78fc>
- <https://www.nltk.org/book/ch02.html>
- <https://towardsdatascience.com/pos-tagging-using-crfs-ea430c5fb78b>

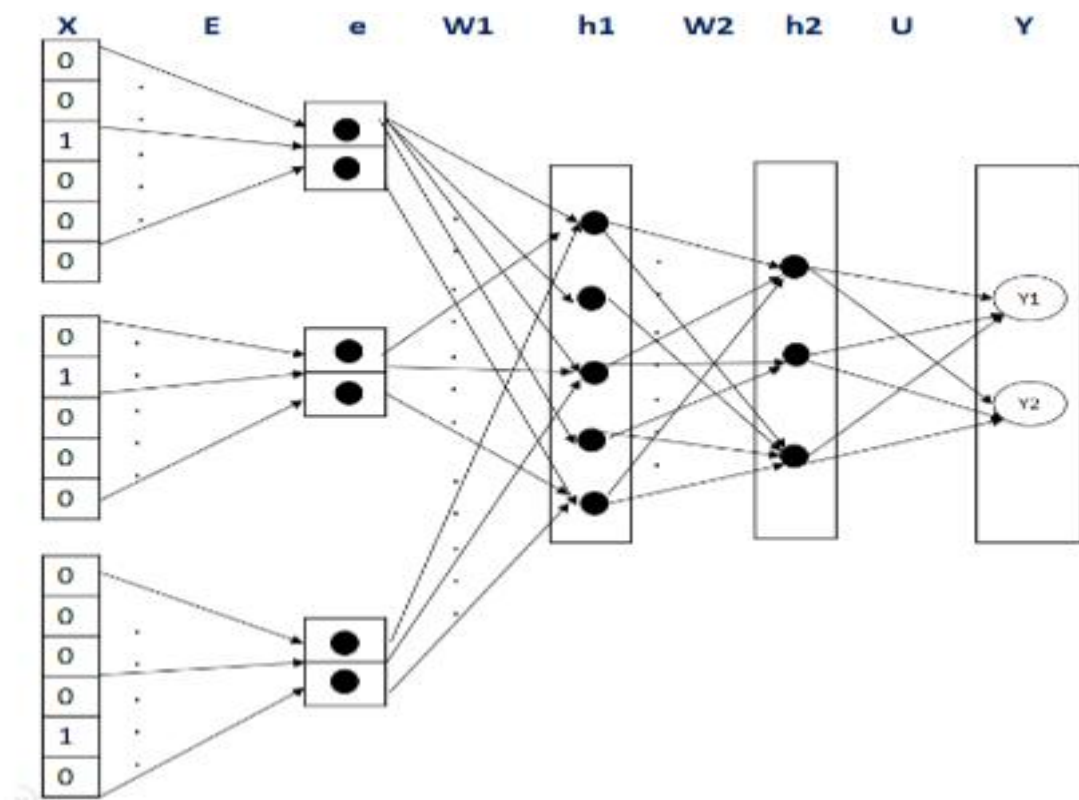
References

- <https://github.com/nadavo/MEMM-POS-Tagger>
- <https://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.352.9257&rep=rep1&type=pdf>
- https://www.alibabacloud.com/blog/hmm-memmm-and-crf-a-comparative-analysis-of-statistical-modeling-methods_592049
- <https://towardsdatascience.com/pos-tagging-using-rnn-7f08a522f849>

POS tagging in Indian Languages

- <https://www.nltk.org/book/ch05.html>

- Neural language models [3 Marks]
- Study the below neural network designed for learning word embedding along with other NLP applications and answer the following questions.



If the input layer 'X' denote the one hot encoding of the vocabulary, "e" is the embedding layer, "h1","h2" are hidden layers with sigmoid activation function and "Y" is the output layer with sigmoid activation function, identify no more than 2 issues/error in the architecture for each of the below two scenarios: If there are no corrections required then mention "No Error" explicitly.

a) Scenario 1: The input with context window of four tokens is used to predict the next fifth token. The candidate fifth word can be either $Y = "."$ Or $Y = "?"$ for this application.

b) Scenario 2: Top three frequent tokens with embedding of size three, is used to predict the sentiment from the set {Positive, Negative, Neutral} of the document from where the tokens were extracted.

c) With respect to the "scenario 1", find the dimension (matrix size) w.r.t to section/layer "W1" and "U"

Solution:

- a) No.of.one-hot-encoding vectors must be 4 but is three in number, Output needs only one neuron for Y-Pred, One-hot-encoding size is not consistent
- b) Layer “e” each embedding is of size 2 but it must have 3 neurons each, Output needs softmax activation and three neurons
- c) $W1: 5 \times 6$, $U: 1 \times 3$

2. “Erring is human. To err is human. Seeing is believing. To see is to believe.”

For the given above training data construct a neural architecture for language modelling for the following case. Mention the dimension at each and every layer and for the weights between layers.

Case: “Using context window of size 4, design a neural network to predict the next word with candidate vocabulary of size 10. Each vocabulary has a word embedding of dimension 3. Use two internal hidden layers. First one with 6 neurons and second with 4 neurons. Each internal hidden layers must be use sigmoid activation function. “

If the above needs to be changed to accommodate the net to learn the word embedding as well, do the necessary modification in above case study requirement and pictorially draw the final network (No need to draw all the connection between immediate layers).

Solution:

I/p layer 4 words with one hot encoding of size 10.

After above layer an embedding layer with 12 ($=4*3$) nodes followed by hidden layer with 6 neurons

Output layer with 10 nodes and softmax activation function

Input layer X'	Input- Embedding E	Embedding layer E	Embedding- Hidden-1 W1	Hidden Layer-1 H1	Hidden1- Hidden2 W2	Hidden Layer-2 H2	Hidden2- Output U	Output Layer y
10 X 4	4 X 10	12 X 1	6 X 12	6 X 1	4 X 6	4 X 1	10 X 4	10 X 1

- Given a training corpus: “I live in Bangalore, India”, use the skip-gram negative sampling method and answer the following: The initial embedding matrix has dimensions $|v| \times 3$ and is given as follows:

1	1	0
0	1	0
1	0	1
1	1	1
0	0	1

The initial context matrix has $|v| \times 3$ dimension and given as follows:

1	0	0
0	1	0
0	1	1
0	1	0
1	0	1

- Generate the training dataset for an input word ‘I’ and context window of 1 next word and hyper parameter value $k=3$ for the negative sampling task. Use the information available in the question.
- Calculate the error for the above dataset during the first iteration of skip-gram training.

Solution:

a) Training dataset (1 Marks)

I	live	1
I	in	0
I	Bangalore	0
I	india	0

b) (3 Marks)

context embeddings				i/p word embedd ing	dot product	exp()	sigmoid()	t	sig-t (error)
live	0	1	0	1	1	0.367879	0.731059	1	-0.26894
in	0	1	1	1	1	0.367879	0.731059	0	0.731059
Bangalore	0	1	0	0	1	0.367879	0.731059	0	0.731059
India	1	0	1		1	0.367879	0.731059	0	0.731059

- Given a training corpus: “played in the morning”, use the skip-gram negative sampling method and answer the following: The initial embedding matrix and initial context matrix has dimensions $|V| \times 3$ and is given as follows:
Note: No need to update or show any weights other than necessary for below questions. Follow only the approach as discussed in class. i.e., Simplified Skip gram negative sampling with binary classification model.
Round all the calculations to exactly two decimal places.
a) Generate the training dataset for an input target word “played” and context window of 1 next word and hyper parameter value $k=2$ for the negative sampling task. Use the information available in the question.
b) Calculate the error for the above dataset for only the first iteration of skip-gram training, with only one hidden layer.

Vocabulary	Initial Embedding Matrix		
<i>on</i>	1	0	0
<i>in</i>	0	1	1
<i>morning</i>	1	0	0
<i>he</i>	1	1	0
<i>was</i>	0	1	1
<i>played</i>	1	0	0
<i>with</i>	0	1	1
<i>the</i>	0	0	1
<i>bat</i>	1	0	0

Vocabulary	Initial Context Matrix		
<i>on</i>	0	1	0
<i>in</i>	0	0	1
<i>morning</i>	0	1	0
<i>he</i>	1	0	1
<i>was</i>	0	0	1
<i>played</i>	0	1	0
<i>with</i>	0	0	1
<i>the</i>	1	0	0
<i>bat</i>	1	1	0

Solution & Marking Scheme:

a) Training data: 1.5 mark

Target	Context	Label
played	in	1
played	the	0
played	morning	0

b) Error:

Dot product = 1 mark

Sigmoid = 1 mark

Error = 1 mark

	context embedding			i/p word embedding	dot product	sigmoid()	t	sig-t
in	0	0	1	1	0	0.5	1	-0.5
the	1	0	0	0	0	0.5	0	0.5
morning	0	1	0	0	1	0.73	0	0.73

c) 0.5 mark Faster/Ease of Computation.

1 mark: Instead of updating all weights, 1 iteration updates only the context word & target word's embedding for a training instance

Birla Institute of Technology & Science, Pilani
Work Integrated Learning Programmes Division
First Semester 2024-2025
M.Tech. in AIML

Mid-Semester Test
(EC-2 Regular Paper)

Course No. : AIMLCZG530
Course Title : Natural Language Processing
Nature of Exam : Closed Book
Weightage : 30%
Duration : 2 Hours
Date of Exam : 19-Jan-2024

No. of Pages	= 3
No. of Questions	= 7

Note to Students:

1. Please follow all the *Instructions to Candidates* given on the cover page of the answer book.
2. All parts of a question should be answered consecutively. Each answer should start from a fresh page.
3. Assumptions made if any, should be stated clearly at the beginning of your answer.

Question 1. [2+2=4 Marks] Introduction

- a) What type of ambiguity exists in the sentences below:
"I want to read a book" and "I want to book a flight"
Which NLP technique will help to resolve the ambiguity?

Solution:

Syntactic or grammatical ambiguity. [1 mark]

Can be resolved by doing part of speech tagging for word "book" in both sentences and identifying them as noun in first and verb in second sentence.[1 mark]

- b) Name the different preprocessing steps in NLP in the below table for the given sentence
"Natural Language Processing is interesting and one of the best courses!!"

Output after processing of sentence	pre-processing step
i. [Natural Language Processing is interesting and one of the best courses]	Removal of punctuation
ii. [Natural, Language, Processing, is, interesting, and, one, of, the, best, courses]	???
iii. ['Natural', 'Language', 'Processing', 'interesting', 'one', 'best', 'courses']	???
iv. ['Natural', 'Language', 'Processing', 'is', 'interesting', 'and', 'one', 'of', 'the', 'best']	???
v. ['Natur', 'language', 'process', 'is, interest, and, one, of, the, best]	???

SOLUTION

0.5 marks each

Output after processing of sentence	pre-processing step
-------------------------------------	---------------------

i.	[Natural Language Processing is interesting and one of the best courses]	Removal of punctuation
ii.	[Natural, Language, Processing, is, interesting, and, one, of, the, best, courses]	Word Tokenization
iii.	['Natural', 'Language', 'Processing', 'interesting', 'one', 'best', 'courses']	Removal of stop words
iv.	['Natural', 'Language', 'Processing', 'is', 'interesting', 'and', 'one', 'of', 'the', 'best']	Lemmatization
v.	['Natur', 'language', 'process', 'is, interest, and, one, of, the, best']	Stemming

Question 2. [4 Marks]

A simple language model needs to choose between two candidate translations:

"The weather is cold"

"The weather are cold"

Using the following bigram probabilities:

$P(\text{weather}|\text{the}) = 0.2$

$P(\text{is}|\text{weather}) = 0.5$

$P(\text{are}|\text{weather}) = 0.1$

$P(\text{cold}|\text{is}) = 0.3$

$P(\text{cold}|\text{are}) = 0.3$

- Calculate the bigram probability for each sentence [2 marks]
- Which translation should be selected and why? [2 marks]

Solution:

a) Sentence 1: $P(\text{the weather is cold}) = P(\text{weather}|\text{the}) \times P(\text{is}|\text{weather}) \times P(\text{cold}|\text{is})$
 $= 0.2 \times 0.5 \times 0.3 = 0.03$ [1 mark]

Sentence 2: $P(\text{the weather are cold}) = P(\text{weather}|\text{the}) \times P(\text{are}|\text{weather}) \times P(\text{cold}|\text{are})$
 $= 0.2 \times 0.1 \times 0.3 = 0.006$ [1 mark]

b) The model should select "The weather is cold" because:

It has a higher probability ($0.03 > 0.006$)

This indicates it's more likely to be grammatically correct

The probability is 5 times higher than the second candidate [2 marks]

Question 3. [5 Marks] neural language models

- Consider following neural network which has 3 outputs and 5 inputs [3+2 marks]

Output 1: Creative: Yes: 1, Not sure:0, No: -1

Output 2: Caring: Yes: 1, Not sure:0, No: -1

Output 3: Rigid: Yes: 1, Not sure:0, No: -1

Inputs:

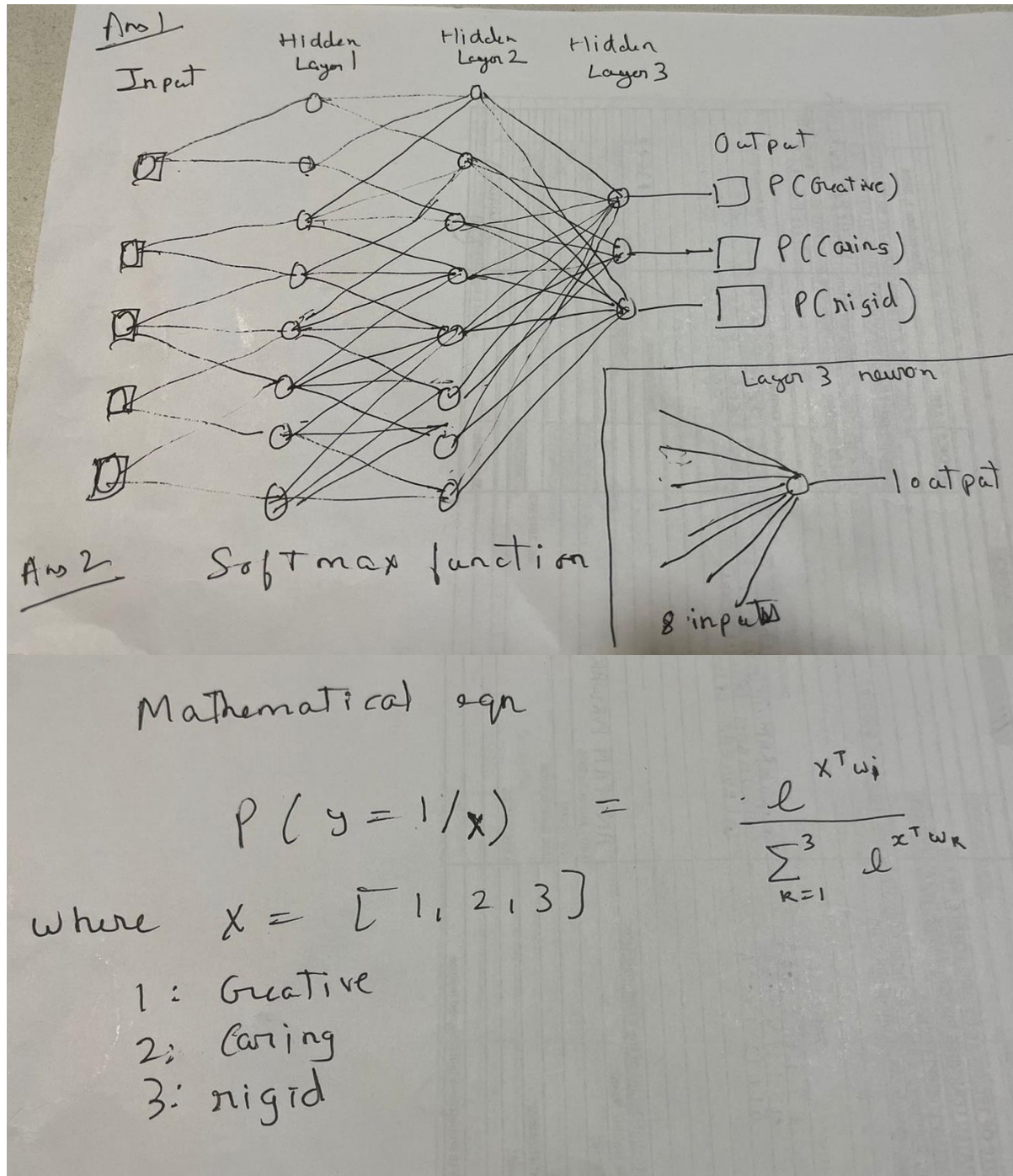
- Conscientiousness: number b/w 1-5
- Agreeableness: number b/w 1-5
- Openness: number b/w 1-5
- Neuroticism: number b/w 1-5
- Extraversion: number b/w 1-5

Design a neural network with the above inputs. It shall have 3 hidden layers: layer 1, layer 2 with 8 neurons and layer 3 (classification layer).

All layers to be fully connected

- 1.1 Draw above network, make suitable assumptions about different layers [3 marks]
- 1.2 Suggest the activation function to be used for layer 3, draw one neuron from this layer and mark all the inputs/outputs from it along with the equation of mathematical operation being performed by this neuron, choose suitable function for neuron of this layer [2 marks]

Solution:



Question 4. [4 Marks]

Assume that following is a portion of a vocabulary based on a corpus of text:

Vocabulary = ['international', 'India', 'Islamabad', 'Lanka', 'Delhi', 'Dhaka', 'neighbors', ..., ...]

Based on some scheme like word2vec, the portion of the word embedding matrix M corresponding to the above vocabulary is given below:

M

1	8	6
3	5	7
2	4	6
4	6	8
1	2	4
6	6	6
6	5	2
...	...	...

(a) Explain the step(s) you will take to extract the embedding vector for 'Lanka', and also write the result: [1 mark]

(b) Derive embedding vectors for between 'Islamabad' and 'Dhaka'? How will you calculate the semantic distance between 'Islamabad' and 'Dhaka'? [2 mark]

(c) Using the word embedding vectors, arrive at embedding vector for the sentence: 'Lanka neighbors India' [1 mark]

Solution

4(a) Explain the step(s) you will take to extract the embedding vector for 'Lanka', and write the result: [1 mark]

The following steps are involved in extracting the embedding vector for any word from the word embedding matrix:

- Derive the one-hot-encoding vector for the target word based on the given vocabulary. For 'Lanka' the one-hot-encoding vector is **[0 0 0 1 0 0 0 ...]**
- Take a dot product of this vector with the embedding matrix M the resulting vector is the embedding vector for 'Lanka'

[0 0 0 1 0 0 0 ...] •

1	8	6
3	5	7
2	4	6
4	6	8
1	2	4
6	6	6
6	5	2
...	...	...

Answer: The embedding vector for 'Lanka' = [4 6 8]

Note: Give 0.5 marks if the steps are adequately explained and 0.5 marks for the correct result.

4(b) How will you calculate the semantic distance between 'Islamabad' and 'Dhaka'? [2 mark]

The 'Cosine Distance' or 'Euclidean Distance' can both be considered as measures of Semantic Distance between two words. Therefore, **any of the following solutions can be accepted.**

'Islamabad' = [2 4 6]

'Dhaka' = [6 6 6]

Based on Cosine Distance

Cosine Distance = $1 - \text{cosine_similarity}$

Cosine similarity = $[2\ 4\ 6] \cdot [6\ 6\ 6] / ([2\ 4\ 6] \times [6\ 6\ 6]) = (12+24+36)/(7.483 \times 10.392) = 0.926$

Hence Cosine Distance = $1 - 0.926 = 0.074$

Based on Euclidean Distance

Semantic distance = $\text{sqrt}((2-6)^2 + (4-6)^2 + (6-6)^2) = \text{sqrt}(16 + 4 + 0) = 4.472$

Note: Give 0.5 marks if the steps are correct, even if there are calculation errors

4(c) Using the word embedding vectors, arrive at an embedding vector for the statement ‘Lanka neighbours India’ [1mark]

The embedding vector for a sentence can be **aggregated** from the embedding vectors of each of the words in the sentence. For the statement ‘Lanka neighbours India’, the embedding vector can be obtained by either averaging all the constituent embedding word vectors, or by even just summing them up. Either of the following solutions is correct.

‘Lanka’ = [4 6 8]

‘neighbours’ = [6 5 2]

‘India’ = [3 5 7]

Solution 1:

Based on averaging the word vectors: $[(4+6+3)/3, (6+5+5)/3, (8+2+7)/3] = [4.333, 5.333, 5.667]$

Solution 2:

Based on summing up the word vectors: $[(4+6+3), (6+5+5), (8+2+7)] = [13, 16, 17]$

Note: Give 0.5 marks if the steps are correct, even if there are calculation errors

Question 5. [5 Marks]

Consider the following word vector embedding’s:

- i. **Cat** [0.1 0.1 0.9]
- ii. **Dog** [0.7 0.3 0.0]
- iii. **Meow** [0.0 0.0 1.0]
- iv. **Tree** [0.1 0.9 0.1]
- v. **Forest** [0.0 0.8 0.2]

Answer the below: (All answers must be accompanied with detailed steps)

1. Find the word embedding vector representation of the word “Bark” corresponding to the words “Cat”, “Meow” and “Dog”. **[1 mark]**
2. Given the sentence – “The **dog began to bark at the tree** atop which the cat meowed”

Train a classifier such that, given the tuple ("bark", "dog") where "bark" is the target word and "dog" is the candidate context word, the classifier returns the probability that "dog" is a real context word for "bark".

- Provide the updated Input weight matrix for the Target Word after one iteration of the Word2Vec algorithm

Support your answer with detailed steps and rationale on the logic and computation.

[4 marks]

The following additional information are provided:

Use Word2Vec with Skip Gram Classifier with a Single Hidden Layer

Negative Sampling word is “Forest”

Activation Function is Sigmoid

Learning Rate = 0.01

Precision of computations to be 3 decimal places

The One Hot Encoded Input Vectors are:

Bark	[1 0 0]
Dog	[0 1 0]
Forest	[0 0 1]

Initial Embedding Matrix for the Single Hidden Layer

Bark	0.1	0	0.1
Dog	0	0.1	0
Forest	0.1	0	0.1

Initial Embedding Matrix for the Output Layer

Bark	0	0.1	0
Dog	0.1	0	0.1
Forest	0	0.1	0

Solution:

1. Using Parallelogram method: [1 marks]

$$\begin{aligned}
 \text{Bark} &= \text{Meow} - \text{Cat} + \text{Dog} \\
 &= [0.0 \ 0.0 \ 1.0] - [0.10.1 \ 0.9] + [0.7 \ 0.3 \ 0.0] \\
 &= [0.6 \ 0.2 \ 0.1]
 \end{aligned}$$

2.

Step 1 – Forward Propagation (Hidden Layer) [0.5 mark]

- The One Hot Encoded Input Matrix: (I)

$$\begin{bmatrix} 100 \\ 010 \\ 001 \end{bmatrix}$$
- Initial Embedding Matrix (W_{input})

$$\begin{bmatrix} 0.1 & 0.0 & 0.1 \\ 0.0 & 0.1 & 0.0 \\ 0.1 & 0.0 & 0.1 \end{bmatrix}$$
- Hidden Layer (h) for Target word “bark” = $W_{\text{input}}^T * I$

$$= \begin{bmatrix} 0.1 \\ 0.0 \\ 0.1 \end{bmatrix}$$

Step 2 – Forward Propagation (Sigmoid Output Layer) [1 mark]

- W_{output} (context) for (dog,forest)

$$\begin{bmatrix} 0.0 & 0.1 & 0.0 \\ 0.1 & 0.0 & 0.1 \\ 0.0 & 0.1 & 0.0 \end{bmatrix}$$

$$\text{Output Layer} = W_{\text{output}} * h$$

$$= \begin{bmatrix} 0.02 \\ 0 \end{bmatrix}$$

Applying Sigmoid Activation,

$$\text{For Positive Samples: } \sigma(x) = \frac{1}{1+e^{-x}}$$

$$\text{For Negative Samples: } \sigma(x) = \frac{1}{1+e^x}$$

$$\sigma(\text{output layer}) = \begin{bmatrix} 0.505 \\ 0.500 \end{bmatrix}$$

Step 3 – Prediction Error [0.5 mark]

$$\begin{aligned} \text{Prediction Error} &= \sigma(\text{output layer}) - \text{1-hot encoded vector for context} \\ &= \begin{bmatrix} 0.505 \\ 0.500 \end{bmatrix} - \begin{bmatrix} 1 & & \\ & 0 & \end{bmatrix} = \begin{bmatrix} -0.495 \\ 0.500 \end{bmatrix} \end{aligned}$$

Step 4 - Backward Propagation (computing ∇W_{input}) step: [1 marks]

Derivative of Loss with respect to Input Word Embeddings for the target word “bank”:

$$\begin{bmatrix} -0.495 \\ 0.5 \end{bmatrix}^{\text{Transpose}} \times \begin{bmatrix} 0.1 & 0 & 0.1 \\ 0 & 0.1 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} -0.0495 & 0.05 & -0.0495 \end{bmatrix} \quad \{ \nabla W_{\text{input}} \}$$

Step 5 - Updated Weight Matrix by applying Learning Rate [1 mark]

Learning Rate = 0.01 (given)

$$W_{\text{input}}^{\text{new}} = \begin{bmatrix} 0.1 & 0.0 & 0.1 \end{bmatrix} - 0.01 * \nabla W = \begin{bmatrix} 0.100 & -0.001 & 0.100 \end{bmatrix}$$

This the updated weight matrix for the Target Word after one iteration of the Word2Vec algorithm

Question 6.

Using an HMM tagger to disambiguate the POS tag for the word "bank" in the following sentence, given the transition and emission probabilities below:

Went to the bank.

Emission probabilities

	Bank	Went	To	The
VB	0.5	1	0	0
TO	0	0	1	0
NN	0.5	0	0	0
DET	0	0	0	1

Transition probabilities

	VB	TO	NN	Det
VB	0.4	0.1	0.2	0.3
TO	0.2	0.7	0.1	0
NN	0.3	0.2	0.4	0.1
Det	0.1	0	0.2	0.7

Use the provided transition and emission probabilities to disambiguate the POS tag for the word "bank" in the sentence.

Solution:

Went to the bank

Possible taggings are:

- i. VB → TO → Det → NN
- ii. VB → TO → Det → VB

We are interested in disambiguating “bank” in the above phrase. Hence the computations will be:

$$P(\text{NN}|\text{Det}) = 0.2$$

$$P(\text{VB}|\text{Det}) = 0.1$$

$$P(\text{bank} | \text{NN}) = 0.5$$

$$P(\text{bank} | \text{VB}) = 0.5$$

$$\text{For tagging (i): } P(\text{NN}|\text{Det}) \times P(\text{bank}|\text{NN}) = 0.2 \times 0.5 = 0.1$$

$$\text{For tagging (ii): } P(\text{VB}|\text{Det}) \times P(\text{bank}|\text{VB}) = 0.1 \times 0.5 = 0.05$$

Hence, tagging (i) is the preferred POS tag, i.e.

VB → TO → Det → NN

Question 7.

Fill up the Viterbi table for the sentence – “She dances”. The tag transition probabilities and word emission probabilities, for the corpus used, are given below:

Tag Transition Probabilities:

	VB	MD	PRP
VB	0.1	0.2	0.05
MD	0.5	0.3	0.01
PRP	0.4	0.4	0.9
START	0.1	0.2	0.7

Word Emission Probabilities:

	She	dances
VB	0	0.3
MD	0	0
PRP	1	0

Viterbi Table:

	She	dances
VB		
MD		
PRP		

Tags:

- **VB:** Verb Base Form
- **MD:** Modal
- **PRP:** Personal Pronoun

You need to fill out the Viterbi table for the sentence "She dances" using the given transition and emission probabilities.

Solution:

	She	dances
VB	0	0.084
MD	0	0
PRP	0.7	0

